Université / École d'Ingénieur

Département Informatique / Génie Logiciel

Projet de Fin d'Études

Rapport de Stage

Conception et Implémentation d'un Pipeline de Prétraitement Intelligent pour la Base de Connaissances STM32Cube

---

Réalisé par :
Abir GHARBI

Encadrant entreprise :
[Nom du tuteur ST]

Encadrant académique :
[Nom du tuteur académique]

Entreprise d'accueil : STMicroelectronics
Division : AI & Digital Tools

Année universitaire : 2025–2026

Remerciements

Je tiens à exprimer ma sincère gratitude envers toute l'équipe STMicroelectronics pour leur accueil, leur encadrement et leur soutien tout au long de ce stage. Je remercie particulièrement [nom du tuteur] pour sa disponibilité et ses conseils avisés.

Table des matières

Remerciements **1**

Introduction Générale **10**

1 Contexte Général du Projet **12**

1.1 Présentation de l'Entreprise 12

1.1.1 Présentation Générale 12

1.1.2 Vision et Marchés 13

1.1.3 STMicroelectronics Tunis 13

1.2 Division Microcontrôleurs (MCD) 13

1.3 Équipe d'Accueil : Technical Marketing and Application Support 14

1.4 L'Écosystème STM32 et les Dépôts GitHub 15

1.4.1 Les Dépôts GitHub STMicroelectronics 16

1.5 Problématique 16

1.5.1 Critique de l'Existant 17

1.6 Objectifs du Projet 18

1.7 Contributions Techniques Originales 18

1.8 Enjeux Éthiques et Responsabilité 19

1.9 Les 25 Séries STM32 Couvertes 20

1.10 Plan du Rapport 21

2 État de l'Art et Fondements Théoriques **22**

2.1 Le Paradigme RAG 22

2.2 Preprocessing et Qualité de la Base de Connaissances 23

2.3 TF-IDF pour la Similarité 23

2.4 Alfred Vision pour la Description d'Images 24

2.5 Étude Comparative des Solutions Existantes 24

2.6 Synthèse de la Solution Proposée 25

2.7 Positionnement par Rapport à l'État de l'Art 25

2.8 Qu'est-ce qu'un Pipeline de Preprocessing ? 26

2.8.1 Principes Fondamentaux 26

2.8.2 Analogie avec l'ETL Classique . . . 26
2.9 Enjeux du Preprocessing pour le RAG . . . 27
2.9.1 Problème "Garbage In, Garbage Out" . . . 27
2.9.2 Exigences Spécifiques au Domaine Embedded . . . 27
2.10 Étapes Canoniques d'un Pipeline NLP . . . 28
2.10.1 1. Collecte (Data Ingestion) . . . 28
2.10.2 2. Nettoyage (Data Cleaning) . . . 28
2.10.3 3. Enrichissement (Feature Engineering) . . . 28
2.10.4 4. Mise en Relation (Linking & Similarity) . . . 29
2.10.5 5. Découpage (Chunking) . . . 29
2.10.6 6. Validation et Export . . . 29
2.11 Métriques de Qualité d'un Pipeline . . . 29
2.12 Patterns d'Architecture Pipeline . . . 30
2.12.1 Pattern Batch vs Streaming . . . 30
2.12.2 Pattern "Rejouable par Stage" . . . 30
2.12.3 Pattern "Gate de Qualité" . . . 30

3 Analyse des Besoins et Conception 31
3.1 Identification des Acteurs . . . 31
3.2 Besoins Fonctionnels . . . 32
3.3 Besoins Non Fonctionnels . . . 33
3.4 Contraintes et Décisions de Conception . . . 34
3.5 Approche Méthodologique . . . 35
3.5.1 Sélection de la Méthodologie : Scrum Adapté . . . 35
3.5.2 Pourquoi CRISP-DM ? Choix et Justification Méthodologique . . . 36
3.5.3 Adaptation de CRISP-DM au Projet . . . 39
3.6 Notre Pipeline : Mapping Théorie → Implémentation . . . 41
3.7 Contraintes de la Plateforme de Retrieval et Adaptation du Pipeline . . . 42
3.7.1 Paramètres Imposés par la Plateforme . . . 42
3.7.2 Découpage PARENT_CHILD . . . 43
3.7.3 Embeddings AdaV2 et Qualité Sémantique du Contenu . . . 44
3.7.4 Tailles de Lots (Batch Sizes) et Débit d'Indexation . . . 45
3.7.5 Synthèse : Contrainte Plateforme → Décision Pipeline → Valeur . . . 46
3.7.6 Valeur du Pipeline par Rapport à un Preprocessing RAG Standard 46
3.8 Architecture Cible . . . 47
3.9 Architecture Globale . . . 48
3.10 Choix Techniques . . . 48
3.11 Diagramme de Flux de Données . . . 48

3.12 Flux Source-to-Answer 50
3.13 Architecture de Delivery et Upload KB 50

4 Réalisation et Implémentation **51**
4.1 Vue d'Orchestration du Pipeline 51
4.2 Ingestion (Stage 1) 53
4.2.1 Objectif 53
4.2.2 Mécanisme 53
4.2.3 Exemple de Sortie — Issue Brute 53
4.3 Nettoyage (Stage 2) 54
4.3.1 Nettoyage des Issues 54
4.3.2 Exemple JSON — Issue Après Nettoyage 55
4.3.3 Nettoyage des Fichiers 55
4.3.4 Exemple JSON — Fichier Après Nettoyage 55
4.4 Nettoyage V3 — Nettoyage Intelligent par Type (Stage 2b) 56
4.4.1 Stratégies de Nettoyage par Type 56
4.4.2 Filtrage CMSIS Device Headers 58
4.4.3 Résultat du Filtrage — Exemple JSON 59
4.4.4 Résultats du Nettoyage V3 59
4.4.5 Compatibilité et Non-Régression 60
4.5 Enrichissement V2 (Stage 3) 60
4.5.1 Heuristiques de Classification 60
4.5.2 Exemple de Classification 60
4.5.3 Résultats de Couverture des Métadonnées 61
4.6 Similarité (Stage 4) 61
4.6.1 Algorithme 61
4.6.2 Exemple JSON — Issue avec Liens de Similarité 62
4.7 Chunking V2 (Stage 5 — Optionnel) 63
4.8 Delivery (Stage 7) 64
4.8.1 Resolver Cases 64
4.8.2 Exemple JSON — Resolver Case 64
4.8.3 Diagnostic Cards 65
4.8.4 Exemple JSON — Diagnostic Card 65
4.8.5 Alfred Vision — Enrichissement d'Images 66
4.8.6 Fiabilité Mesurée de l'Enrichissement Visuel 67
4.9 Architecture Agents — Persona ST ChatGPT 68
4.9.1 Vue d'ensemble 68
4.9.2 Persona : ST GitHub Analyzer 68

4.9.3 Tool 1 : KB Unstructured (Recherche Hybride) 69
4.9.4 Tool 2 : NovaX Fallback 69
4.9.5 Stratégie de Réponse (System Prompt) 70
4.9.6 Pipeline comme Agent de Maintenance 70
4.10 Synthèse de l'Implémentation 70
4.11 Configuration de la Plateforme ST ChatGPT 71
4.11.1 Architecture de la Plateforme 71
4.11.2 Configuration du Tool KB 71
4.11.3 Configuration du Tool NovaX (Fallback) 76
4.11.4 System Prompt de la Persona 77
4.11.5 Évolution Incrémentale du Quality Index 78
4.11.6 Automatisation CI/CD et Opérations 81
4.11.7 Couverture Actuelle et Future 84

5 Résultats et Évaluation 86
5.1 Cadrage Théorique et Définitions Préalables 86
5.1.1 Métriques Fondamentales d'Information Retrieval 86
5.1.2 Similarité Textuelle 87
5.1.3 Métriques de Qualité des Données 88
5.1.4 Confiance Contextuelle 88
5.1.5 Score Global Pondéré 89
5.1.6 Quality Index (QI) 89
5.1.7 Retrieval Hybride et Fallback 89
5.1.8 Tableau Récapitulatif des Notions 90
5.2 Justification des Seuils Décisionnels 90
5.2.1 Cadre Normatif de Référence 90
5.2.2 Échelle de Classification 91
5.2.3 Logique de Calibration de Chaque Borne 91
5.2.4 Seuils Différenciés par Champ 92
5.2.5 Seuil de Cohérence Alfred ($\tau = 0.12$) 92
5.3 Évaluation par Stage du Pipeline 93
5.3.1 Stage 1 : Ingestion — Complétude des Données Brutes 93
5.3.2 Stage 2 : Cleaning — Qualité de Préparation Textuelle 94
5.3.3 Stage 3a : Enrichissement — Classification par Heuristiques NLP . 95
5.3.4 Stage 4 : Similarité TF-IDF 96
5.3.5 Stage 5 : Delivery — Complétude des Artefacts Exportés 96
5.3.6 Synthèse des Scores par Stage 97
5.3.7 Score Global : Application Numérique 97

5.4 Évaluation de la Confiance Alfred Vision 98
5.4.1 Fiabilité Opérationnelle 98
5.4.2 Méthode de Calcul de la Cohérence Contextuelle 98
5.4.3 Résultats de Cohérence Contextuelle 100
5.4.4 Analyse de l'Écart Issues vs PDF 100
5.5 Évolution Incrémentale du Quality Index 101
5.5.1 Chronologie et Logique de Progression 101
5.5.2 Analyse des Points d'Inflexion 102
5.5.3 Matrice Valeur × Effort 103
5.6 Architecture de Fallback : d'Alfred Permissif à NovaX Restrictif 104
5.6.1 Problème Initial : Fallback Trop Permissif 104
5.6.2 Solution : Description Restrictive 104
5.6.3 Évolution vers NovaX 105
5.6.4 Synthèse de l'Évolution Fallback 105
5.7 Étude Comparative : Alfred Générique vs Persona KB 105
5.7.1 Protocole Expérimental 105
5.7.2 Grille de Scoring 106
5.7.3 Résultats Synthétiques 106
5.7.4 Analyse par Catégorie 107
5.8 Comparaison des Stratégies de Retrieval 107
5.9 Assurance Qualité du Chapitre d'Évaluation 108
5.9.1 Protocole de Vérification 108
5.9.2 Résultats du Contrôle 109
5.9.3 Points d'Amélioration Identifiés 110
5.9.4 Applicabilité au Reste du Rapport 110
5.10 Correspondance Métriques ↔ Normes ↔ Références 110
5.11 Synthèse et Bilan du Chapitre 111

6 Automatisation et CI/CD 113
6.1 GitHub Actions — Mise à Jour Automatique 113
6.1.1 Architecture Opérationnelle Actuelle 113
6.1.2 Déclenchement et Paramètres 113
6.1.3 Gouvernance Dynamique des Séries (Source Unique) 114
6.1.4 Politique Add vs Replace 115
6.1.5 Gestion des Datasource IDs et Reprise après Incident 115
6.1.6 Disponibilité Runner et Déploiement Industriel 115
6.1.7 Garanties de Robustesse 116
6.1.8 Gestion des Credentials en CI 116

6.1.9 Procédure Opérationnelle Recommandée 116

Conclusion Générale et Perspectives **117**

6.2 Bilan du Projet 117

6.3 Valeur Industrielle 118

6.4 Leçons Apprises 118

6.5 Perspectives 118

Références **120**

Annexes **124**

Table des figures

1.1 Logo officiel de STMicroelectronics 13
1.2 Structure organisationnelle de la division Microcontrôleurs (MCD) au site ST Tunis. L'équipe Maintenance (mise en évidence) est l'équipe d'accueil de ce projet. 14

2.1 Le paradigme RAG appliqué au projet : le pipeline conditionne la qualité de la KB interrogée à chaque question 23

3.1 Positionnement des acteurs autour du système : humains (gauche), cœur logiciel (centre), services externes (droite) 32
3.2 Frise des 7 sprints Scrum et jalons majeurs du projet 36
3.3 Flux `PARENT_CHILD` : les chunks enfants (1000 car.) sont indexés par AdaV2 (phase hors-ligne, flèches bleues) ; à la requête (phase en ligne, flèches orange), l'enfant le plus proche est retrouvé puis son chunk parent (3000 car.) est restitué comme contexte au LLM. 44
3.4 Vue d'ensemble du pipeline de preprocessing (7 stages) 49
3.5 Architecture détaillée du pipeline : stages, fichiers intermédiaires et branchement Alfred 49
3.6 Flux de données end-to-end : de la source GitHub à la réponse utilisateur . 50
3.7 Architecture de livraison et upload vers la Knowledge Base #793 50

4.1 Vue d'orchestration : `run_full_workflow.py` enchaîne les 7 stages avec gate de validation et flags d'exécution, en bouclant sur les 25 séries 52
4.2 Détail de l'étape d'ingestion : scripts et sorties 54
4.3 Arbre de décision du nettoyage V3 : routage par type de fichier avec taux de réduction 58
4.4 Taux de couverture des métadonnées par heuristique NLP (série H7) . . . 61
4.5 Détail du processus d'enrichissement et de calcul de similarité 63
4.6 Génération des Resolver Cases et Diagnostic Cards 66
4.7 Les 4 types d'artefacts delivery et leurs relations vers la KB #793 66
4.8 Intégration Alfred Vision AI pour la description automatique d'images . . 67

4.9 Confiance contextuelle des descriptions Alfred : images d'issues (200 évaluées) vs figures PDF (242 évaluées) 67
4.10 Architecture multi-agents du système ST GitHub Analyzer 68
4.11 Quality Index des stratégies Unstructured KB sur 96 questions — S1 maximise le rappel, S0 reste le point d'équilibre courant 74
4.12 Recherche hybride : fusion des branches sémantique et full-text avant génération 75
4.13 Évolution du Quality Index au fil des améliorations incrémentales 79
4.14 Automatisation CI/CD : orchestration GitHub Actions et upload KB via runner self-hosted 81

5.1 Complétude d'ingestion par série : toutes métriques au-dessus du seuil EXCELLENT 94
5.2 Couverture d'enrichissement STM32CubeH7 : score vs seuil par champ . . 95
5.3 Décomposition du score global 87.1% : contribution pondérée de chaque stage 98
5.4 Confiance contextuelle Alfred — seuil d'alerte $\tau = 0.12$ 100
5.5 Évolution du Quality Index : le saut majeur (+33) provient des Diagnostic Cards, la régression (−34) de la dilution multi-séries 104
5.6 Comparaison Alfred vs Persona KB par catégorie de question 107
5.7 Comparaison des stratégies de retrieval KB sur 96 questions 108

6.1 Architecture CI/CD actuelle : normalisation, contrôle runner, exécution matricielle et politique de mise à jour KB 113

Introduction Générale

Le développement de systèmes embarqués basés sur les microcontrôleurs STM32 de STMicroelectronics est un domaine en constante évolution, où les ingénieurs font face quotidiennement à des problèmes techniques complexes : bugs de drivers, comportements inattendus des périphériques, incompatibilités entre versions firmware, configurations matérielles spécifiques. Pour résoudre ces problèmes, ils doivent aujourd'hui parcourir manuellement des milliers d'issues GitHub, des centaines de fichiers de documentation, et des dizaines de Release Notes répartis sur plus de 200 dépôts — un processus lent, fragmenté et inefficace.

Ce projet de fin d'études, réalisé au sein de l'équipe **Technical Marketing and Application Support** de la division Microcontrôleurs (MCD) du site STMicroelectronics de Tunis, propose une solution à cette problématique : la conception et l'implémentation d'un **pipeline de prétraitement intelligent** qui transforme automatiquement les données brutes des dépôts GitHub STM32Cube en une base de connaissances structurée. Cette base alimente un chatbot IA spécialisé ("ST GitHub Analyzer") déployé sur la plateforme interne ST ChatGPT, capable de fournir des diagnostics précis, sourcés et actionnables aux ingénieurs.

Le pipeline couvre les **25 séries** de la famille STM32 (13 en production, 12 configurées) et produit 4 types d'artefacts complémentaires : issues structurées, fichiers documentés, resolver cases (chaînes issue→PR→commit) et diagnostic cards (analyses de pannes structurées). Un enrichissement IA multi-modal par Alfred Vision rend le contenu visuel (captures d'écran, figures PDF) exploitable par la recherche.

Les résultats démontrent une **amélioration nette de la pertinence** par rapport à un LLM générique sur les questions techniques STM32, avec un Quality Index atteignant 94/100 sur la série pilote H7 grâce aux Diagnostic Cards.

Ce rapport est organisé en **cinq chapitres**, encadrés par la présente introduction générale et une conclusion générale :

— **Chapitre 1 — Contexte Général du Projet** : présentation de l'entreprise d'accueil, du site de Tunis, de la division Microcontrôleurs et de l'équipe d'accueil, de l'écosystème STM32/GitHub, de la problématique, des objectifs et des contributions

— **Chapitre 2 — État de l'Art et Fondements Théoriques** : le paradigme RAG,

les enjeux du preprocessing pour une KB, la similarité TF-IDF, Alfred Vision, le positionnement par rapport aux solutions existantes, et la notion générale de pipeline de preprocessing (étapes canoniques, métriques, patterns)

— **Chapitre 3 — Analyse des Besoins et Conception** : identification des acteurs, besoins fonctionnels et non fonctionnels, contraintes de conception, choix méthodologique (Scrum adapté et CRISP-DM — avec une justification comparative face aux méthodologies alternatives), contraintes de la plateforme de retrieval et architecture cible

— **Chapitre 4 — Réalisation et Implémentation** : les 7 stages du pipeline (ingestion à delivery), l'enrichissement Alfred Vision, puis la configuration de la persona sur la plateforme ST ChatGPT et l'automatisation CI/CD

— **Chapitre 5 — Résultats et Évaluation** : métriques par stage, confiance Alfred, évolution incrémentale du Quality Index, et étude comparative Alfred générique vs Persona KB

Chapitre 1

Contexte Général du Projet

Introduction

Ce chapitre présente l'entreprise d'accueil et le cadre dans lequel ce projet de fin d'études a été mené, avant d'exposer l'écosystème STM32/GitHub, la problématique précise à laquelle répond le projet, ses objectifs et ses contributions. Nous décrivons successivement STMicroelectronics à l'échelle mondiale, le site de Tunis, la division Microcontrôleurs (MCD) et l'équipe Technical Marketing and Application Support — et en son sein l'équipe Maintenance et Validation — qui a accueilli ce travail. Cette présentation constitue le socle sur lequel s'appuiera le chapitre suivant pour introduire les fondements théoriques du projet.

1.1 Présentation de l'Entreprise

1.1.1 Présentation Générale

STMicroelectronics est une entreprise mondiale de semi-conducteurs dédiée à l'amélioration de la vie quotidienne grâce à des solutions électroniques innovantes [4]. Avec l'un des portefeuilles de produits les plus étendus de l'industrie, ST dessert plus de 200 000 clients dans le monde, réalisant un chiffre d'affaires net de 13,27 milliards USD en 2024 [5].

L'entreprise a été fondée en juin 1987 sous le nom de **SGS-THOMSON Microelectronics**, suite à la fusion de SGS Microelettronica (Italie) et de Thomson Semiconductors (France). Le nom "STMicroelectronics" a été officiellement adopté en mai 1998. Avec plus de 80 bureaux commerciaux et 14 sites de fabrication principaux à travers le monde, l'entreprise emploie plus de 50 000 personnes [5].

[Logo STMicroelectronics — exporter depuis draw.io]

FIGURE 1.1 – Logo officiel de STMicroelectronics

1.1.2 Vision et Marchés

STMicroelectronics a fait de l'innovation technologique une priorité stratégique depuis plus de 25 ans. L'entreprise réalise des investissements orientés marché dans le développement technologique, avec l'objectif de commercialiser des innovations de pointe créant une valeur immédiate pour ses clients. Aujourd'hui, ST livre des produits toujours plus petits, plus rapides, plus économes en énergie et dotés de nouvelles fonctionnalités.

Portée par une culture mondiale de *lean manufacturing*, ST adresse **trois marchés principaux** :

- **Automobile** (37%) : capteurs, microcontrôleurs, gestion de puissance pour véhicules électriques et autonomes
- **Industriel** (30%) : automatisation, contrôle moteur, IoT industriel, smart grid
- **Électronique personnelle** (20%) : smartphones, wearables, audio, gaming
- **Communications et équipements** (13%) : infrastructure réseau, serveurs

ST propose des modèles d'engagement allant des systèmes embarqués spécifiques à l'application jusqu'aux produits standard spécialisés (ASSP).

1.1.3 STMicroelectronics Tunis

Le site **ST Tunis**, situé dans le **Technopark El Ghazela**, a débuté ses opérations en 2001 avec neuf ingénieurs. Aujourd'hui, il compte plus de **200 employés** travaillant dans les domaines du design, des outils, des applications, de la qualité et du support [5]. Le site participe à chaque étape, depuis la conception initiale de nouveaux circuits jusqu'à la vérification finale de leur fonctionnalité avant la production de masse.

1.2 Division Microcontrôleurs (MCD)

La **Microcontroller Division (MCD)** est dédiée à la conception de microcontrôleurs, de plateformes de démonstration et de solutions pour les applications grand public. Elle est responsable du développement de drivers et de logiciels embarqués — en particulier la validation et le développement des bibliothèques et firmware qui pilotent les microcontrôleurs ST. La division couvre l'ensemble du cycle de vie du firmware : conception initiale, portage entre séries, validation fonctionnelle, maintenance corrective et support aux clients externes.

La Figure 1.2 présente l'organisation de la MCD sur le site de Tunis. L'équipe **Maintenance** (mise en évidence) est l'équipe d'accueil de ce projet de fin d'études.

[Organigramme MCD — ST Tunis
à exporter depuis draw.io :
Figures/chapter1/st-site-organization.pdf]

FIGURE 1.2 – Structure organisationnelle de la division Microcontrôleurs (MCD) au site ST Tunis. L'équipe Maintenance (mise en évidence) est l'équipe d'accueil de ce projet.

1.3 Équipe d'Accueil : Technical Marketing and Application Support

Ce projet a été réalisé au sein de l'équipe **Technical Marketing and Application Support**, rattachée à la MCD sur le site de Tunis. Cette équipe assure la jonction entre les équipes de conception (design), les clients externes utilisant les microcontrôleurs STM32, et les processus internes de qualité et d'industrialisation. Elle comprend deux sous-équipes complémentaires :

— **L'équipe GitHub** : assure le support client via les dépôts GitHub publics de ST-Microelectronics, publie les mises à jour des drivers HAL, et maintient les projets d'exemples pour faciliter le développement d'applications basées sur les microcontrôleurs STM32.

— **L'équipe Maintenance et Validation** : garantit la satisfaction client par la maintenance des bugs et la validation fonctionnelle des microcontrôleurs fabriqués et de leurs versions HAL associées. Cette équipe représente la jonction entre les processus de conception logicielle/matérielle et l'industrialisation. **C'est au sein de cette équipe que le présent projet de fin d'études a été réalisé.**

C'est dans ce contexte opérationnel que le besoin d'un **chatbot IA spécialisé STM32** a émergé : face au volume croissant d'issues GitHub (plusieurs milliers par an, répartis sur 200+ dépôts) et à la richesse inexploitée de ces données, l'équipe Maintenance et Validation a identifié l'opportunité d'automatiser le support de premier niveau par un pipeline

intelligent de prétraitement alimentant une base de connaissances structurée. Les sections suivantes détaillent cet écosystème, la problématique précise qui en découle, ainsi que les objectifs et contributions du projet.

1.4 L'Écosystème STM32 et les Dépôts GitHub

Phase CRISP-DM : *Data Understanding (analyse des sources de données brutes), avec un ancrage en Business Understanding pour le cadrage de l'écosystème STM32.*

Au cœur de l'offre de ST se trouve la famille **STM32**, une gamme de microcontrôleurs basés sur les cœurs ARM Cortex-M (M0, M0+, M3, M4, M7, M33, M55) et Cortex-A. Lancée en 2007, la famille STM32 compte aujourd'hui plus de **1 500 références** regroupées en plus de 30 séries, chacune ciblant un profil d'application spécifique :

Série	Cœur	Cible applicative
STM32F0/F1/F2	Cortex-M0/M3	Entrée de gamme, migration depuis 8-bit
STM32F3/F4	Cortex-M4F	Applications signal mixte, contrôle moteur, audio
STM32F7	Cortex-M7	Haute performance, graphisme, connectivité
STM32G0/G4	Cortex-M0+/M4	Nouvelle génération mainstream, efficacité énergétique
STM32H5/H7/H7RS	Cortex-M33/M7	Ultra-haute performance, dual-core, temps réel
STM32L0/L1/L4/L5	Cortex-M0+/M3/M4/M33	Ultra-basse consommation, IoT, wearables
STM32U0/U3/U5	Cortex-M0+/M33	Ultra-basse consommation nouvelle génération
STM32WB/WBA/WL	Cortex-M4+M0/M33	Connectivité sans fil (BLE, Thread, LoRa, Sub-GHz)
STM32N6	Cortex-M55+NPU	IA embarquée, vision par ordinateur

Pour chaque série, ST fournit un **Firmware Package** complet sous le nom **STM32Cube**, comprenant :

— Le **HAL** (Hardware Abstraction Layer) : couche d'abstraction portable entre séries

— Les **drivers LL** (Low-Layer) : accès registre optimisé pour la performance

— Les **BSP** (Board Support Packages) : drivers spécifiques aux cartes de développement (NUCLEO, Discovery, Eval)

— Les **CMSIS Device Headers** : définitions de registres ARM standardisées
— Des **Middlewares** : FreeRTOS, LwIP, FatFs, USB, TouchGFX, LoRaWAN
— Des **exemples d'application** : projets prêts à compiler par board et par IDE

1.4.1 Les Dépôts GitHub STMicroelectronics

Depuis 2019, STMicroelectronics publie l'intégralité de ses firmware packages STM32Cube en open source sur GitHub (`github.com/STMicroelectronics`). Cette présence GitHub représente un écosystème considérable :

— **25 dépôts principaux** (un par série MCU : `STM32CubeH7`, `STM32CubeF4`, etc.)
— **200+ sous-dépôts** pour les drivers individuels (`stm32h7xx-hal-driver`, `stm32f4xx-nucleo-bsp`, `cmsis_device_h7`, etc.)
— **Des milliers d'issues** ouvertes par la communauté (bug reports, questions d'utilisation, demandes de fonctionnalités)
— **Des centaines de Pull Requests** avec des corrections de code et des discussions techniques
— De la **documentation technique** embarquée : Release Notes (HTML), README (Markdown), documentation PDF

Ce projet porte sur les **25 séries** de la famille STM32 : **13 séries en production** (C0, F0, F1, F2, F3, F4, F7, G0, H5, H7, H7RS, U5, WL) couvrant 47 dépôts et **12 séries supplémentaires** configurées et prêtes au déploiement (G4, L0, L1, L4, L5, N6, U0, U3, WB, WB0, WBA, WL3). La couverture totale cible est de **25 séries MCU** avec plus de 200 sous-dépôts associés.

1.5 Problématique

Phase CRISP-DM : *Business Understanding (formulation du problème métier et des objectifs de support).*

Les ingénieurs STM32 — qu'ils soient internes à ST ou clients — rencontrent quotidiennement des problèmes techniques lors du développement firmware : bugs HAL, comportements inattendus des périphériques, incompatibilités entre versions, configurations complexes. Pour résoudre ces problèmes, ils doivent aujourd'hui parcourir manuellement les ressources disponibles, un processus qui souffre de quatre limitations majeures :

1. **Volume et dispersion** : les données sont réparties sur plus de 200 dépôts GitHub, totalisant des milliers d'issues, des centaines de PRs et des dizaines de milliers de fichiers source. Trouver l'information pertinente revient à chercher une aiguille dans une botte de foin numérique.

2. **Bruit et hétérogénéité** : le contenu des issues mélange du texte technique utile avec du bruit considérable — templates vides, messages automatiques de bots GitHub, discussions hors-sujet, doublons, markdown malformé. Les fichiers source contiennent des licences volumineuses (30% du texte pour certains fichiers) et les headers CMSIS dépassent 1,7 Mo de définitions bit-à-bit inutiles pour le diagnostic.
3. **Fragmentation des connaissances** : la résolution d'un bug implique souvent de recouper une issue, une PR avec sa discussion, un commit dans un sous-dépôt différent, et une Release Note qui documente le correctif. Ces liens ne sont pas toujours explicites dans GitHub et nécessitent une investigation manuelle fastidieuse.
4. **Opacité du contenu visuel** : de nombreuses issues contiennent des captures d'écran (erreurs IDE STM32CubeIDE, oscillogrammes, configurations CubeMX, schémas de câblage) qui portent une information diagnostique critique mais sont totalement invisibles pour la recherche textuelle. De même, les Release Notes PDF contiennent des figures techniques non indexables.

La question centrale de ce projet est donc : *comment transformer cette masse de données brutes, hétérogènes et fragmentées en une base de connaissances structurée, exploitable par un chatbot IA capable de fournir des diagnostics précis, sourcés et actionnables aux ingénieurs STM32 ?*

1.5.1 Critique de l'Existant

Avant ce projet, les solutions disponibles présentaient des limitations significatives :

Solution existante	**Limitation principale**	**Impact utilisateur**
Recherche GitHub native	Recherche textuelle brute, pas de filtrage par composant, layer ou sévérité	L'ingénieur parcourt 50+ issues sans garantie de trouver la bonne
Alfred (LLM ST)	Connaissances génériques, aucun accès aux issues GitHub spécifiques	Conseils généraux sans root cause précise ni traçabilité
Stack Overflow	Communauté large mais non spécifique ST, pas de traçabilité vers les correctifs officiels	Solutions non vérifiées, souvent obsolètes ou inapplicables
Documentation PDF	Non indexable, figures opaques, pas de lien avec les bugs connus	L'ingénieur ne peut pas corréler son problème avec les bugs documentés

Support direct (email/forum)	Temps de réponse de 48h à 2 semaines	Blocage du développement client

Constat quantifié : Sur un benchmark de 20 questions techniques couvrant 5 séries MCU, Alfred générique obtient un score de 116/200 (58%) tandis que notre solution avec Knowledge Base obtient 161/200 (80,5%), soit un **gain de +39%** en pertinence, actionnabilité et traçabilité (cf. Section 5.7 pour le détail de l'étude).

1.6 Objectifs du Projet

Les objectifs de ce projet de fin d'études sont les suivants :

1. **Concevoir** un pipeline de prétraitement automatisé capable d'ingérer les données brutes depuis l'API GitHub, de les nettoyer, enrichir et structurer pour un système RAG
2. **Implémenter** les 7 étapes du pipeline (ingestion, nettoyage V1+V3, enrichissement NLP, similarité TF-IDF, chunking, validation, delivery) avec une architecture modulaire et rejouable
3. **Déployer** un chatbot IA spécialisé STM32 sur la plateforme ST ChatGPT, alimenté par la Knowledge Base générée par le pipeline
4. **Évaluer** la qualité du système de bout en bout via un Quality Index sur 52 questions et une étude comparative avec Alfred générique
5. **Automatiser** la mise à jour continue via CI/CD (GitHub Actions) et des scripts d'orchestration
6. **Industrialiser** la solution pour couvrir l'intégralité des 25 séries STM32

1.7 Contributions Techniques Originales

1. **Pipeline modulaire 7 stages** rejouable indépendamment par étape avec artefacts JSON traçables
2. **Nettoyage V3 par type** : routage intelligent avec réduction de 96% sur les headers CMSIS et 33% sur le HTML
3. **Filtre CMSIS intelligent** ne conservant que les sections à valeur diagnostique (IRQ, base addresses, TypeDef)
4. **4 types d'artefacts delivery** complémentaires : issues, files, resolver cases, diagnostic cards

5. **Heuristiques NLP domain-specific** pour la classification STM32 (layer, component, severity, board, issue_kind)
6. **Description automatique d'images** par Alfred Vision AI (captures d'écran issues + figures PDF)
7. **Architecture KB-first + NovaX fallback** avec description restrictive évitant le court-circuit de la KB
8. **Workflow Delete+Add** résolvant les conflits de noms lors de l'upload des datasources
9. **Couverture multi-séries** : 13 séries en production, 12 configurées, processus reproductible en 3 étapes

1.8 Enjeux Éthiques et Responsabilité

L'utilisation de l'intelligence artificielle dans un contexte de support technique industriel soulève plusieurs enjeux éthiques que nous avons pris en compte :

1. **Traçabilité et transparence** : chaque réponse du chatbot est accompagnée de citations vérifiables (numéro d'issue, URL GitHub, version firmware). L'utilisateur peut toujours remonter à la source pour vérifier l'exactitude de l'information. Cette traçabilité est un choix de conception délibéré.
2. **Fiabilité et hallucinations** : le paradigme RAG réduit significativement les hallucinations par rapport à un LLM générique, car les réponses sont ancrées dans des documents factuels. Néanmoins, le risque n'est pas nul — d'où l'importance des gates de validation et du Quality Index régulier.
3. **Confidentialité des données** : les données sources proviennent des dépôts GitHub de STMicroelectronics (issues, fichiers, PRs). Bien que ces dépôts soient accessibles sur GitHub, les données traitées, la Knowledge Base générée et la plateforme ST ChatGPT sont des ressources **internes à ST**. Le pipeline ne traite aucune donnée personnelle des utilisateurs GitHub au-delà des identifiants publics (pseudo, commentaires publics).
4. **Biais de couverture** : certaines séries MCU ont plus d'issues que d'autres, ce qui crée un biais de couverture. Les séries historiques (F4, H7) sont mieux représentées que les séries récentes (N6, U0). Nous documentons cette limitation dans le chapitre d'évaluation.
5. **Impact sur l'emploi** : le chatbot ne remplace pas les ingénieurs de support mais **augmente leur productivité** en automatisant la recherche de premier niveau. Les cas complexes nécessitent toujours une expertise humaine.

1.9 Les 25 Séries STM32 Couvertes

Le pipeline est conçu pour couvrir l'intégralité de l'écosystème STM32. Le tableau suivant détaille l'état de chaque série :

Série	Statut	Repos principaux	Issues	Sous-dépôts drivers
STM32CubeC0	Production	1	∼50	3 (HAL, CMSIS, BSP)
STM32CubeF0	Production	1	∼120	3
STM32CubeF1	Production	1	∼180	3
STM32CubeF2	Production	1	∼60	3
STM32CubeF3	Production	1	∼100	3
STM32CubeF4	Production	1	∼800	16 (HAL, CMSIS, 14 BSP)
STM32CubeF7	Production	1	∼350	6
STM32CubeG0	Production	1	∼150	3
STM32CubeH5	Production	1	∼200	6
STM32CubeH7	Production	1	∼500	12 (HAL, CMSIS, 10 BSP)
STM32CubeH7RS	Production	1	∼80	3
STM32CubeU5	Production	1	∼350	8
STM32CubeWL	Production	1	∼150	5
STM32CubeG4	Configuré	1	∼49	4 (HAL, CMSIS, 2 BSP)
STM32CubeL0	Configuré	1	∼31	4
STM32CubeL1	Configuré	1	∼13	4
STM32CubeL4	Configuré	1	∼94	4
STM32CubeL5	Configuré	1	—	4
STM32CubeN6	Configuré	1	—	3
STM32CubeU0	Configuré	1	—	3
STM32CubeU3	Configuré	1	—	3
STM32CubeWB	Configuré	1	—	3
STM32CubeWB0	Configuré	1	—	3
STM32CubeWBA	Configuré	1	—	3
STM32CubeWL3	Configuré	1	—	3
Total		**25**	**∼3 350**	**∼110**

1.10 Plan du Rapport

Ce rapport est désormais organisé en **cinq chapitres**. Le Chapitre 1 (le présent chapitre) présente l'entreprise d'accueil, l'écosystème STM32/GitHub, la problématique et les objectifs. Le Chapitre 2 pose les fondements théoriques (RAG, preprocessing, TF-IDF, Alfred Vision) et la notion de pipeline de preprocessing. Le Chapitre 3 détaille l'analyse des besoins et la conception (méthodologie Scrum, adaptation de CRISP-DM — et sa justification par rapport aux méthodologies alternatives —, architecture cible). Le Chapitre 4 couvre la réalisation : les 7 stages du pipeline et la configuration de la plateforme ST ChatGPT. Enfin, le Chapitre 5 présente les résultats, l'évaluation quantifiée et l'automatisation CI/CD, avant la conclusion générale.

Chapitre 2

État de l'Art et Fondements Théoriques

Introduction

Ce chapitre pose les fondements théoriques mobilisés par le projet : le paradigme RAG, les enjeux de qualité du preprocessing pour une base de connaissances, la similarité TF-IDF, et l'enrichissement visuel par Alfred Vision. Il situe ensuite notre solution par rapport aux approches existantes (recherche GitHub native, LLM générique, Stack Overflow, GitHub Copilot), avant de généraliser la réflexion à la notion de pipeline de preprocessing pour le RAG — ses étapes canoniques, ses métriques de qualité et ses patterns d'architecture. Ce chapitre relève essentiellement de la phase **Data Understanding** de CRISP-DM, en posant les bases conceptuelles qui seront mobilisées pour la conception (Chapitre 3).

2.1 Le Paradigme RAG

Notre projet repose sur le paradigme **RAG** (Retrieval-Augmented Generation) [1], qui combine la recherche documentaire avec la génération de texte par un LLM. Concrètement dans notre système, lorsqu'un ingénieur pose une question, le moteur de recherche de la KB #793 récupère les documents les plus pertinents (issues, fichiers, diagnostic cards), puis le LLM (GPT-5.1 via Azure OpenAI) génère une réponse ancrée dans ces documents avec des citations vérifiables.

Ce paradigme nous apporte trois avantages concrets :

— **Réduction des hallucinations** : les réponses sont ancrées dans des issues GitHub réelles, pas dans les connaissances génériques du modèle

— **Mise à jour sans ré-entraînement** : quand de nouvelles issues apparaissent, il suffit de relancer le pipeline et uploader — pas besoin de ré-entraîner le modèle

— **Traçabilité** : chaque réponse cite un numéro d'issue, une URL GitHub, une version firmware

La Figure 2.1 illustre le flux RAG appliqué à notre système : la question de l'ingénieur déclenche une recherche dans la KB #793, les documents pertinents sont récupérés, puis le LLM génère une réponse sourcée. Le pipeline de prétraitement (partie gauche) est ce qui **alimente et conditionne** la qualité de la KB indexable.

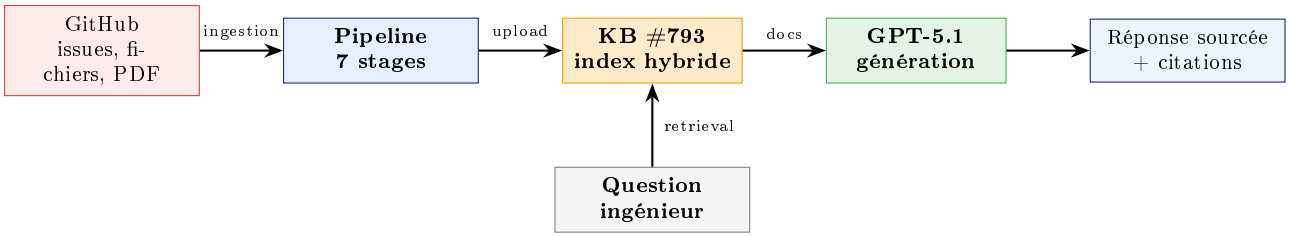


FIGURE 2.1 – Le paradigme RAG appliqué au projet : le pipeline conditionne la qualité de la KB interrogée à chaque question

2.2 Preprocessing et Qualité de la Base de Connaissances

La qualité des réponses RAG dépend directement de la qualité des documents indexés — c'est le principe "**Garbage In, Garbage Out**". Dans notre projet, nous avons constaté expérimentalement que :

— Les headers CMSIS bruts (1,7 Mo de `#define`) polluent le contexte du LLM avec du bruit inutile → d'où le filtre V3 (-96%)

— Les balises HTML des Release Notes diluent le signal technique → d'où le nettoyage `strip_html()` (-33%)

— Les licences volumineuses dans le code source occupent de l'espace dans les chunks → d'où la suppression automatique

Pour le découpage en chunks, nous utilisons une approche **paragraph-based** pour les issues et **section-based** pour les fichiers, avec une taille maximale de 512 tokens et un overlap de 50 tokens.

2.3 TF-IDF pour la Similarité

Pour le calcul de similarité inter-issues, nous utilisons **TF-IDF** (Term Frequency – Inverse Document Frequency) [2], qui pondère les termes par leur spécificité dans le corpus. Nous avons choisi TF-IDF plutôt que des embeddings neuronaux car il est déterministe,

interprétable et ne nécessite pas de modèle pré-entraîné — ce qui est cohérent avec notre choix d'heuristiques plutôt que de ML pour la classification.

2.4 Alfred Vision pour la Description d'Images

Pour rendre les captures d'écran et figures PDF "cherchables" par le RAG, nous utilisons **Alfred Vision**, le service de description d'images interne de ST basé sur Azure OpenAI. Alfred reçoit une image et un prompt spécialisé STM32, et produit une description textuelle orientée diagnostic (noms de périphériques, messages d'erreur, valeurs de registres).

2.5 Étude Comparative des Solutions Existantes

Avant de concevoir notre pipeline, nous avons évalué les solutions existantes pour le support technique assisté par IA :

Solution	**Ce qu'elle fait bien**	**Pourquoi insuffisante pour nous**
Recherche GitHub native	Gratuite, intégrée	Pas de sémantique, pas de filtrage par composant/layer, résultats noyés dans le bruit
Alfred (LLM ST générique)	Toujours disponible, connaissances larges	Pas d'accès aux issues GitHub, conseils génériques sans root cause (score 116/200)
Stack Overflow	Communauté large	Non spécifique ST, solutions non vérifiées et souvent obsolètes
GitHub Copilot	Complétion de code intelligente	Pas de KB custom, aucun accès aux issues et bugs connus
Notre approche	Pipeline domain-specific, 4 artefacts, enrichissement IA	Pipeline custom à maintenir

Conclusion : aucune solution existante ne combine le preprocessing domain-aware, les artefacts de diagnostic structurés et l'enrichissement d'images. C'est ce constat qui a

motivé la conception de notre pipeline.

2.6 Synthèse de la Solution Proposée

Face à cette problématique et aux limites des solutions existantes, nous proposons un **pipeline intelligent de prétraitement en 7 étapes** qui transforme les données GitHub brutes en une Knowledge Base structurée, alimentant un chatbot IA spécialisé STM32 déployé sur la plateforme ST ChatGPT. L'architecture repose sur trois piliers fondamentaux :

1. **Pipeline de preprocessing modulaire** : Ingestion → Nettoyage (V1+V3 par type) → Enrichissement NLP → Similarité TF-IDF → Chunking → Validation schéma → Delivery. Chaque étape est **rejouable indépendamment**, produit des artefacts JSON traçables, et peut être évaluée en isolation.
2. **4 types d'artefacts complémentaires** couvrant l'intégralité du spectre de connaissances :
   - **Issues structurées** : symptôme, root cause, fix, métadonnées enrichies (layer, component, severity, board)
   - **Fichiers documentés** : HAL/BSP/CMSIS/PDF extraits et nettoyés par type avec filtrage intelligent
   - **Resolver Cases** : chaînes Issue → PR → Commit avec preuves de résolution et fichiers modifiés
   - **Diagnostic Cards** : analyses structurées (symptôme → hypothèses → validation → fix → prévention)
3. **Enrichissement IA multi-modal** : Alfred Vision AI convertit automatiquement les captures d'écran des issues et les figures techniques des PDFs en descriptions textuelles cherchables, rendant le contenu visuel exploitable par la recherche RAG.

2.7 Positionnement par Rapport à l'État de l'Art

Notre approche se positionne comme une **solution RAG verticale** (domain-specific) par opposition aux solutions RAG horizontales (génériques). Les contributions originales par rapport à l'état de l'art sont :

1. **Preprocessing domain-aware** : là où les solutions génériques indexent le contenu brut, notre pipeline applique un nettoyage adapté au type de fichier (HTML pour les Release Notes, filtrage intelligent pour les CMSIS headers, suppression de licence pour le code source)

2. **Enrichissement métadonnées par heuristiques** : classification automatique des issues par layer architecturale, composant matériel, board, sévérité et type — sans données d'entraînement supervisées, grâce à des heuristiques regex spécifiques au domaine STM32
3. **Artefacts de diagnostic structurés** : les Diagnostic Cards et les Resolver Cases sont des formats d'artefacts originaux qui n'existent pas dans les pipelines RAG standards. Ils ont démontré un impact majeur sur la qualité (+33 points de Quality Index)
4. **Enrichissement Vision IA des images** : l'utilisation d'Alfred Vision pour rendre cherchables les captures d'écran et les figures PDF est une contribution technique inédite dans le contexte STM32

2.8 Qu'est-ce qu'un Pipeline de Preprocessing ?

Un **pipeline de preprocessing** (ou pipeline de prétraitement de données) est une chaîne de transformations automatisées qui convertit des données brutes et non structurées en données propres, enrichies et exploitables par un système en aval (ici, un moteur RAG).

2.8.1 Principes Fondamentaux

1. **Séquentialité** : Chaque étape prend en entrée la sortie de l'étape précédente
2. **Idempotence** : Rejouer une étape avec les mêmes entrées produit les mêmes sorties
3. **Modularité** : Chaque étape est indépendante et remplaçable
4. **Traçabilité** : Chaque transformation est documentée et versionnable
5. **Validation** : Des gates de qualité empêchent la propagation d'erreurs

2.8.2 Analogie avec l'ETL Classique

Le pipeline de preprocessing s'apparente à un processus **ETL** (Extract, Transform, Load) adapté au domaine NLP/RAG :

Phase ETL	Équivalent Pipeline	Description
Extract	Ingestion	Extraction depuis l'API GitHub (issues, fichiers, PRs)
Transform	Cleaning + Enrichment + Chunking	Normalisation, classification NLP, découpage

Load	Delivery + Upload	Export JSON + upload vers la KB cloud

2.9 Enjeux du Preprocessing pour le RAG

Le paradigme RAG (Retrieval-Augmented Generation) est extrêmement sensible à la qualité des documents indexés. Un mauvais preprocessing cause directement :

2.9.1 Problème "Garbage In, Garbage Out"

- **Bruit** : du texte non pertinent (signatures email, messages bot, headers GitHub) dilue les signaux techniques utiles
- **Incomplétude** : un document sans métadonnées (layer, component) ne peut pas être filtré efficacement lors de la recherche
- **Duplication** : des contenus dupliqués saturent le contexte du LLM avec des informations redondantes
- **Opacité** : les images (captures d'écran, logs visuels) sont invisibles pour la recherche textuelle

2.9.2 Exigences Spécifiques au Domaine Embedded

Le domaine STM32 pose des défis supplémentaires par rapport au preprocessing de texte générique :

Défi	**Impact sur le Preprocessing**
Vocabulaire technique dense	Les noms d'API (`HAL_SPI_Transmit_DMA`), les registres (`CR1`), et les codes d'erreur doivent être préservés intacts
Contexte multi-couches	Un problème peut impliquer simultanément le hardware, le HAL, le middleware et l'application — le pipeline doit identifier la couche concernée
Dépendance à la version	Un workaround valide pour CubeH7 v1.9 peut être obsolète en v1.11
Images techniques	Oscillogrammes, screenshots IDE, schémas de connexion contiennent des informations critiques

Multi-repository	Une série MCU (ex : H7) s'étend sur 12+ repos (HAL, BSP, CMSIS, exemples)

2.10 Étapes Canoniques d'un Pipeline NLP

La littérature identifie les étapes suivantes pour un pipeline de preprocessing NLP appliqué au RAG [1] :

2.10.1 1. Collecte (Data Ingestion)

Récupération exhaustive des données depuis les sources :

— API REST (GitHub, JIRA, Confluence)

— Systèmes de fichiers (repositories clonés)

— Bases de données internes

Critère de qualité : complétude (aucune donnée manquée) et fraîcheur (données récentes).

2.10.2 2. Nettoyage (Data Cleaning)

Suppression du bruit et normalisation :

— Suppression des éléments non informatifs (messages bot, templates vides, duplicatas)

— Normalisation des encodages et formats

— Extraction du texte utile depuis des structures complexes (HTML, Markdown)

— Remplacement des éléments non textuels (images → markers)

Critère de qualité : ratio signal/bruit élevé.

2.10.3 3. Enrichissement (Feature Engineering)

Ajout de métadonnées structurées par analyse automatique :

— **Classification** : catégoriser chaque document (type, sévérité, couche architecturale)

— **Extraction d'entités** : identifier les boards, composants, versions mentionnées

— **Scoring** : évaluer la force probante (*evidence strength*) de chaque document

— **Filtrage qualité** : marquer les documents invalides (trop courts, hors-scope, spam)

Critère de qualité : précision et rappel de la classification.

2.10.4 4. Mise en Relation (Linking & Similarity)

Création de liens entre documents :

— Similarité textuelle (TF-IDF, embeddings)

— Liens structurels (issue → PR → commit)

— Regroupement thématique (clustering)

Critère de qualité : pertinence des liens (pas de faux positifs bruyants).

2.10.5 5. Découpage (Chunking)

Division des documents longs en segments optimaux pour le retrieval :

— Respect des frontières sémantiques (paragraphes, sections)

— Taille bornée (typiquement 256–512 tokens)

— Overlap pour maintenir la cohérence contextuelle

Critère de qualité : chaque chunk est auto-suffisant et pertinent.

2.10.6 6. Validation et Export

Dernière étape avant l'indexation :

— Validation structurelle (JSON Schema)

— Vérification des champs obligatoires

— Export dans le format attendu par le système cible (KB)

— Écriture de fichiers de traçabilité (summary, manifests)

Critère de qualité : conformité 100% au schéma cible.

2.11 Métriques de Qualité d'un Pipeline

Pour évaluer la qualité d'un pipeline de preprocessing, on utilise des métriques à chaque niveau :

$$Q_{\text{pipeline}} = f(Q_{\text{complétude}},\ Q_{\text{classification}},\ Q_{\text{enrichissement}},\ Q_{\text{conformité}}) \tag{2.1}$$

— $Q_{\text{complétude}}$: pourcentage de données source effectivement récupérées

— $Q_{\text{classification}}$: taux de couverture des champs de classification (non-null)

— $Q_{\text{enrichissement}}$: taux de succès des transformations IA (descriptions d'images)

— $Q_{\text{conformité}}$: pourcentage de documents passant la validation schéma

L'objectif est de maximiser ces métriques tout en maintenant la **traçabilité** (pouvoir remonter de tout artefact final vers sa source brute).

2.12 Patterns d'Architecture Pipeline

2.12.1 Pattern Batch vs Streaming

Notre pipeline utilise le pattern **batch** : toutes les données d'un repository sont traitées en une seule exécution. C'est adapté car :

— Les données GitHub changent lentement (quelques issues/semaine)

— Le calcul de similarité nécessite le corpus complet

— La validation globale (schéma) ne peut se faire que sur l'ensemble

2.12.2 Pattern "Rejouable par Stage"

Chaque stage lit un fichier JSON d'entrée et écrit un fichier JSON de sortie. Cela permet de :

— Relancer une étape sans tout recalculer (`-skip-ingestion`)

— Débugger une étape en isolation

— Tester de nouvelles heuristiques sur les données intermédiaires existantes

— Comparer les résultats entre deux versions d'un même stage

2.12.3 Pattern "Gate de Qualité"

Avant l'upload final vers la KB, un gate de validation bloque la progression si les données sont non conformes :

```
Pipeline OK → validate_schemas.py → PASS → Upload autorisé
                                  → FAIL → Upload bloqué, alerte
```

Ce pattern empêche la propagation de données corrompues vers la production.

Chapitre 3

Analyse des Besoins et Conception

Introduction

Ce chapitre traduit la problématique et les fondements théoriques des chapitres précédents en une **conception concrète** : il identifie les acteurs et les besoins du système, justifie le choix méthodologique (Scrum adapté et CRISP-DM — en le comparant explicitement aux méthodologies alternatives), détaille comment le pipeline s'adapte aux contraintes imposées par la plateforme de retrieval, puis présente l'architecture cible retenue. Ce chapitre couvre principalement les phases **Business Understanding** (besoins, acteurs) et **Modeling** (choix méthodologique, conception orientée retrieval) de CRISP-DM.

3.1 Identification des Acteurs

Le système implique plusieurs catégories d'acteurs aux rôles et attentes distincts :

Acteur	Rôle	Interaction avec le système
Ingénieur STM32 (client/interne)	Utilisateur final du chatbot	Pose des questions techniques, reçoit des diagnostics sourcés avec liens GitHub
Équipe Support ST	Mainteneur de la KB	Lance le pipeline, valide les exports, surveille le Quality Index
Administrateur KB	Gestionnaire de la plateforme	Crée les datasources, configure les personas et tools sur ST ChatGPT

Pipeline automatisé	Agent de maintenance	Ingère périodiquement les nouvelles issues, met à jour la KB, déclenche les validations
GitHub API	Source de données	Fournit les issues, PRs, commits, fichiers via l'API REST
Alfred Vision AI	Service d'enrichissement	Décrit automatiquement les images et figures PDF

La Figure 3.1 positionne ces acteurs autour du système : les acteurs **humains** (ingénieur, support, administrateur) à gauche, le **cœur logiciel** (pipeline + KB + persona) au centre, et les **services externes** (GitHub API, Alfred Vision) à droite.

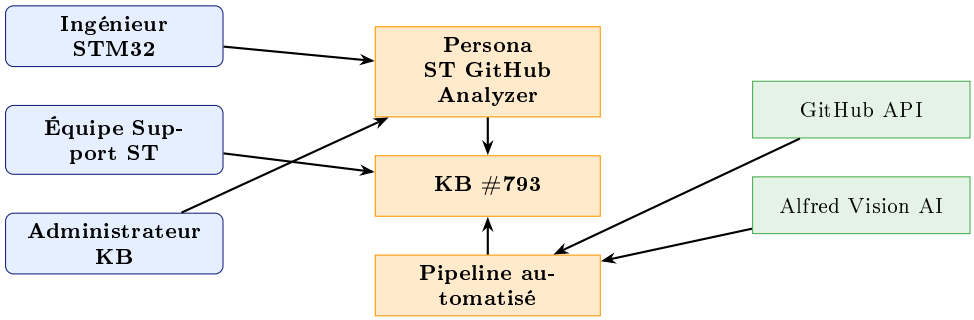


FIGURE 3.1 – Positionnement des acteurs autour du système : humains (gauche), cœur logiciel (centre), services externes (droite)

3.2 Besoins Fonctionnels

ID	Besoin	Priorité	Critère d'acceptation
BF1	Ingérer automatiquement les issues, PRs, commits et fichiers depuis GitHub	Critique	100% des issues récupérées pour un repo donné
BF2	Nettoyer le contenu textuel en supprimant le bruit (bots, HTML, licences)	Critique	Ratio signal/bruit $> 90\%$
BF3	Enrichir les issues avec des métadonnées structurées (layer, component, board, severity)	Haute	Couverture layer $\geq 85\%$, component $\geq 70\%$
BF4	Calculer la similarité inter-issues et établir des liens	Moyenne	Liens pertinents (précision $> 80\%$)

BF5	Générer 4 types d'artefacts delivery (issues, files, resolver cases, diagnostic cards)	Critique	Conformité 100% au JSON Schema
BF6	Uploader les artefacts vers la KB #793 sur ST ChatGPT	Critique	Datasources créées et indexées
BF7	Décrire automatiquement les images des issues par Alfred Vision	Haute	Taux de succès $\geq 95\%$
BF8	Décrire automatiquement les figures des PDFs de Release Notes	Moyenne	Intégration dans le texte livré
BF9	Valider les artefacts contre un JSON Schema avant upload	Critique	Gate bloquant en cas d'erreur
BF10	Fournir un chatbot IA qui répond aux questions STM32 avec citations	Critique	Quality Index $\geq 80/100$

3.3 Besoins Non Fonctionnels

ID	Besoin	Catégorie	Mesure
BNF1	Le pipeline doit être rejouable par étape sans effet de bord	Modularité	Chaque stage indépendant avec entrée/sortie JSON
BNF2	Le pipeline doit traiter une série complète en moins de 30 minutes	Performance	Temps mesuré sur H7 (12 repos, 500 issues)
BNF3	Le pipeline doit être reproductible (mêmes entrées $\rightarrow$ mêmes sorties)	Idempotence	Tests de non-régression
BNF4	Le code doit être maintenable et extensible à de nouvelles séries	Maintenabilité	Ajout d'une série en 3 étapes documentées

BNF5	Les données sources proviennent des dépôts GitHub ST ; la KB et la plateforme restent internes	Sécurité / Confidentialité	Aucune donnée personnelle traitée au-delà des identifiants publics
BNF6	Le système doit supporter 25+ séries MCU sans refactoring	Scalabilité	Configs séparées par série
BNF7	La mise à jour de la KB doit être automatisable	Automatisation	Workflow CI/CD fonctionnel
BNF8	Le chatbot doit répondre en moins de 15 secondes	Temps de réponse	Mesure via API ST ChatGPT

3.4 Contraintes et Décisions de Conception

Le projet est soumis à plusieurs contraintes qui ont orienté les choix techniques :

Contrainte	**Impact**	**Décision prise**
Rate limiting GitHub API (5000 req/h)	Limite la vitesse d'ingestion	Sleep adaptatif + retry exponentiel
Plateforme ST ChatGPT (Azure OpenAI)	Format et API imposés	Export JSON avec processor params base64
Pas de données d'entraînement supervisées	Impossible d'utiliser du ML classique pour la classification	Heuristiques regex domain-specific
Headers CMSIS > 1.7 Mo	Explosion de la taille des chunks et bruit dans le retrieval	Filtre V3 intelligent (−96%)
Images opaques dans les issues	Contenu diagnostique invisible pour la recherche textuelle	Enrichissement Alfred Vision AI
Multi-séries (25+ repos)	Risque de dilution du contexte dans la KB	Datasources séparées par série + métadonnées de filtrage

3.5 Approche Méthodologique

Le projet suit une approche itérative :

1. **Phase 1** : Pipeline H7 (série pilote, 12 repos)
2. **Phase 2** : Extension multi-séries (F4, H5, U5, WL)
3. **Phase 3** : Enrichissement IA (Alfred Vision) et évaluation

3.5.1 Sélection de la Méthodologie : Scrum Adapté

Le projet a été conduit selon une méthodologie **Scrum adaptée** au contexte d'un projet de fin d'études, avec des sprints de 2 semaines et des revues régulières avec l'encadrant entreprise.

Organisation en Sprints

Sprint	Période	Objectifs et livrables
Sprint 1	Fév – Mars 2026	Étude de l'existant, état de l'art RAG/NLP, POC ingestion H7
Sprint 2	Mars 2026	Pipeline H7 complet (ingestion → delivery), première version KB
Sprint 3	Avr 2026	Création persona ST ChatGPT, calibration recherche hybride, premiers Quality Index
Sprint 4	Avr – Mai 2026	Diagnostic Cards + Resolver Cases, nettoyage V3, QI = 94 sur H7
Sprint 5	Mai 2026	Extension multi-séries (F4, H5, U5, WL), étude comparative Alfred vs Persona
Sprint 6	Mai – Juin 2026	Alfred Vision (images issues + PDF), enrichissement 13 séries
Sprint 7	Juin 2026	Nouvelles séries (G4, L0, L1, L4), automatisation CI/CD, évaluation finale

La Figure 3.2 présente la frise des 7 sprints et met en évidence les jalons majeurs : pipeline H7 complet (S2), première persona (S3), saut de Quality Index à 94 grâce aux Diagnostic Cards (S4), et extension multi-séries (S5–S7).

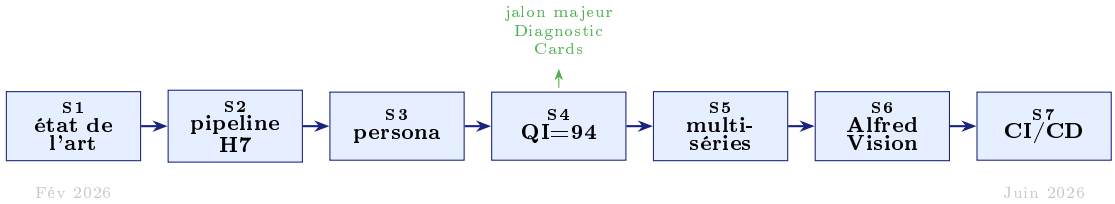


FIGURE 3.2 – Frise des 7 sprints Scrum et jalons majeurs du projet

Artefacts Scrum Utilisés

- **Product Backlog** : liste priorisée des fonctionnalités (pipeline stages, types d'artefacts, enrichissement IA)
- **Sprint Backlog** : sélection des tâches pour chaque sprint de 2 semaines
- **Daily Stand-up** : point quotidien avec l'équipe ST (15 min)
- **Sprint Review** : démonstration des livrables à l'encadrant (Quality Index, démo chatbot)
- **Sprint Retrospective** : analyse des blocages et améliorations (ex : API rate limiting, JSON corruption)

3.5.2 Pourquoi CRISP-DM ? Choix et Justification Méthodologique

Le choix d'une méthodologie n'est jamais neutre : il structure la façon dont le problème est cadré, dont les données sont analysées, et dont la valeur du travail est démontrée. Plusieurs cadres méthodologiques auraient pu s'appliquer à ce projet. Cette section les compare explicitement à **CRISP-DM** et justifie pourquoi ce dernier a été retenu comme principe organisateur du rapport et de la démarche de conception, malgré l'absence d'un modèle prédictif entraîné.

Les alternatives considérées :

- **Agile/Scrum pur (sans surcouche data)** : Scrum organise *le temps de travail* (sprints, revues, rétrospectives) mais ne dit rien sur *comment raisonner sur les données* — il ne prescrit ni analyse de la qualité des sources, ni méthode de validation d'un traitement de données, ni notion de *Business Understanding* distincte du backlog produit. Nous l'utilisons (Section 3.5.1) pour le **pilotage temporel** du projet, mais il ne peut pas, à lui seul, structurer la **démarche data** (compréhension du corpus GitHub, choix des heuristiques, validation de la qualité).
- **KDD** (*Knowledge Discovery in Databases*) : cadre historique (Fayyad et al., 1996) centré sur la découverte de *patterns* dans de grandes bases de données, en 5 étapes

(sélection, prétraitement, transformation, data mining, interprétation/évaluation). KDD est plus **académique et exploratoire** que CRISP-DM : il ne formalise pas la phase de *Business Understanding* ni la phase de *Deployment*, deux dimensions pourtant essentielles ici puisque notre projet a une finalité industrielle explicite (un chatbot déployé en production, avec des utilisateurs réels et un cadrage métier fort dès le départ).

— **SEMMA** (*Sample, Explore, Modify, Model, Assess*) : méthodologie propriétaire SAS, centrée sur la préparation et la modélisation statistique d'un échantillon de données en vue d'un modèle prédictif. SEMMA suppose implicitement un pipeline *modélisation supervisée classique* (échantillonnage, exploration statistique, modélisation, évaluation d'un score) — un cadre mal adapté à un projet où il n'y a ni échantillonnage statistique, ni modèle à évaluer par une métrique de type AUC/RMSE, mais un pipeline de transformation texte-vers-artefacts alimentant un moteur de retrieval.

— **TDSP** (*Team Data Science Process*, Microsoft) : cadre plus récent et plus proche de CRISP-DM (il en reprend d'ailleurs largement l'esprit), avec un accent fort sur les **rôles d'équipe**, l'**infrastructure cloud** et l'**intégration CI/CD**. TDSP est pertinent pour de grandes équipes data science avec une gouvernance formalisée (rôles de *Project Lead*, *Data Scientist*, *Solution Architect* distincts). Pour un projet de fin d'études mené par une seule personne, cette granularité organisationnelle est surdimensionnée ; en revanche, ses bonnes pratiques de *Deployment* et de CI/CD ont été reprises **dans notre pipeline** (Chapitre 5), sans adopter tout le cadre.

— **Approche ad hoc / empirique** : consisterait à ne suivre aucun cadre formel et à documenter le travail au fil de l'eau. Cette option a été écartée car elle aurait rendu difficile la démonstration, devant un jury, que **chaque décision technique répond à un besoin métier identifié** — exactement ce que CRISP-DM impose de justifier à travers ses six phases.

Grille de comparaison synthétique :

Critère	**KDD**	**SEMMA**	**CRISP-DM**
Phase métier explicite (*Business Understanding*)	Absente	Absente	**Présente et structurante**
Phase de déploiement explicite	Implicite (interprétation)	Absente	**Présente** (Deployment)

Neutralité vis-à-vis du secteur/outil	Oui	Non (liée à SAS)	**Oui** (norme ouverte, inter-industries)
Adoption industrielle et reconnaissance	Académique	Niche (SAS)	**Standard de facto le plus répandu**
Adaptabilité à un projet non-supervisé/retrieval	Moyenne	Faible	**Bonne** (phase *Modeling* réinterprétable, cf. Section 3.5.3)

Pourquoi CRISP-DM l'emporte pour ce projet : trois arguments déterminants ont guidé ce choix.

1. **Un cycle complet, du besoin métier au déploiement.** Contrairement à KDD et SEMMA, CRISP-DM impose de partir d'un *Business Understanding* explicite (la problématique du support STM32, Section 1.5) et de refermer la boucle par un *Deployment* réel (upload KB, persona ST ChatGPT, CI/CD). Notre projet a précisément cette forme : un problème métier identifié en amont, une chaîne de transformation de données, et une mise en production effective mesurée en conditions réelles. Aucune autre méthodologie candidate ne couvre ce cycle complet aussi naturellement.
2. **Une phase *Modeling* suffisamment flexible pour être réinterprétée honnêtement.** SEMMA et KDD supposent implicitement un modèle statistique ou un algorithme de *data mining* au centre du processus ; les y substituer par nos décisions de conception retrieval (similarité TF-IDF, heuristiques, chunking `PARENT_CHILD`) serait plus artificiel qu'avec CRISP-DM, dont la littérature reconnaît explicitement des usages élargis à des projets ne produisant pas de modèle prédictif au sens strict, tant que la logique itérative de valorisation des données est respectée (cf. justification détaillée en Section 3.5.3).
3. **Un standard reconnu, communicable et vérifiable par un jury.** CRISP-DM est le cadre le plus largement enseigné et audité dans les formations en science des données ; l'utiliser permet de **structurer la soutenance** autour d'un référentiel que le jury connaît déjà, et de démontrer une maîtrise méthodologique transférable au-delà de ce projet spécifique — un avantage qu'un cadre propriétaire (SEMMA) ou strictement académique (KDD) n'offre pas.

Conclusion : CRISP-DM n'est pas retenu par défaut ou par habitude, mais à l'issue d'une comparaison explicite qui a écarté KDD (trop académique, sans Deployment),

SEMMA (trop liée à la modélisation statistique propriétaire) et l'approche ad hoc (non auditable). Scrum reste utilisé en complément pour le pilotage temporel (Section 3.5.1), les deux cadres étant **complémentaires et non concurrents** : Scrum organise le *comment on travaille dans le temps*, CRISP-DM organise le *comment on raisonne sur les données et la valeur produite*. La section suivante détaille précisément comment chacune des six phases de CRISP-DM a été adaptée à la nature data-centric et retrieval-centric de ce projet.

3.5.3 Adaptation de CRISP-DM au Projet

Le projet est piloté par la méthodologie **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), retenue à l'issue de la comparaison méthodologique de la Section 3.5.2. Cependant, CRISP-DM a été conçu à l'origine pour des projets de *data mining* et d'apprentissage supervisé, où la phase centrale (*Modeling*) consiste à entraîner un modèle prédictif. Or, **notre projet n'entraîne aucun modèle prédictif** : il s'agit d'un projet **orienté données** (data-centric) et **orienté recherche documentaire** (retrieval-centric). Cette section explicite l'adaptation de CRISP-DM à ce contexte particulier, et justifie pourquoi cette réinterprétation est appropriée.

Nature du Projet : Data-Centric et Retrieval-Centric

Le cœur du projet n'est pas la prédiction mais la **transformation et la structuration de connaissances** pour un système de recherche existant (RAG déployé sur la plateforme ST ChatGPT). La valeur ne provient pas d'un modèle appris sur des données étiquetées, mais de la **qualité du prétraitement** et de la **compatibilité des artefacts produits** avec l'architecture de retrieval imposée par la plateforme.

En conséquence, la phase *Modeling* de CRISP-DM ne peut pas être interprétée au sens classique (entraînement d'un classifieur ou d'un régresseur). Nous la réinterprétons comme une phase de **conception algorithmique et de structuration orientée retrieval** regroupant :

— la conception de la similarité **TF-IDF** inter-issues (définie en Section 5.1.2) et le choix du seuil décisionnel ;

— les **heuristiques** de classification des métadonnées (layer, component, board, severity) ;

— la **stratégie de chunking** compatible avec le découpage `PARENT_CHILD` de la plateforme (Section 3.7) ;

— la **compatibilité avec le retrieval hybride** (semantic + full-text) ;

— le **routage de fallback** KB-first (KB → NovaX) ;

— la conception des **formats d'artefacts KB-ready** (issues, files, resolver cases, diagnostic cards).

Justification de l'adaptation : cette réinterprétation est appropriée car elle respecte l'esprit de CRISP-DM (un processus itératif de valorisation des données) tout en reflétant honnêtement la réalité technique du projet. Prétendre qu'un modèle supervisé existe serait un contresens méthodologique ; à l'inverse, ignorer la phase *Modeling* masquerait les décisions de conception les plus importantes (seuils, chunking, formats). Nous conservons donc les six phases de CRISP-DM comme **principe organisateur**, en explicitant la nature de chaque phase dans notre contexte.

Correspondance CRISP-DM → Pipeline

Le tableau suivant établit la correspondance explicite entre les six phases de CRISP-DM et les étapes réelles de notre pipeline :

Phase CRISP-DM	**Interprétation dans le projet**	**Étapes / livrables**
Business Understanding	Cadrage du problème de support STM32 et objectifs métier	Ch. 1 : problématique, réduction du temps de recherche, traçabilité, consolidation des connaissances GitHub
Data Understanding	Analyse du corpus brut GitHub	Ch. 1 : issues, PRs, commits, fichiers, Release Notes, captures, figures PDF ; volume, bruit, hétérogénéité, métadonnées manquantes
Data Preparation	Prétraitement multi-stage	Stages 1–3, 5–7 : ingestion, nettoyage V1/V3, enrichissement, chunking, validation, delivery
Modeling *(réinterprété)*	Conception algorithmique et structuration orientée retrieval	Similarité TF-IDF + seuil, heuristiques métadonnées, stratégie de chunking `PARENT_CHILD`, formats d'artefacts KB-ready, routage KB-first

Evaluation	Mesure de qualité end-to-end	Ch. 5 : Quality Index, métriques par stage, confiance Alfred, étude comparative
Deployment	Mise en production sur la plateforme	Ch. 4 (Section 4.11) : upload KB #793, persona ST ChatGPT, recherche hybride, workflow CI/CD automatisé

Lecture : la phase *Modeling* ne produit pas un modèle entraîné mais un ensemble de **décisions de conception** (seuils, découpage, formats) qui déterminent comment la connaissance est structurée pour être exploitable par le retrieval de la plateforme. Chaque section technique du rapport rappelle explicitement la phase CRISP-DM à laquelle elle appartient.

3.6 Notre Pipeline : Mapping Théorie → Implémentation

Le tableau suivant fait le lien entre les étapes canoniques présentées au Chapitre 2 et notre implémentation concrète :

Étape Canonique	**Stage Pipeline**	**Implémentation**
Collecte	Stage 1 (Ingestion)	`fetch_issues.py`, `fetch_files.py`, `repo_sync_service.py`
Nettoyage	Stage 2 (Cleaning)	`clean_issues.py`, `clean_files.py` (V1), `clean_files_v3.py` (V3 par type)
Enrichissement	Stage 3 (Enrichment)	`issues_to_docs_v2.py` — heuristiques NLP (layer, severity, board, component)
Mise en Relation	Stage 4 (Similarity)	`compute_issue_similarity_v2.py` — TF-IDF cosine, top-k voisins

Découpage	Stage 5 (Chunking)	`docs_to_chunks_*_v2.py` — validation locale ; en production, découpage `PARENT_CHILD` (1000/3000) par la plateforme
Validation	Stage 6 (Evaluation)	`validate_schemas.py`, `test_stats_*_v2.py`
Export	Stage 7 (Delivery)	`export_st_ready_*.py` — 4 types d'artefacts JSON

3.7 Contraintes de la Plateforme de Retrieval et Adaptation du Pipeline

Un point méthodologique essentiel de ce projet est que **l'architecture de retrieval n'a pas été conçue par le projet**. La plateforme ST ChatGPT impose une configuration d'indexation et de recherche **préexistante et non modifiable**. Le pipeline de prétraitement a donc été **conçu pour s'adapter à ces paramètres imposés**, et non l'inverse. Cette section définit chaque paramètre, explique la contrainte qu'il impose, décrit ce que le pipeline a dû faire pour être compatible, et démontre la valeur ajoutée de chaque choix. Cette section relève de la phase **Modeling** de CRISP-DM (conception orientée retrieval, cf. Section 3.5.3).

3.7.1 Paramètres Imposés par la Plateforme

La configuration d'indexation de la KB #793 est fixée par la plateforme :

Paramètre	**Valeur imposée**	**Signification**
`chunkStrategy`	`PARENT_CHILD`	Découpage hiérarchique à deux niveaux (parent contextuel, enfant précis)
`childChunkSize`	1000	Taille (caractères) des chunks enfants indexés pour la recherche
`parentChunkSize`	3000	Taille (caractères) des chunks parents restitués comme contexte
Modèle d'embedding	`AdaV2`	`text-embedding-ada-002` : vectorisation sémantique (1536 dimensions)

`jobBatchSize`	60	Nombre de documents traités par lot d'indexation
`embeddingsBatchSize`	20	Nombre d'embeddings calculés par appel d'API

Position méthodologique : ces valeurs sont des **données d'entrée** du projet. Le pipeline ne les redéfinit pas ; il structure la connaissance de façon à ce que le retrieval de la plateforme soit le plus performant possible *sous ces contraintes*.

3.7.2 Découpage PARENT_CHILD

Définition : la stratégie `PARENT_CHILD` (aussi appelée *small-to-big retrieval*) découpe chaque document en deux niveaux. Les **chunks enfants** (petits, 1000 caractères) sont vectorisés et indexés : ils servent de **cibles de recherche** précises. Les **chunks parents** (plus grands, 3000 caractères) englobent plusieurs enfants et sont **restitués comme contexte** au LLM lorsqu'un enfant correspond à la requête.

	Chunk enfant (1000)	**Chunk parent (3000)**
Rôle	Cible de recherche (matching précis)	Contexte restitué au LLM
Avantage	Précision : match sur un passage exact	Complétude : contexte suffisant pour répondre
Indexé (embeddings) ?	Oui	Non (restitué seulement)

Contrainte imposée : le retrieval matche sur des fragments de 1000 caractères mais répond avec 3000 caractères. Un découpage aveugle (coupe au milieu d'une phrase, d'une fonction ou d'un log) produit des chunks enfants non auto-suffisants, dont l'embedding est bruité, et des chunks parents incohérents.

Ce que le pipeline a dû faire :

— Produire des **unités documentaires auto-suffisantes** (une issue = un symptôme + root cause + fix ; une diagnostic card = un problème complet) dont le sens survit à un découpage à 1000/3000 caractères.

— **Placer les termes discriminants en tête** de document (titre, périphérique, code d'erreur), pour qu'ils tombent dans le premier chunk enfant.

— **Éviter les documents monolithiques** : un header CMSIS brut de 1,4 Mo générerait des centaines de chunks enfants sans valeur diagnostique — d'où le filtrage V3 (−96%).

— Structurer les artefacts pour que chaque **chunk parent reste thématiquement cohérent** (pas de mélange de deux issues dans un même parent).

Valeur ajoutée : nos artefacts (issues structurées, diagnostic cards) sont naturellement compatibles avec le *small-to-big retrieval* car ils sont déjà organisés en unités sémantiques bornées. Le pipeline transforme du texte brut fragmenté en documents dont la granularité correspond à celle attendue par `PARENT_CHILD`.

Lien CRISP-DM : phase *Modeling* (conception du découpage) et *Data Preparation* (production des unités). **Lien évaluation** : cette compatibilité contribue au saut de Quality Index apporté par les Diagnostic Cards (+33 points, cf. Section 5.5), car leurs *query aliases* augmentent la probabilité qu'un chunk enfant corresponde à une reformulation utilisateur.

La Figure 3.3 illustre ce flux : l'artefact ST-ready est découpé en chunks enfants (1000 caractères) indexés par AdaV2 ; à la requête, l'enfant le plus proche est retrouvé, puis son chunk parent (3000 caractères) est restitué comme contexte au LLM.

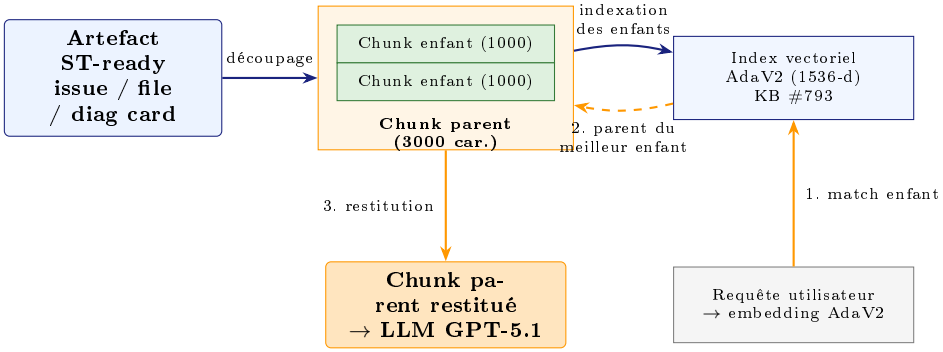


FIGURE 3.3 – Flux `PARENT_CHILD` : les chunks enfants (1000 car.) sont indexés par AdaV2 (phase hors-ligne, flèches bleues) ; à la requête (phase en ligne, flèches orange), l'enfant le plus proche est retrouvé puis son chunk parent (3000 car.) est restitué comme contexte au LLM.

3.7.3 Embeddings AdaV2 et Qualité Sémantique du Contenu

Définition : `AdaV2` (`text-embedding-ada-002`) est le modèle d'embedding utilisé par la plateforme pour vectoriser les chunks enfants. Il projette chaque texte dans un espace vectoriel de 1536 dimensions, où la proximité (similarité cosinus, définie en Section 5.1.2) reflète la proximité sémantique. **À distinguer du modèle de génération** (GPT-5.1), qui rédige la réponse finale : AdaV2 sert uniquement à l'**indexation et la recherche sémantique**.

Contrainte imposée : la qualité du retrieval sémantique dépend directement de la qualité des embeddings. Or un embedding est **dilué** par le bruit : si un chunk contient 70% de balises HTML, de licence ou de `#define` bit-à-bit, le vecteur résultant représente

majoritairement du bruit, et la recherche sémantique devient imprécise.

Ce que le pipeline a dû faire :

— **Maximiser la densité sémantique** de chaque chunk (nettoyage V3 : −96% CMSIS, −33% HTML) pour que l'embedding capture le signal technique et non le bruit structurel.

— **Préserver intacts les identifiants techniques** (noms d'API, registres, codes d'erreur) qui portent le sens diagnostique et doivent apparaître dans le texte vectorisé.

— **Enrichir le contenu visuel** (Alfred Vision) : une capture d'écran non décrite est invisible pour AdaV2 ; sa description textuelle la rend indexable sémantiquement.

Valeur ajoutée : le pipeline produit un contenu « **embedding-friendly** » — dense, propre, technique — qui exploite pleinement la capacité de discrimination sémantique d'AdaV2. C'est le lien direct entre le nettoyage V3 et la qualité du retrieval.

Lien CRISP-DM : *Data Preparation* (nettoyage, enrichissement) au service de la phase *Modeling* (compatibilité sémantique). **Lien évaluation** : la densité information/chunk augmentée de +31% (Section 5.3.2) améliore directement la pertinence des embeddings AdaV2.

3.7.4 Tailles de Lots (Batch Sizes) et Débit d'Indexation

Définitions :

— `jobBatchSize = 60` : la plateforme indexe les documents par lots de 60. C'est l'unité de traitement d'un *job* d'ingestion.

— `embeddingsBatchSize = 20` : les embeddings AdaV2 sont calculés par appels groupés de 20 textes, pour optimiser le débit de l'API de vectorisation.

Contrainte imposée : ces tailles de lots déterminent le **débit** et la **stabilité** de l'indexation. Des documents trop volumineux ou en nombre imprévisible peuvent provoquer des dépassements de quota, des timeouts, ou une indexation partielle instable.

Ce que le pipeline a dû faire :

— Produire des documents de **taille bornée et prévisible** (un header CMSIS de 1,4 Mo saturerait plusieurs lots à lui seul — d'où le filtrage V3).

— Générer des artefacts en **comptes maîtrisés par datasource et par série**, pour que l'upload se répartisse proprement en lots de 60.

— Exporter un **JSON par type d'artefact** découpable en unités homogènes, compatibles avec le calcul d'embeddings par groupes de 20.

— Adopter un workflow d'upload **Delete + Add** qui réindexe proprement sans conflit, garantissant une indexation déterministe.

Valeur ajoutée : en normalisant la taille et le nombre des documents, le pipeline assure une **indexation stable et reproductible** de la KB #793, sans dépassement de débit ni indexation partielle. Le respect implicite des tailles de lots est un effet direct de la discipline de taille imposée par le nettoyage et la structuration.

Lien CRISP-DM : phase *Deployment* (upload et indexation) préparée dès la *Data Preparation*. **Lien évaluation** : la stabilité d'indexation conditionne la reproductibilité des campagnes de Quality Index (Section 5.5).

3.7.5 Synthèse : Contrainte Plateforme → Décision Pipeline → Valeur

Paramètre plateforme	Contrainte	Décision pipeline	Valeur ajoutée
`PARENT_CHILD` (1000/3000)	Match sur enfants, contexte sur parents	Unités autosuffisantes, termes discriminants en tête	Compatibilité *small-to-big*, recall accru
`AdaV2`	Embeddings sensibles au bruit	Nettoyage V3, densité sémantique, Alfred Vision	Contenu embedding-friendly, retrieval précis
`jobBatchSize=60`	Indexation par lots	Documents bornés, comptes maîtrisés	Indexation stable
`embeddingsBatchSize=20`	Vectorisation groupée	Artefacts découpables homogènes	Débit maîtrisé, pas de timeout

Conclusion : le pipeline n'invente pas l'architecture de retrieval ; il **adapte le prétraitement et la structuration de la connaissance** à une configuration de plateforme imposée. Sa valeur ne réside pas dans la définition des paramètres (fixés par la plateforme) mais dans la **production d'artefacts optimalement compatibles** avec ces paramètres, ce qui améliore la qualité finale du retrieval et du support.

3.7.6 Valeur du Pipeline par Rapport à un Preprocessing RAG Standard

Un preprocessing RAG standard se limite généralement à : découper le texte brut, calculer les embeddings, indexer. Notre pipeline se distingue sur plusieurs axes, tout en

restant compatible avec la même plateforme :

Aspect	**Preprocessing RAG standard**	**Notre pipeline**
Nettoyage	Générique (strip basique)	Par type (V3) : HTML, CMSIS, licences
Métadonnées	Absentes ou minimales	Heuristiques domain-specific (layer, component, board, severity)
Unités documentaires	Chunks bruts	Artefacts structurés (issues, files, resolver, diagnostic)
Contenu visuel	Ignoré	Décrit par Alfred Vision (issues + PDF)
Traçabilité	Faible	Chaîne Issue→PR→Commit (resolver cases)
Compatibilité chunking	Fortuite	Conçue pour `PARENT_CHILD`

Valeur des artefacts structurés : les 4 types d'artefacts ne sont pas des chunks bruts mais des unités de connaissance conçues pour le retrieval. Les **diagnostic cards** (Symptôme → Root Cause → Fix) et les **resolver cases** (Issue → PR → Commit) apportent respectivement le raisonnement diagnostique et la traçabilité du correctif — deux dimensions absentes d'un preprocessing générique et directement responsables du gain de Quality Index mesuré (Section 5.5).

3.8 Architecture Cible

L'architecture cible du système s'articule en trois couches :

1. **Couche Data (Sources)** : les dépôts GitHub STMicroelectronics fournissent les données brutes via l'API REST. Les fichiers source sont récupérés par clonage local des repositories.
2. **Couche Processing (Pipeline)** : le pipeline Python en 7 étapes transforme les données brutes en artefacts structurés JSON. Chaque étape lit un fichier d'entrée et produit un fichier de sortie, permettant la rejouabilité et le débogage en isolation.
3. **Couche Delivery (Plateforme)** : les artefacts JSON sont uploadés vers la KB #793 sur la plateforme ST ChatGPT (Azure OpenAI). La persona "ST GitHub

Analyzer" interroge cette KB via une recherche hybride (semantic + full-text) et génère des réponses sourcées.

Note : le diagramme d'architecture détaillé est présenté en Figure 3.5 plus loin dans ce chapitre, et les diagrammes draw.io exportés sont référencés en Annexes.

3.9 Architecture Globale

Le système s'articule autour de 3 couches :

1. **Couche Data** : ingestion et stockage structuré (JSON)
2. **Couche Processing** : pipeline Python en 7 étapes
3. **Couche Delivery** : export et upload vers la KB cloud

3.10 Choix Techniques

Choix	Justification
Python 3.14	Écosystème riche (requests, scikit-learn), déploiement simple
JSON comme format pivot	Natif pour APIs REST, lisible, versionnable
TF-IDF (scikit-learn)	Léger, rapide, suffisant pour la similarité textuelle
Heuristiques vs ML	Déterministe, explicable, pas de données d'entraînement requises
Architecture modulaire	Chaque étape est rejouable indépendamment

3.11 Diagramme de Flux de Données

La Figure 3.4 illustre la vue globale du pipeline et le flux de données entre les 7 stages.

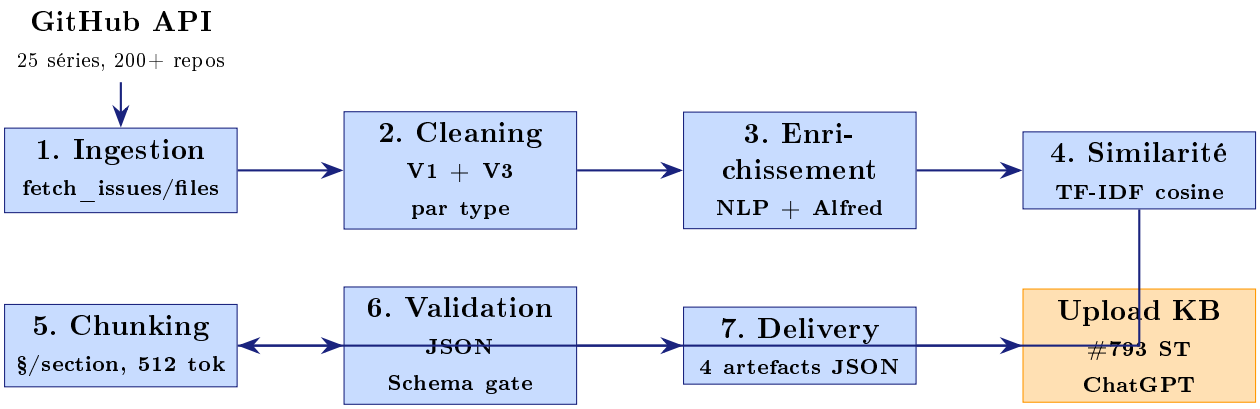


FIGURE 3.4 – Vue d'ensemble du pipeline de preprocessing (7 stages)

La Figure 3.5 détaille les connexions entre stages, les fichiers intermédiaires produits à chaque étape, et le branchement de l'enrichissement Alfred Vision AI.

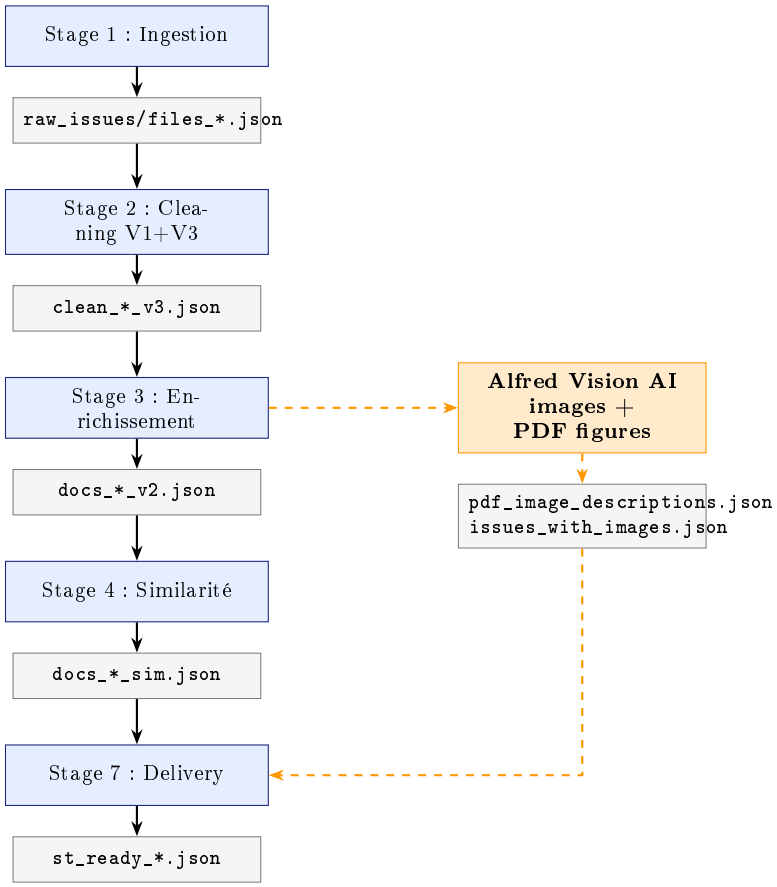


FIGURE 3.5 – Architecture détaillée du pipeline : stages, fichiers intermédiaires et branchement Alfred

Le diagramme montre le flux : GitHub (25 repos) → Ingestion (raw_*) → Cleaning V1/V3 (clean_*) → Enrichment (docs_*) → Similarity (_sim) → Validation → Delivery

(st_ready_*) → Upload KB #793 → ST ChatGPT. Alfred Vision intervient en parallèle pour décrire les images des issues et les figures PDF.

3.12 Flux Source-to-Answer

La Figure 3.6 montre le parcours complet d'une donnée depuis GitHub jusqu'à la réponse au client final.

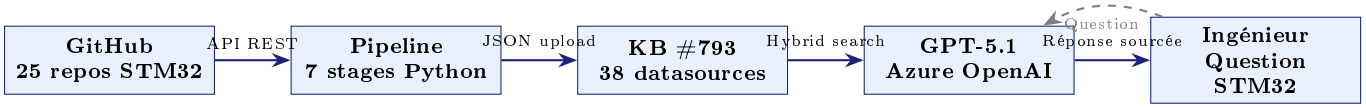


FIGURE 3.6 – Flux de données end-to-end : de la source GitHub à la réponse utilisateur

3.13 Architecture de Delivery et Upload KB

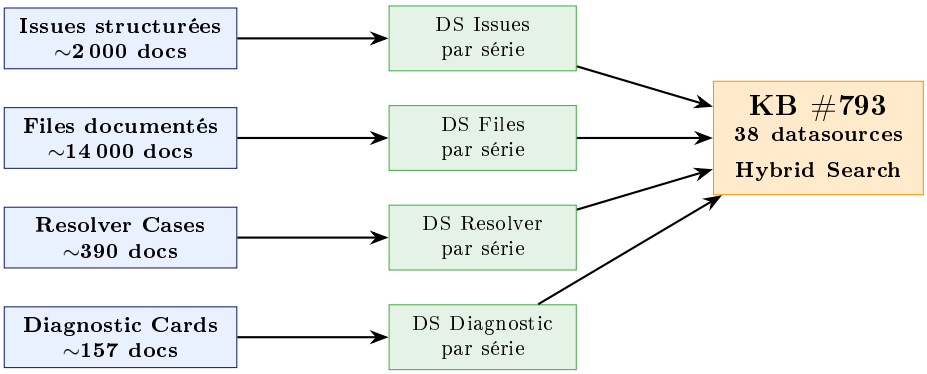


FIGURE 3.7 – Architecture de livraison et upload vers la Knowledge Base #793

Chapitre 4

Réalisation et Implémentation

Introduction

Phase CRISP-DM : *ce chapitre couvre principalement la phase Data Preparation, avec les décisions de Modeling (similarité, heuristiques, formats d'artefacts) et la charnière vers le Deployment (delivery, upload KB et configuration de la plateforme ST ChatGPT).*

Ce chapitre décrit la **réalisation concrète** du pipeline de prétraitement, c'est-à-dire le passage des concepts et de la conception (Chapitre 3) à un système Python exécutable, rejouable et automatisé. Chaque stage y est présenté selon un même schéma : objectif, mécanisme et scripts, exemple de sortie JSON réel, et résultats quantifiés. Il se conclut par la configuration de la plateforme ST ChatGPT qui exploite les artefacts produits.

4.1 Vue d'Orchestration du Pipeline

Avant de détailler chaque stage, la Figure 4.1 présente la **vue d'orchestration** : l'orchestrateur `run_full_workflow.py` enchaîne les 7 stages pour une série donnée, applique les *flags* d'exécution (`-skip-ingestion`, `-skip-chunking`, `-continue-on-error`, `-v3`), et boucle sur les 25 séries STM32. Cette vue complète le diagramme de flux de données (Figure 3.4) en montrant le **contrôle d'exécution** plutôt que le flux de données.

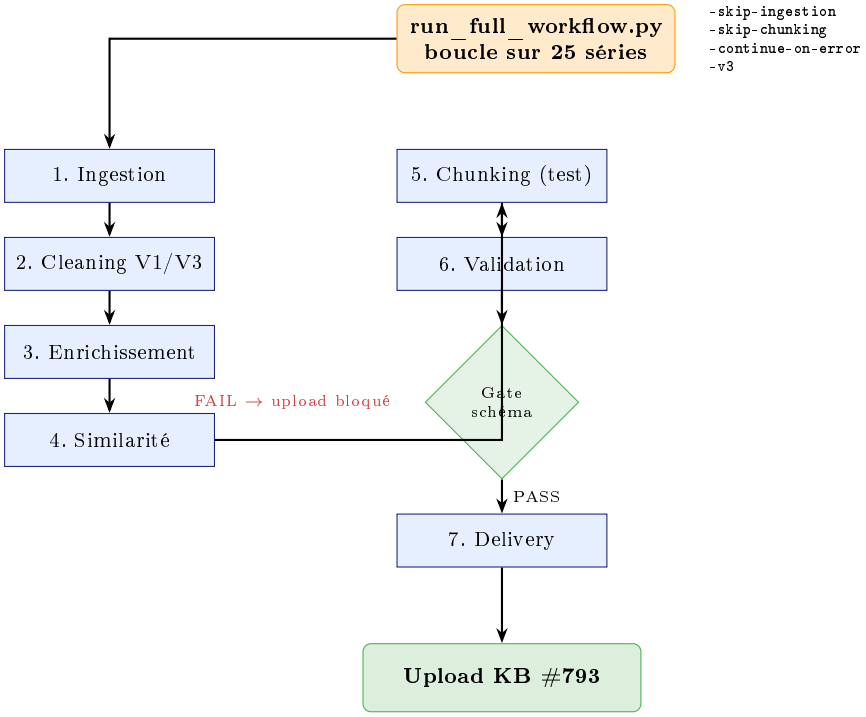


FIGURE 4.1 – Vue d'orchestration : `run_full_workflow.py` enchaîne les 7 stages avec gate de validation et flags d'exécution, en bouclant sur les 25 séries

Le Tableau 4.1 résume, pour chaque stage, le script principal, le fichier d'entrée, le fichier de sortie et la métrique de qualité associée. Il sert de fil conducteur au reste du chapitre.

TABLE 4.1: Tableau de synthèse des 7 stages du pipeline

Stage	Script principal	Entrée	Sortie	Métrique clé
1. Ingestion	`fetch_issues.py`, `fetch_files.py`	GitHub API	`raw_*.json`	Complétude 99,7%
2. Cleaning V1/V3	`clean_files_v3.py`	`raw_*.json`	`clean_*_v3.json`	CMSIS $-96,4\%$, corpus -38%
3. Enrichissement	`issues_to_docs_v2.py`	`clean_*.json`	`docs_*_v2.json`	Layer 88,2%, comp. 72,6%
4. Similarité	`compute_issue_similarity_v2.py`	`docs_*_v2.json`	`docs_*_sim.json`	Couverture voisins 22% (H7)
5. Chunking (test)	`docs_to_chunks_*_v2.py`	`docs_*_sim.json`	`chunks_*.json`	Cohérence `PARENT_CHILD`

Stage	Script principal	Entrée	Sortie	Métrique clé
6. Validation	`validate_schemas.py`	`st_ready_*.json`	Rapport gate	Conformité 100%
7. Delivery	`export_st_ready_*.py`	`docs_*_sim.json`	`st_ready_*.json`	4 artefacts / série

4.2 Ingestion (Stage 1)

Phase CRISP-DM : *Data Preparation (collecte exhaustive des données brutes, préalable à toute transformation).*

4.2.1 Objectif

Récupérer exhaustivement toutes les données des repositories GitHub configurés pour une série MCU donnée. La configuration est définie dans un fichier `config_XX.json` qui liste les repos à ingérer.

4.2.2 Mécanisme

L'ingestion s'appuie sur 5 scripts spécialisés :

- `fetch_issues.py` : récupère toutes les issues (ouvertes + fermées) avec leurs commentaires, via l'API GitHub REST avec pagination automatique (100 items/page)
- `fetch_prs.py` : récupère les Pull Requests et leurs reviews
- `fetch_commits.py` : récupère l'historique des commits pour le linking Issue→PR→Commit
- `fetch_issue_pr_commit_links.py` : construit les chaînes de traçabilité croisées
- `repo_sync_service.py` : clone/met à jour les repos localement pour l'extraction de fichiers
- `fetch_files.py` : parcourt l'arborescence des repos clonés et extrait le contenu des fichiers pertinents (README, Release Notes, sources, PDFs)

Gestion du rate-limiting : l'API GitHub autorise 5 000 requêtes/heure avec un token d'authentification. Le pipeline implémente un sleep adaptatif (0,2s entre les issues, 0,1s entre les commentaires) et un retry exponentiel en cas d'erreur 403/429.

4.2.3 Exemple de Sortie — Issue Brute

Listing 4.1 – Extrait de raw_issues_stm32cubeh7.json (issue brute)

```
{
  "number": 42,
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "state": "closed",
  "labels": ["bug"],
  "body": "When using HAL_SPI_TransmitReceive_DMA with...",
  "comments": [
    {"user": "dev123", "body": "Same issue on NUCLEO-H743ZI..."},
    {"user": "st-support", "body": "Fixed in v1.11.0..."}
  ],
  "created_at": "2023-06-15T10:30:00Z",
  "closed_at": "2023-08-20T14:00:00Z"
}
```

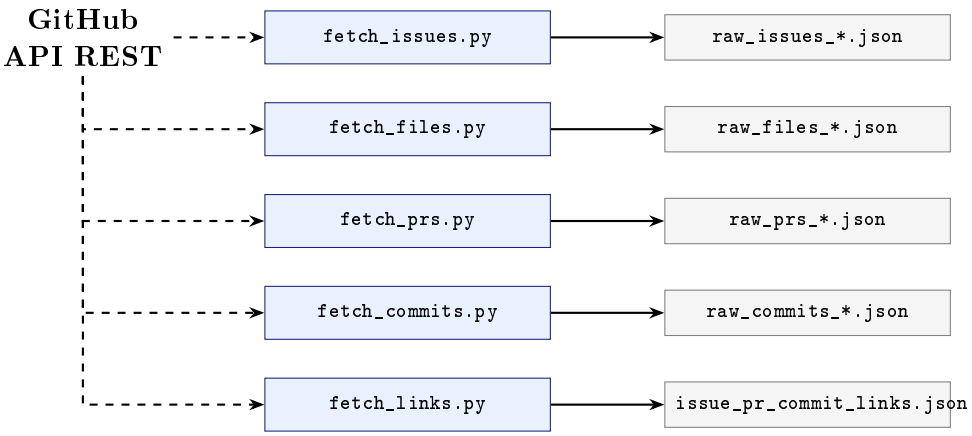


FIGURE 4.2 – Détail de l'étape d'ingestion : scripts et sorties

4.3 Nettoyage (Stage 2)

Phase CRISP-DM : *Data Preparation (élimination du bruit textuel et normalisation).*

Le nettoyage prépare les données brutes pour l'enrichissement en éliminant le bruit textuel :

4.3.1 Nettoyage des Issues

— Normalisation du texte (retours chariot, backticks, espaces)

— Extraction des images en markers `[IMAGE ATTACHED]` avec collecte des URLs

— Structuration : assemblage Title + Body + Comments en un seul champ `clean_text`

— Conservation des métadonnées brutes (labels, dates, état)

4.3.2 Exemple JSON — Issue Après Nettoyage

Listing 4.2 – Extrait de clean_issues_stm32cubeh7.json (issue nettoyée)


```
{
  "number": 42,
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "state": "closed",
  "labels": ["bug"],
  "clean_text": "HAL_SPI_TransmitReceive_DMA fails on STM32H743.
     When using HAL_SPI_TransmitReceive_DMA with circular mode and
     cache enabled, a hard fault occurs after the second transfer.
     [IMAGE ATTACHED: screenshot_debugger.png] Workaround: disable
     D-cache before SPI init. Same issue on NUCLEO-H743ZI with
     CubeH7 v1.10.0. Fixed in v1.11.0 by adding cache maintenance
     in HAL_SPI_TxRxCpltCallback.",
  "images_count": 1,
  "image_urls": ["https://user-images.githubusercontent.com/xxx/
     screenshot_debugger.png"],
  "created_at": "2023-06-15T10:30:00Z",
  "closed_at": "2023-08-20T14:00:00Z"
}
```


Transformation clé : le champ `clean_text` fusionne le titre, le body et tous les commentaires de l'issue en un seul texte contigu, prêt pour l'enrichissement NLP. Les images sont remplacées par des markers textuels avec leurs URLs collectées séparément pour l'enrichissement Alfred Vision.

4.3.3 Nettoyage des Fichiers

- Lecture du contenu (UTF-8, fallback Latin-1)
- Extraction du texte des PDFs (pypdf + rendu pages + détection figures OpenCV)
- Construction de l'URL GitHub pour chaque fichier
- Filtrage : seuls les fichiers pertinents sont retenus (README, Release Notes, source technique, documentation PDF)

4.3.4 Exemple JSON — Fichier Après Nettoyage

Listing 4.3 – Extrait de clean_files_stm32h7xx-hal-driver.json


```
{
```

```
  "path": "Src/stm32h7xx_hal_spi.c",
  "repo": "stm32h7xx-hal-driver",
  "file_type": "source_code",
  "url": "https://github.com/STMicroelectronics/stm32h7xx-hal-
     driver/blob/master/Src/stm32h7xx_hal_spi.c",
  "original_size": 89247,
  "clean_size": 80322,
  "is_valid": true,
  "clean_text": "HAL_StatusTypeDef HAL_SPI_Init(SPI_HandleTypeDef *
     hspi) { ... } HAL_StatusTypeDef HAL_SPI_TransmitReceive_DMA
     (...) { ... }"
}
```


Transformation clé : le bloc de licence en-tête (copyright STMicroelectronics, 20–40 lignes) est supprimé par `strip_license()`, réduisant le bruit de ∼10%. Le champ `is_valid` passe à `false` si le texte nettoyé fait moins de 100 caractères.

Note importante : Le nettoyage ne fait **aucune classification** (pas de détection de layer, board, component). Cette tâche est réalisée à l'étape suivante (Enrichissement).

4.4 Nettoyage V3 — Nettoyage Intelligent par Type (Stage 2b)

Phase CRISP-DM : *Data Preparation (nettoyage adaptatif par type de fichier au service de la qualité des embeddings AdaV2).*

Le nettoyage V1/V2 appliquait une stratégie uniforme à tous les fichiers. Or les sources ingérées sont hétérogènes : Release Notes en HTML, README en Markdown avec badges, fichiers source C avec licences volumineuses, et headers CMSIS de plus d'1 Mo. La version 3 introduit un **nettoyage par type** qui adapte la stratégie au contenu réel de chaque fichier.

4.4.1 Stratégies de Nettoyage par Type

Le script `clean_files_v3.py` route chaque fichier vers une stratégie spécialisée :

Type détecté	Stratégie	Transformations appliquées
Release Notes	`strip_html()`	Suppression balises HTML, conservation texte pur, normalisation espaces

README / .md	`clean_markdown()`	Suppression badges CI, liens images, normalisation titres
Source code (.c/.h)	`clean_source_code()`	Suppression bloc licence (copyright), conservation code technique
CMSIS device header	`filter_cmsis_device_header()`	Filtrage intelligent (voir ci-dessous)
Autres	Inchangé	Aucune transformation

La Figure 4.3 représente l'arbre de décision du routage V3, avec les taux de réduction mesurés pour chaque stratégie.

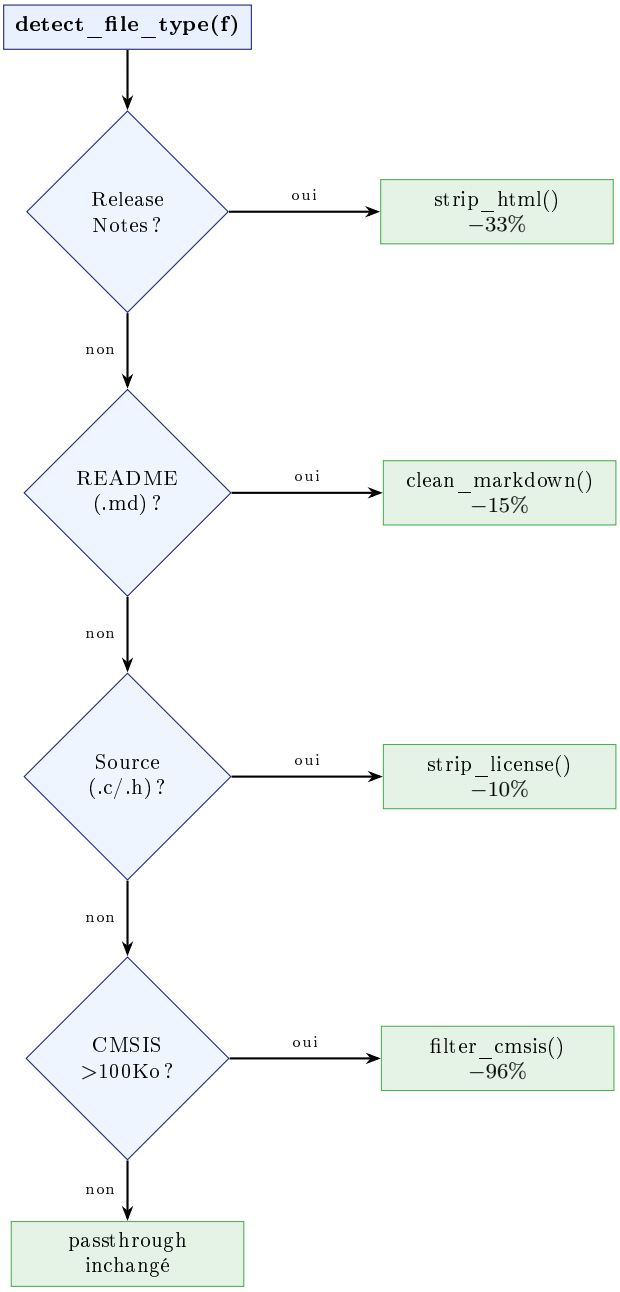


FIGURE 4.3 – Arbre de décision du nettoyage V3 : routage par type de fichier avec taux de réduction

L'arbre de décision route chaque fichier via `detect_file_type()` *vers la stratégie adaptée : release_notes → strip_html() (−33%), readme → clean_md() (−15%), source → strip_license() (−10%), cmsis >100 Ko → filter_cmsis() (−96%), other → passthrough.*

4.4.2 Filtrage CMSIS Device Headers

Les fichiers de type `stm32*xx.h` (CMSIS device) sont les plus volumineux du corpus (1–1.7 Mo). Ils contiennent principalement des définitions de registres bit-à-bit inutiles

pour le RAG. Le filtre V3 ne conserve que les sections à valeur diagnostique :

1. **Énumération IRQ** : liste des vecteurs d'interruption (`typedef enum IRQn`)
2. **Adresses de base** : adresses mémoire des périphériques (`#define PERIPH_BASE`)
3. **Structures TypeDef** : définitions de registres par périphérique (`typedef struct {...}`)

Seuil d'activation : le filtre ne s'applique qu'aux fichiers dépassant 100 Ko (les petits headers sont conservés intégralement).

4.4.3 Résultat du Filtrage — Exemple JSON

Voici un extrait du fichier `clean_files_cmsis_device_h7_v3.json` montrant un header CMSIS après filtrage V3 :

Listing 4.4 – Header CMSIS après filtrage V3 (extrait)

```
{
  "path": "Include/stm32h743xx.h",
  "repo": "cmsis_device_h7",
  "file_type": "cmsis_device_header",
  "original_size": 1458301,
  "clean_size": 52140,
  "reduction_pct": 96.4,
  "is_valid": true,
  "clean_text": "typedef enum { NonMaskableInt_IRQn = -14, ... }
     IRQn_Type; #define PERIPH_BASE 0x40000000UL ... typedef struct
      { __IO uint32_t CR1; __IO uint32_t CR2; ... } SPI_TypeDef;"
}
```

Le fichier passe de 1 458 301 caractères à 52 140 ($-96,4\%$), ne conservant que les IRQ, adresses de base et TypeDef.

4.4.4 Résultats du Nettoyage V3

Fichier	Taille V2	Taille V3	Réduction
`stm32h503xx.h`	1 074 499	36 499	-96.6%
`stm32h562xx.h`	1 722 072	65 470	-96.2%
`stm32h743xx.h`	1 458 301	52 140	-96.4%
Release Notes (moy.)	45 200	30 300	-33.0%

Sources avec licence	12 800	11 500	−10.2%

Impact global : Sur les 47 repos (5 séries MCU), le volume total des fichiers nettoyés est réduit de **38%** en moyenne, ce qui améliore directement la densité d'information des chunks et la pertinence du retrieval.

4.4.5 Compatibilité et Non-Régression

Le V3 n'écrase jamais les sorties V1/V2 :

- Fichiers d'entrée : `clean_files_<repo>.json` (V1) ou `clean_files_<repo>_v3.json` (V3)
- Les étapes aval (enrichissement, delivery) acceptent un flag `-v3` pour router automatiquement vers les fichiers V3
- Les datasources V3 sont uploadées vers des IDs séparés (29041–29045)

4.5 Enrichissement V2 (Stage 3)

Phase CRISP-DM : *Data Preparation dominante, avec une composante Modeling (conception des heuristiques de classification).*

4.5.1 Heuristiques de Classification

Le module `stm32cube_preprocessing_heuristics.py` applique des règles basées sur :

- **Regex patterns** : détection de séries MCU, composants, boards
- **Analyse des labels** : mapping label→layer (bug→bug_report, etc.)
- **Scoring pondéré** : longueur du body, présence de code, de logs d'erreur

4.5.2 Exemple de Classification

```
{
  "issue_number": 42,
  "title": "HAL_SPI_TransmitReceive DMA fails on STM32F446",
  "layer": "HAL",
  "component": "SPI",
  "board": "NUCLEO-F446RE",
  "issue_kind": "bug_report",
  "severity": "high",
  "evidence_strength": "high"
}
```

4.5.3 Résultats de Couverture des Métadonnées

La qualité de l'enrichissement se mesure par le **taux de couverture** de chaque champ, c'est-à-dire le pourcentage de documents pour lesquels l'heuristique a pu affecter une valeur non nulle. La Figure 4.4 présente ces taux mesurés sur la série H7.

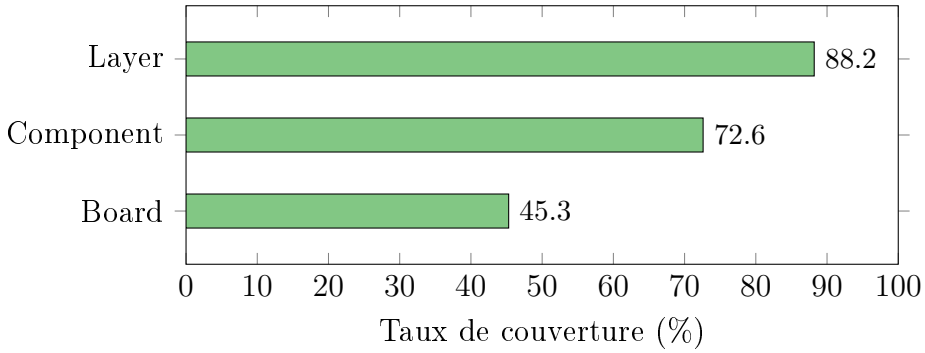


FIGURE 4.4 – Taux de couverture des métadonnées par heuristique NLP (série H7)

Lecture : la couche architecturale (`layer`) est identifiée dans 88,2% des cas et le composant dans 72,6%, ce qui suffit au filtrage facetté du retrieval. En revanche, la couverture `board` plafonne à 45,3% : il ne s'agit pas d'un défaut de l'heuristique, mais d'une **limite de la donnée source** — de nombreuses issues ne mentionnent aucun board explicite. Cette distinction entre défaut de pipeline et limite de source est reprise dans l'évaluation par stage (Chapitre 5).

4.6 Similarité (Stage 4)

Phase CRISP-DM : *Modeling (conception de la mesure de similarité TF-IDF et du seuil décisionnel).*

4.6.1 Algorithme

1. Construction de la matrice TF-IDF sur le corpus (titre + body + commentaires) :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{N}{|\{d' \mid t \in d'\}|} \tag{4.1}$$

2. Calcul de la similarité cosinus paire-à-paire :

$$\text{sim}(d_i, d_j) = \frac{\vec{v}_i \cdot \vec{v}_j}{\|\vec{v}_i\| \times \|\vec{v}_j\|} \tag{4.2}$$

3. Sélection des k plus proches voisins avec seuil $\theta = 0.3$:

$$\text{related}(d_i) = \{d_j \mid \text{sim}(d_i, d_j) \geq \theta,\ j \neq i\} \tag{4.3}$$

4.6.2 Exemple JSON — Issue avec Liens de Similarité

Listing 4.5 – Extrait de docs_issues_stm32cubeh7_v2_sim.json (avec voisins)

```
{
  "issue_number": 42,
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "similar_issues": [
    {"issue_number": 78, "similarity": 0.72, "title": "SPI DMA
       circular mode crash H750"},
    {"issue_number": 156, "similarity": 0.45, "title": "DMA cache
       coherency issue H743"},
    {"issue_number": 203, "similarity": 0.38, "title": "Hard fault
       after second SPI transfer"}
  ],
  "related_count": 3
}
```

Paramètres : le seuil $\theta = 0.3$ élimine les liens trop faibles (bruit). Sur H7, seulement 22% des issues ont au moins un voisin au-dessus de ce seuil, ce qui reflète la diversité des problèmes (chaque issue traite un bug unique). Le nombre k de voisins n'est pas borné — toutes les paires dépassant θ sont conservées.

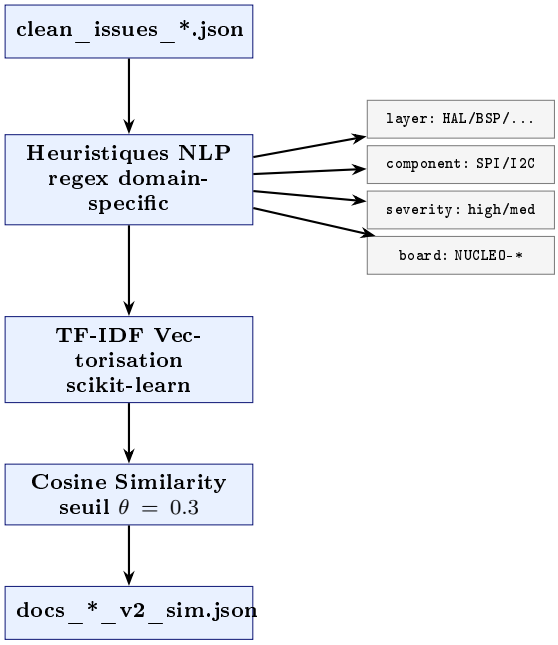


FIGURE 4.5 – Détail du processus d'enrichissement et de calcul de similarité

4.7 Chunking V2 (Stage 5 — Optionnel)

Phase CRISP-DM : *Data Preparation, avec une composante Modeling (adaptation au découpage* `PARENT_CHILD` *imposé par la plateforme, cf. Section 3.7.2).*

Note importante : le chunking est réalisé localement à des fins de **test et validation** uniquement. En production, la plateforme ST ChatGPT gère elle-même le découpage lors de l'indexation, selon la stratégie `PARENT_CHILD` imposée (chunks enfants de 1000 caractères indexés, chunks parents de 3000 caractères restitués — cf. Section 3.7.2). Le chunking local nous permet de :

— Vérifier la taille et la qualité des segments avant upload

— Tester la recherche TF-IDF en local (évaluation préliminaire)

— Vérifier que nos artefacts restent auto-suffisants sous un découpage à 1000/3000 caractères

Stratégie de découpage local (approximation du comportement plateforme) :

— Issues : chunks par paragraphe, alignés sur des unités sémantiques auto-suffisantes

— Files : chunks par section (README headers, fonctions C)

Le flag `-skip-chunking` dans le workflow permet de sauter cette étape pour les runs de production, car la plateforme applique elle-même le découpage `PARENT_CHILD` à l'upload. Le rôle du pipeline n'est donc pas de découper, mais de **produire des documents dont le découpage plateforme préserve la cohérence sémantique** (Section 3.7).

4.8 Delivery (Stage 7)

Phase CRISP-DM : *Data Preparation (production des artefacts finaux), charnière vers la phase Deployment (upload KB).*

4.8.1 Resolver Cases

Générés à partir d'issues fermées avec PRs liées :

1. Identification de la chaîne Issue → PR → Commits via `fetch_issue_pr_commit_links.py`
2. Évaluation du `resolution_status` :
   - `merged_fix` : PR mergée avec fichiers modifiés identifiés (confiance haute)
   - `candidate_fix` : PR liée mais non mergée ou lien faible (confiance moyenne)
   - `reference_only` : issue résolue sans PR formelle (information contextuelle)
3. Construction du texte structuré : Problème → Signal → Évaluation → Preuves

4.8.2 Exemple JSON — Resolver Case

Listing 4.6 – Extrait de st_ready_resolver_stm32cubeh7.json

```
{
  "issue_number": 42,
  "repo": "stm32cubeh7",
  "label": "SPI DMA TransmitReceive fails on STM32H743 in circular
     mode",
  "resolution_status": "merged_fix",
  "link_confidence": "high",
  "primary_pr_number": 87,
  "primary_changed_file": "Src/stm32h7xx_hal_spi.c",
  "resolver_card_text": "PROBLEM: HAL_SPI_TransmitReceive_DMA
     causes hard fault after second transfer in circular mode.
     SIGNAL: D-cache stale read on DMA buffer. EVALUATION: Cache
     maintenance missing in TxRxCpltCallback. FIX: Added
     SCB_InvalidateDCache_by_Addr() call in
     HAL_SPI_TxRxCpltCallback. PROOF: PR #87 merged 2023-08-15,
     verified on NUCLEO-H743ZI.",
  "impacted_modules": ["SPI", "DMA"],
  "impacted_boards": ["NUCLEO-H743ZI"]
}
```

Valeur ajoutée : contrairement aux issues brutes, le Resolver Case fournit la **chaîne de preuves complète** (issue → PR #87 → fichier modifié `stm32h7xx_hal_spi.c`) en un seul document. L'ingénieur peut vérifier le fix exact en suivant les liens.

4.8.3 Diagnostic Cards

Cartes de diagnostic structurées pour les problèmes récurrents :

1. Sélection des issues à haute valeur diagnostique (issues fermées avec root cause identifié, `evidence_strength` ≥ `medium`)
2. Génération : Symptôme → Cause Racine → Résolution → Validation
3. Ajout de **query aliases** (reformulations multiples du problème) pour améliorer le recall de la recherche RAG

4.8.4 Exemple JSON — Diagnostic Card

Listing 4.7 – Extrait de st_ready_diagnostic_stm32cubeh7.json

```
{
  "diagnostic_id": "DIAG-H7-042",
  "repo": "stm32cubeh7",
  "issue_number": 42,
  "symptom": "Hard fault occurs after second SPI DMA transfer in
     circular mode on STM32H743",
  "root_cause": "D-cache not invalidated before DMA reception. CPU
     reads stale data from cache instead of DMA-updated SRAM.",
  "resolution": "Call SCB_InvalidateDCache_by_Addr() before reading
      SPI RX buffer, or place DMA buffers in non-cacheable SRAM
     region (MPU configuration).",
  "affected_versions": ["CubeH7 v1.9.0", "CubeH7 v1.10.0"],
  "fixed_version": "CubeH7 v1.11.0",
  "query_aliases": [
    "SPI DMA hard fault STM32H7",
    "cache coherency SPI circular mode",
    "HAL_SPI_TransmitReceive_DMA crash",
    "D-cache DMA corruption H743"
  ],
  "evidence_strength": "high",
  "impacted_boards": ["NUCLEO-H743ZI", "STM32H743I-EVAL"]
}
```

Structure clé : les `query_aliases` sont le mécanisme principal d'amélioration du recall. Lorsqu'un utilisateur pose la question "mon SPI crash après le 2e transfert DMA", la recherche hybride retrouve cette Diagnostic Card grâce aux alias qui couvrent différentes formulations du même problème. C'est cette structure qui explique le saut de QI de 61 à 94.

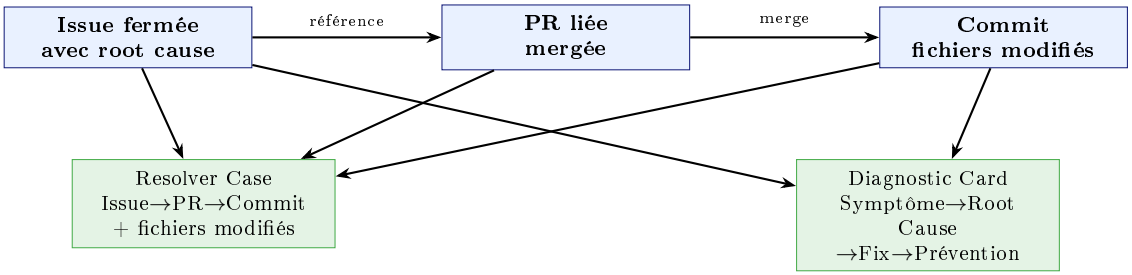


FIGURE 4.6 – Génération des Resolver Cases et Diagnostic Cards

La Figure 4.7 synthétise les relations entre les 4 types d'artefacts, leurs sources dans le pipeline, et leur destination dans la KB.

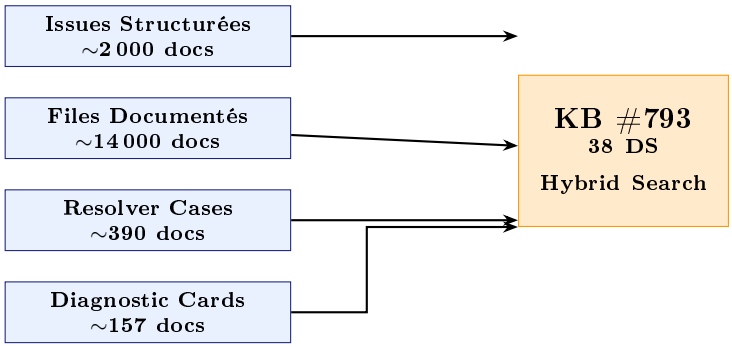


FIGURE 4.7 – Les 4 types d'artefacts delivery et leurs relations vers la KB #793

Le diagramme montre les 4 types d'artefacts : Issues Structurées (~2 000 docs), Files Documentés (~14 000 docs), Resolver Cases (~390 docs) et Diagnostic Cards (~157 docs), tous alimentés vers la KB #793 (38 datasources). Les Resolver Cases et Diagnostic Cards sont dérivés des Issues via les champs issue_number et metadata.

4.8.5 Alfred Vision — Enrichissement d'Images

De nombreuses issues GitHub contiennent des captures d'écran (erreurs IDE, oscillogrammes, configurations CubeMX). Ces images sont opaques pour la recherche textuelle RAG.

Le pipeline intègre **Alfred Vision AI**, un service LLM interne ST, pour convertir automatiquement les images en descriptions textuelles :

1. Le script d'export génère des markers `[IMAGE ATTACHED]` dans le texte

2. Les URLs des images sont extraites dans le champ `image_urls`
3. Alfred Vision télécharge chaque image et produit une description sémantique
4. Le marker est remplacé par `[IMAGE DESCRIPTION] ...`

Résultats H7 : 97/98 images décrites avec succès (taux de succès 98.9%).

Fix appliqué : La fonction `first_sentence()` supprime les markers `[IMAGE ATTACHED]` du résumé `root_cause_hint` pour éviter la duplication des markers entre les sections "Root cause" et "Key comments", ce qui causait des descriptions `UNAVAILABLE` par épuisement des URLs.

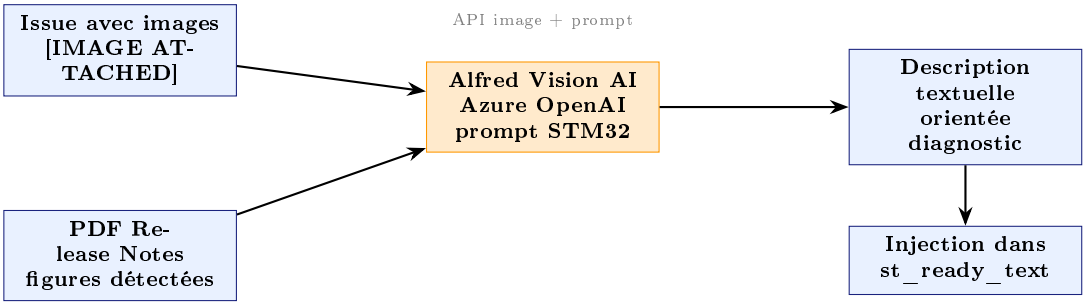


FIGURE 4.8 – Intégration Alfred Vision AI pour la description automatique d'images

4.8.6 Fiabilité Mesurée de l'Enrichissement Visuel

Produire une description n'est pas suffisant : encore faut-il qu'elle soit **cohérente avec le contexte** de l'image. Le script `validate_alfred_coherence.py` compare chaque description au texte source (`st_ready_text` pour les issues, `page_context_text` pour les PDF) et calcule un score de cohérence combinant recouvrement lexical, similarité sémantique et probabilité de contradiction. La Figure 4.9 compare la **confiance contextuelle** des deux sources visuelles.

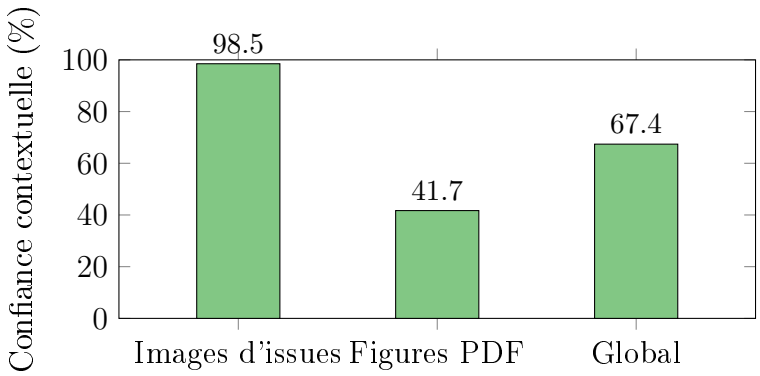


FIGURE 4.9 – Confiance contextuelle des descriptions Alfred : images d'issues (200 évaluées) vs figures PDF (242 évaluées)

Interprétation : sur 200 images d'issues évaluées, seules 3 sont signalées à faible cohérence, soit une confiance contextuelle de **98,5%** ; ces captures (logs IDE, écrans CubeMX, oscillogrammes) sont fortement ancrées dans le texte de l'issue. À l'inverse, sur 242 figures PDF, 141 sont à faible cohérence (confiance **41,7%**) : beaucoup de *crops* correspondent à des tableaux, historiques de révision ou fragments de page plutôt qu'à de vrais schémas. Conclusion opérationnelle : l'enrichissement des images d'issues est **mature**, tandis que celui des figures PDF est conservé comme **rappel multimodal à confiance limitée**.

4.9 Architecture Agents — Persona ST ChatGPT

Le système final s'articule autour d'une architecture **multi-agents** où chaque composant a un rôle bien défini.

4.9.1 Vue d'ensemble

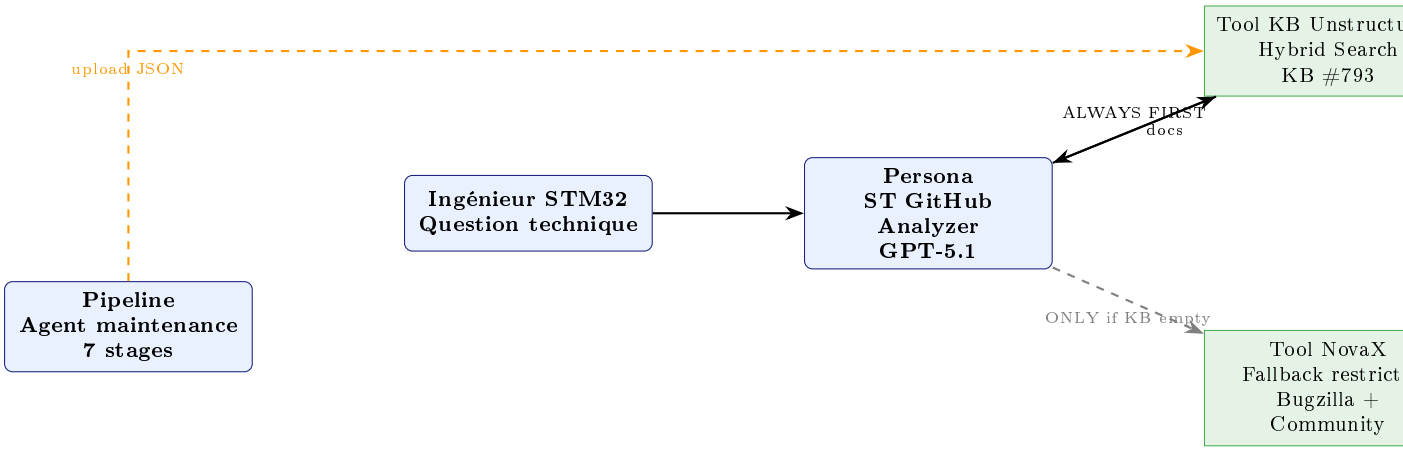


FIGURE 4.10 – Architecture multi-agents du système ST GitHub Analyzer

Le diagramme montre le flux : Utilisateur (Ingénieur STM32) → Persona ST GitHub Analyzer → KB Tool (Unstructured, recherche hybride) avec fallback vers NovaX Tool (si la KB ne retourne aucun résultat pertinent). Le Pipeline Preprocessing alimente la KB depuis GitHub STMicroelectronics via des uploads JSON.

4.9.2 Persona : ST GitHub Analyzer

La persona est un agent conversationnel déployé sur la plateforme **ST ChatGPT** (Azure OpenAI interne). Elle est configurée avec :

Paramètre	Valeur / Description
Nom	ST GitHub Analyzer
Modèle	GPT-5.1 (Azure OpenAI)
Température	0.2–0.3 (réponses factuelles)
KB associée	KB #793 (ST GitHub Analyzer)
Tool principal	ST-GitHub-Analyzer-Unstructured (Hybrid Search)
Tool fallback	NovaX (STM32 Bugzilla + STCommunity Knowledge Articles)
Search type	Hybrid (Semantic + Full-text)
Semantic threshold	0.55
Max documents	20 (semantic) + 12 (full-text)

4.9.3 Tool 1 : KB Unstructured (Recherche Hybride)

C'est le tool **principal**. Il interroge la KB #793 qui contient les artefacts générés par le pipeline :

— **Issues** : cartes structurées avec métadonnées, symptôme, root cause, fix

— **Files** : documentation HAL/BSP/CMSIS extraite des repositories

— **Resolver Cases** : chaînes Issue → PR → Commit avec preuves

— **Diagnostic Cards** : analyses de pannes structurées (symptôme → hypothèses → validation)

La recherche hybride combine :

1. **Semantic** : compréhension du sens ("mon SPI ne marche plus" → retrouve les issues SPI DMA)
2. **Full-text** : correspondance exacte ("HAL_SPI_Transmit_DMA" → match direct)

4.9.4 Tool 2 : NovaX Fallback

Quand la KB ne contient pas de réponse pertinente, la persona peut appeler **NovaX**, un agent STM32 basé sur STM32 Bugzilla et STCommunity Knowledge Articles.

Configuration critique — La description du tool doit **forcer la priorité KB** :

> *"I am a fallback assistant for STM32 firmware questions that are NOT covered by the Knowledge Base. The Knowledge Base is ALWAYS the primary source, search it FIRST for every question. Use this tool ONLY when the Knowledge Base returned no relevant result."*

Risque identifié : Si la description est trop permissive, la persona appelle NovaX au lieu de chercher dans la KB → réponses générales non ancrées dans les artefacts GitHub → dégradation du Quality Index. NovaX doit rester un secours pour les questions hors couverture KB.

4.9.5 Stratégie de Réponse (System Prompt)

Le system prompt de la persona impose une stratégie en 4 cas :

1. **Match direct KB** → citer la source avec URL GitHub
2. **Match partiel** → diagnostic + workaround candidat + étapes de validation
3. **Question configuration générale** → explication technique directe
4. **Aucun match** → appeler Alfred OU proposer une action concrète (jamais "je ne sais pas")

4.9.6 Pipeline comme Agent de Maintenance

Le pipeline de preprocessing joue lui-même le rôle d'un **agent de maintenance** de la KB :

Rôle	Description
Ingestion périodique	Détecte les nouvelles issues et mises à jour GitHub (bi-mensuel)
Enrichissement	Classifie automatiquement par heuristiques NLP (layer, component, severity)
Validation qualité	Gate de schéma bloquant les uploads de données corrompues
Mise à jour KB	Delete + Re-add automatique via GitHub Actions
Évaluation continue	Quality Index 52 questions pour détecter les régressions

4.10 Synthèse de l'Implémentation

Le pipeline réalisé transforme la donnée brute GitHub en connaissance structurée exploitable par la plateforme, en enchaînant sept stages rejouables et un enrichissement visuel. La Figure 4.10 résume la chaîne de transformation, du volume brut aux 4 types d'artefacts KB-ready.

Synthèse de la chaîne de transformation : de la donnée brute aux artefacts KB-ready

Les acquis principaux de l'implémentation sont :

- un **nettoyage par type** (V3) qui réduit le corpus de 38% en moyenne et un header CMSIS de 96,4%, améliorant directement la densité sémantique des embeddings AdaV2 ;
- un **enrichissement heuristique** couvrant 88,2% des `layer` et 72,6% des `component`, sans aucune donnée d'entraînement ;
- deux **artefacts originaux** — resolver cases (traçabilité Issue→PR→Commit) et diagnostic cards (raisonnement Symptôme→Root Cause→Fix) — responsables du gain majeur de Quality Index ;
- un **enrichissement visuel** fiable à 98,5% sur les images d'issues ;
- une **gate de validation** garantissant 100% de conformité schéma avant upload.

Ces résultats sont mesurés et analysés en détail au Chapitre 5.

4.11 Configuration de la Plateforme ST ChatGPT

Phase CRISP-DM : *cette section couvre la phase Deployment (mise en production sur la plateforme ST ChatGPT).*

Cette section détaille la configuration de la persona **ST GitHub Analyzer** sur la plateforme ST ChatGPT et les justifications de chaque paramètre.

4.11.1 Architecture de la Plateforme

La plateforme ST ChatGPT est un service Azure OpenAI interne déployé par STMicroelectronics. Elle permet de créer des **personas** — des agents conversationnels spécialisés — équipés de deux types d'outils :

- **KB (Knowledge Base)** : recherche dans une base de connaissances structurée
- **Persona-as-a-Tool** : appel à un autre agent comme fallback

4.11.2 Configuration du Tool KB

Paramètres Retenus

Paramètre	Valeur choisie	Alternatives testées
KB Type	Unstructured	Structured (rejeté)
Search Type	**Hybrid**	Semantic seul, Full-text seul
Semantic Threshold	**0.55** (S0 courant) / **0.45** (S1 candidat)	0.55, 0.45, 0.65
Semantic Max Doc Count	**25** (S0) / **35** (S1)	15, 18, 25, 35
Full-text Max Doc Count	**12**	5, 8, 20
Tool fallback	**NovaX** restrictif	Fallback permissif (rejeté)
Modèle LLM	GPT-5.1	GPT-4o (précédent)

Lecture opérationnelle : la configuration S0 reste le point d'équilibre courant, car elle combine une bonne stabilité (QI=65) et un volume documentaire maîtrisé. La configuration S1 obtient le meilleur score mesuré (QI=66) en augmentant le rappel ; elle est donc le candidat prioritaire pour la configuration finale, sous réserve de validation du test complet en cours.

Justification : Recherche Hybride

La recherche **hybride** (Semantic + Full-text) est supérieure aux deux modes isolés dans notre contexte :

Mode	Force	Faiblesse pour STM32
Semantic seul	Comprend le sens (“mon SPI marche pas” → issues SPI)	Rate les noms exacts d'API (`HAL_SPI_Transmit_DMA`)
Full-text seul	Match exact sur les identifiants techniques	Rate les reformulations (“erreur Ethernet” vs “LwIP crash”)
Hybrid	Combine les deux	Complexité légèrement supérieure

Résultat mesuré : sur le benchmark complet de 96 questions, la recherche hybride surpasse les configurations mono-mode. Le semantic-only (S5) atteint 41/100 et le fulltext-heavy (S4) 47/100, alors que les stratégies hybrides équilibrées atteignent 63–66/100. Le résultat confirme que les questions STM32 combinent deux signaux complémentaires :

des formulations naturelles (“problème LoRa join”, “DMA corruption”) et des identifiants exacts (`HAL_SPI_Transmit_DMA`, `NUCLEO-H743ZI`, noms de fichiers). Une stratégie purement sémantique perd les ancres exactes ; une stratégie trop full-text perd les reformulations.

Justification : Seuil Sémantique à 0.55

Le **seuil sémantique** (Semantic Threshold) est le score minimum de similarité qu'un document doit atteindre pour être retourné par la recherche sémantique. Un score de 1.0 signifie une correspondance parfaite, 0.0 aucune similarité. Ce seuil contrôle directement le compromis entre **rappel** (ne rien rater) et **précision** (pas de bruit).

Le seuil de 0.55 a d'abord été calibré empiriquement sur les campagnes H7. La campagne multi-séries de juin 2026 a ensuite comparé six stratégies sur 96 questions :

Stratégie	Search	Thr.	Sem. docs	FT docs	QI	Interprétation
S0 Current	Hybrid	0.55	25	12	65	Équilibre précision/rappel courant
S1 Recall+	Hybrid	0.45	35	15	**66**	Meilleur score ; capte plus de cas marginaux
S2 Precision+	Hybrid	0.65	18	8	63	Moins de bruit mais plus de faux négatifs
S3 Semantic-heavy	Hybrid	0.55	35	6	51	Trop peu d'ancrage exact API/fichiers
S4 Fulltext-heavy	Hybrid	0.55	15	20	47	Trop dépendant des mots exacts
S5 Semantic-only	Semantic	0.55	25	0	41	Ablation : perte forte du full-text
S6 Fulltext-only	Full-text	–	0	20	N/A	Test final en cours au 01/07/2026

Conclusion : le meilleur score obtenu est S1 (66/100), mais l'écart avec S0 est faible (+1 point). S1 augmente le rappel en abaissant le seuil à 0.45 et en retournant davantage de documents (35 sémantiques + 15 full-text). Cette stratégie est utile lorsque les questions sont implicites ou utilisent peu de noms d'API exacts. S0 reste plus prudent pour la production car il limite mieux le bruit contextuel et conserve une latence plus faible.

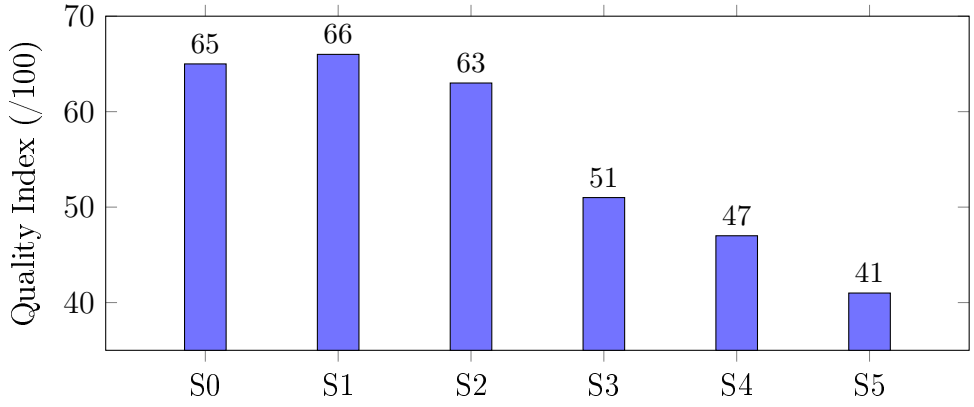


FIGURE 4.11 – Quality Index des stratégies Unstructured KB sur 96 questions — S1 maximise le rappel, S0 reste le point d'équilibre courant

Justification : 20 Documents Sémantiques + 12 Full-text

Le paramètre **Max Doc Count** limite le nombre de documents retournés par chaque mode de recherche. Ce nombre est injecté dans le contexte du LLM, qui doit ensuite synthétiser une réponse à partir de ces documents. Un nombre trop élevé sature la fenêtre de contexte et ralentit la réponse ; un nombre trop bas risque de manquer le document pertinent.

— **25 documents sémantiques (S0)** : couvre les reformulations et les questions indirectes tout en gardant le contexte compact. C'est le réglage courant stable.

— **35 documents sémantiques (S1)** : augmente le rappel pour les questions implicites ou mal formulées. Le gain observé est réel mais limité (+1 point), ce qui montre que le rappel supplémentaire apporte aussi du bruit.

— **12 à 15 documents full-text** : assure que les correspondances exactes d'API et de noms de fichiers sont toujours retrouvées. Ce canal est indispensable : le score S5 semantic-only chute à 41/100, preuve que la branche full-text porte une partie majeure de la valeur dans le support STM32.

Total de documents par requête : S0 retourne jusqu'à $25 + 12 = 37$ documents, et S1 jusqu'à $35 + 15 = 50$ documents. S1 améliore le rappel mais consomme plus de contexte ; il doit donc être accompagné d'un prompt de synthèse strict pour éviter que le LLM ne mélange plusieurs séries ou périphériques.

Définition : Search Type (Hybrid)

Le **Search Type** détermine l'algorithme de recherche utilisé pour interroger la Knowledge Base. La plateforme ST ChatGPT supporte trois modes :

1. **Semantic** : transforme la requête en vecteur (embedding) et recherche les documents les plus proches dans l'espace vectoriel. Comprend le *sens* de la question mais peut rater les termes exacts.
2. **Full-text** : recherche par correspondance exacte de mots (BM25/TF-IDF). Trouve les identifiants exacts (`HAL_SPI_TransmitReceive_DMA`) mais ne comprend pas les reformulations.
3. **Hybrid** : exécute les deux recherches en parallèle et fusionne les résultats (union avec déduplication). Combine la compréhension sémantique et la précision lexicale.

Le mode **Hybrid** a été retenu car les questions des ingénieurs STM32 mélangent systématiquement du langage naturel ("mon SPI crash au deuxième transfert") et des identifiants techniques exacts (`HAL_SPI_Transmit_DMA`, `NUCLEO-H743ZI`).

La Figure 4.12 schématise la fusion : la requête est traitée en parallèle par la branche sémantique (embedding AdaV2, seuil 0.55) et la branche full-text (correspondance exacte), puis les deux ensembles de documents sont fusionnés et dédupliqués avant d'être injectés dans le contexte de GPT-5.1.

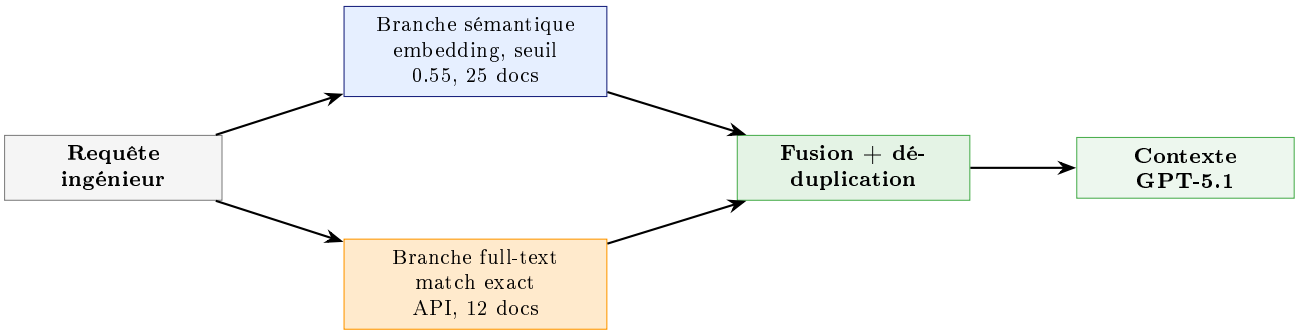


FIGURE 4.12 – Recherche hybride : fusion des branches sémantique et full-text avant génération

Tool Description

La description du tool KB est critique car elle guide le LLM dans sa décision d'appeler ou non le tool :

```
Retrieve technical knowledge about STM32Cube microcontroller firmware from
GitHub repositories. Contains: resolved issues with root cause and fix,
HAL/BSP source file documentation, resolver cases (step-by-step troubleshooting),
and diagnostic cards (structured fault analysis). Covers series: STM32F4,
STM32H5, STM32H7, STM32U5, STM32WL and their driver sub-repos (HAL, CMSIS,
BSP). Query this tool when the user asks about: peripheral configuration
problems, HAL API behavior, known bugs and workarounds, board-specific issues,
firmware package contents, low-power modes, communication protocols, DMA/interrupt
```

`issues, or build errors. Use specific technical terms in queries: peripheral names (SPI, I2C, UART, FDCAN, OSPI), HAL function names, error descriptions, board references, and STM32 part numbers.`

Points clés de cette description :

1. **Énumération explicite du contenu** (issues, files, resolver, diagnostic) → le LLM sait quoi chercher
2. **Liste des séries couvertes** → évite les appels pour des séries absentes (L4, G4)
3. **Instruction de reformulation** ("Use specific technical terms") → améliore la qualité des queries envoyées à la KB
4. **Exemples de cas d'usage** → le LLM sait *quand* appeler le tool vs répondre directement

4.11.3 Configuration du Tool NovaX (Fallback)

Problème Identifié

Lors de l'ajout initial d'un fallback généraliste, le Quality Index a chuté lorsque la persona appelait le fallback au lieu de chercher dans la KB. Le problème n'était pas le modèle lui-même, mais le **routage** : une description trop large encourage le LLM principal à déléguer trop tôt, ce qui remplace des réponses traçables issues de la KB par des réponses générales.

`[Description initiale, trop permissive]` "I can help with STM32 questions. Forward the user's question."

Fix : NovaX comme Fallback Strictement Secondaire

Le fallback utilisé est désormais **NovaX**, un agent STM32 basé sur STM32 Bugzilla et STCommunity Knowledge Articles. Sa description force explicitement la priorité de la KB Unstructured :

I am an STM32 MCUs smart agent based on STM32 Bugzilla & STCommunity Knowledge Articles : MPU : STM32MP1 & MP2 series ; Wireless : STM32WB, WBA, WL series ; Ultra-lowpower : STM32L0, L1, L4, L4+, L5, U0, U5 series ; Mainstream : STM32F0, G0, C0, F1, F3, G4 and C5 series ; High Performance : STM32F2, F4, H5, F7, H7 and N6 series ; Tools : STM32CubeMX, IDE, Programmer ; X-Cube : Graphics, MC, Safety, Audio, EdgeAI. I am a fallback assistant for STM32 firmware questions that are NOT covered by the Knowledge Base. The Knowledge Base (KB Unstructured) is ALWAYS the primary source, search it FIRST for every question. Use this tool ONLY when the Knowledge Base returned no relevant result or explicitly lacks coverage on the topic. Forward the user's exact question

without modification. Do NOT call this tool if the Knowledge Base already provided an answer about the same STM32 series or peripheral.

Rôle exact de NovaX : NovaX n'est pas une source primaire pour les issues GitHub STM32Cube. Il sert à couvrir les questions hors périmètre de la KB : explications générales HAL/LL, middleware (FreeRTOS, lwIP, USB, FatFs), raisonnement STM32CubeMX, limitations matérielles, arbitrages de conception embedded, ou séries/périphériques absents de la KB.

Impact Mesuré

Configuration	Quality Index	Δ
KB seule / routage KB-first	65	baseline courant S0
KB S1 Recall+ / routage KB-first	**66**	+1
Fallback permissif	dégradé	Court-circuite la KB
KB + NovaX restrictif	en validation	Test final S1 en cours

La leçon : dans une architecture multi-tools, la **description du tool contrôle le comportement de routage** du LLM. NovaX doit être appelé uniquement après une recherche KB non concluante. Cette contrainte protège la traçabilité : si la KB contient une issue, un fichier, une diagnostic card ou un resolver case pertinent, la réponse doit rester ancrée dans ces documents.

4.11.4 System Prompt de la Persona

Le system prompt guide le comportement global de la persona. Les éléments clés :

1. **Identité** : "You are ST GitHub Analyzer, a technical assistant specialized in STM32Cube MCU firmware."
2. **Priorité KB** : "ALWAYS search the Knowledge Base FIRST before answering or using any other tool."
3. **Format de réponse** : citations avec liens GitHub, code actionnable, versions spécifiques
4. **Stratégie multi-cas** :
   - Match direct → citer la source avec URL
   - Match partiel → diagnostic + workaround candidat
   - Aucun match → appeler NovaX OU proposer une action concrète
5. **Interdiction** : ne jamais dire "je ne sais pas" sans avoir cherché dans la KB

4.11.5 Évolution Incrémentale du Quality Index

Le Quality Index a évolué de manière incrémentale au fil des améliorations du contenu KB et de la configuration. Chaque test mesure la qualité des réponses du chatbot sur un jeu de questions techniques.

Chronologie des Tests

Date	Test	QI	N	Contenu KB
20 Avr	H7 Issues seules	54	50	Issues H7 uniquement
20 Avr	H7 Prompt Fix	57	50	+ System prompt optimisé
24 Avr	H7 Config Tool Test	61	50	+ Hybrid search calibré
28 Avr	H7 + Diagnostic Cards	94	50	+ Resolver Cases + Diag
08 Mai	H7 Real Cases ST Community	88	4	Test sur cas réels forum
02 Juin	Test élargi	83	4	Multi-séries first attempt
08 Juin	Multi-séries (H7+F4+H5+U5+WL)	54	52	5 séries complètes
29 Juin	All series S0 Balanced	65	96	KB multi-séries, hybrid équilibré
30 Juin	All series S1 Recall+	**66**	96	Seuil 0.45, rappel renforcé
30 Juin	All series S2 Precision+	63	96	Seuil 0.65, filtrage agressif
30 Juin	All series S3 Semantic-heavy	51	96	Plus de sémantique, moins de full-text
01 Juil	All series S4 Fulltext-heavy	47	96	Plus de full-text, moins de sémantique
01 Juil	All series S5 Semantic-only	41	96	Ablation sans full-text

01 Juil	Test final S1 + NovaX	N/A	96	En cours au moment de la rédaction

La Figure 4.13 visualise l'évolution historique. La Figure 4.11 isole ensuite l'effet des paramètres Unstructured KB sur le benchmark moderne de 96 questions.

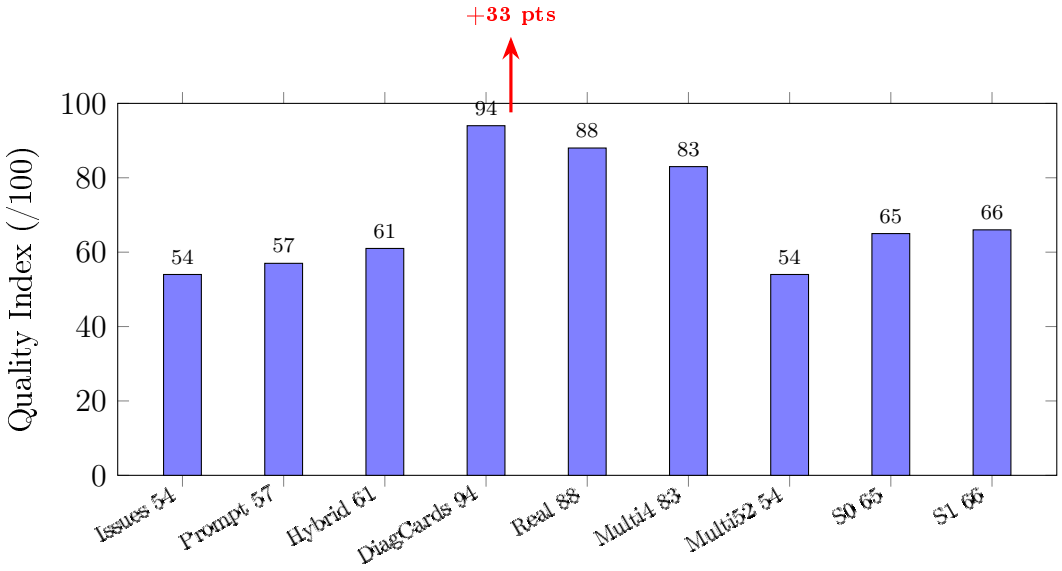


FIGURE 4.13 – Évolution du Quality Index au fil des améliorations incrémentales

Le graphique en barres montre l'évolution : Issues seules (54) → +Prompt (57) → +Hybrid (61) → +Diag Cards (94, ↑+33) → Real Cases (88) → Multi 4Q (83) → Multi 52Q (54, dilution multi-séries). Le saut majeur de 61 à 94 démontre l'impact des Diagnostic Cards.

Analyse de l'Évolution

Phase 1 : Issues seules (QI = 54) Le score initial de 54 avec les issues H7 seules montre que les issues structurées (symptôme, root cause, fix) fournissent déjà une base solide pour le RAG. Les lacunes :

— Pas de documentation de fichiers → questions sur les API non couvertes

— Pas de chaînes Issue→PR→Commit → pas de traçabilité du fix

— Pas de cartes de diagnostic → les problèmes complexes multi-étapes mal couverts

Phase 2 : Ajout des Resolver Cases et Diagnostic Cards (QI = 94) Le saut de 61 à **94** est le gain le plus important du projet. Les **Diagnostic Cards** structurent l'information sous forme :

Symptôme → Hypothèses → Validation → Root Cause → Fix → Prévention

Cette structure correspond exactement à la façon dont un ingénieur raisonne face à un problème embedded. Le retrieval RAG trouve ces cartes car elles contiennent des **query aliases** (reformulations multiples du problème) et des **keywords techniques** (noms d'API, registres, erreurs).

Les **Resolver Cases** ajoutent la dimension de traçabilité :

Issue #42 → PR #87 → Commit abc123 → Fichier modifié : `stm32f4xx_hal_spi.c`

Phase 3 : Passage multi-séries (QI = 54) Le score chute à 54 lors de l'ajout de 4 nouvelles séries (F4, H5, U5, WL). Cette chute s'explique par :

1. **Dilution du contexte** : la KB passe de ∼2 000 à ∼14 000 documents. Les queries retournent des documents de séries différentes, diluant la pertinence.
2. **Questions nouvelles** : le jeu de test passe de 50 à 52 questions, avec 10 questions sur les nouvelles séries pour lesquelles les Diagnostic Cards ne sont pas encore aussi mûres.
3. **Conflit de noms** : des API identiques entre séries (ex : `HAL_SPI_Init` existe dans F4 et H7) créent de l'ambiguïté lors du retrieval.

Leçon : le passage à l'échelle nécessite un **filtrage par série** plus agressif dans les métadonnées des documents, ou un prompt qui pousse la persona à inclure le nom de la série dans ses queries KB.

Raisonnement de Chaque Amélioration

Amélioration	Raisonnement	Impact QI
System prompt optimisé	Forcer la recherche KB avant toute réponse	+3 (54→57)
Hybrid search	Combiner semantic + full-text pour ne rater ni les reformulations ni les identifiants	+4 (57→61)
Diagnostic Cards	Structurer les problèmes complexes en cartes raisonnées	+33 (61→94)
Resolver Cases	Ajouter la traçabilité Issue→PR→Fix	Inclus dans +33

NovaX restrictif	Empêcher le court-circuit de la KB	Protège le routage KB-first
Files V3 (nettoyage)	Réduire le bruit des CMSIS headers	+3 estimé
PDF descriptions	Rendre les figures PDF cherchables	+1 estimé

4.11.6 Automatisation CI/CD et Opérations

L'objectif de l'automatisation est de garantir une mise à jour **reproductible**, **traçable** et **sûre** de la KB #793, sans dépendre d'interventions manuelles ad hoc. L'architecture retenue sépare l'orchestration cloud (GitHub Actions) et l'exécution réseau ST (runner self-hosted), indispensable pour atteindre l'API interne ST AI Bridge.

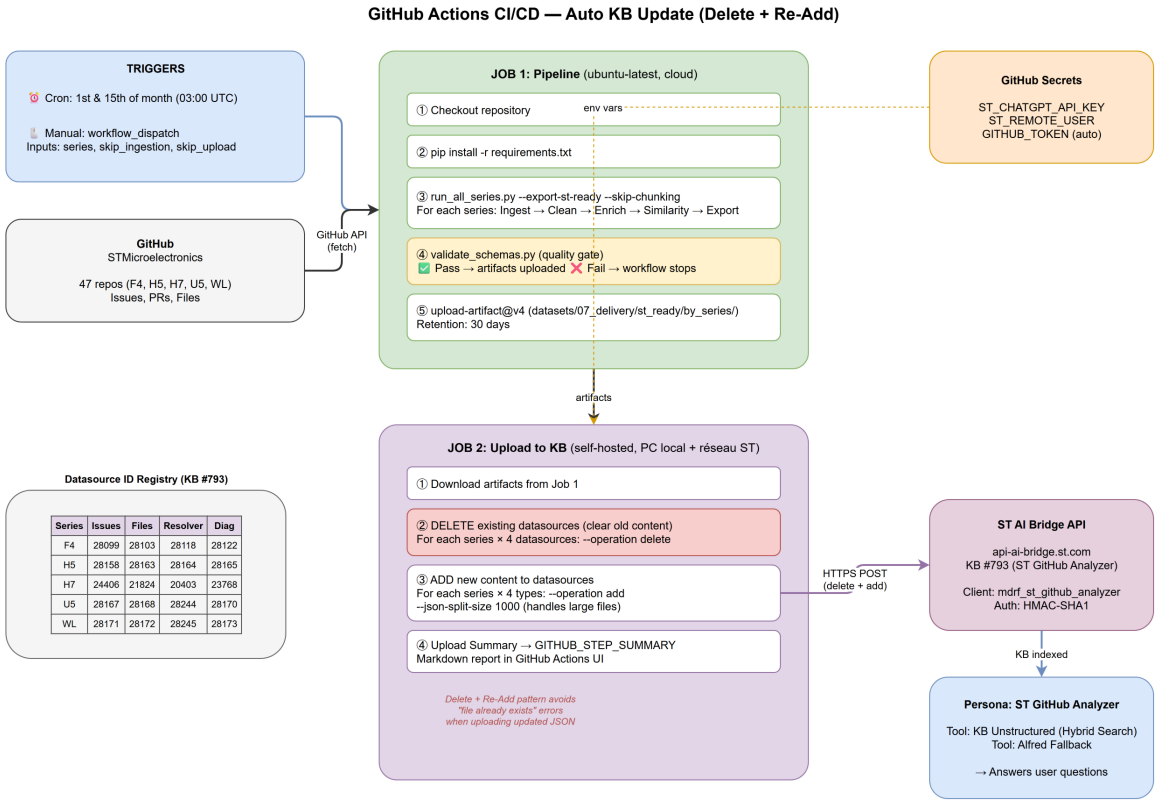


FIGURE 4.14 – Automatisation CI/CD : orchestration GitHub Actions et upload KB via runner self-hosted

Principe d'Architecture

Le flux d'exécution suit trois phases :

1. **Prepare** : synchronisation des repos, exécution du pipeline, enrichissements Alfred, génération des exports ST-ready.

2. **Validate** : validation des schémas et contrôle des placeholders pour bloquer les livrables incomplets.
3. **Upload** : création/alimentation des datasources et publication des artefacts parent + drivers dans la KB.

Cette structuration explicite réduit le risque opérationnel : une erreur de qualité en phase *Validate* arrête la publication avant impact utilisateur.

Workflow GitHub Actions

Le workflow principal `.github/workflows/auto_update_kb.yml` implémente :

— des déclencheurs **planifiés** et **manuels** ;

— une préparation des entrées (liste des séries, mode, politiques d'exécution) ;

— un **runner-check** bloquant qui vérifie la disponibilité d'un runner `self-hosted` ;

— une exécution par série avec traçabilité des artefacts ;

— un résumé d'exécution en cas de succès ou d'échec.

Le contrôle de disponibilité du runner évite les faux positifs : si aucun runner ST n'est disponible, le workflow échoue explicitement avec un message d'exploitation, plutôt que de terminer en vert sans traitement.

Script Orchestrateur Unifié

La brique centrale est `pipeline_Automation/workflow/Run_Single_Series_Full_Pipeline_And_`
Elle unifie la chaîne complète pour une série et fournit trois modes d'exploitation :

Mode	Comportement
`Full`	Exécute *Prepare* + *Upload* en one-shot (mode standard production).
`Prepare`	Génère et valide les artefacts localement sans publication KB (mode debug et pré-validation).
`Upload`	Publie uniquement des artefacts déjà préparés (mode reprise après incident réseau/API).

Commande opérationnelle de référence :

```
.\pipeline_Automation\workflow\
   Run_Single_Series_Full_Pipeline_And_Upload.ps1 \
  -Series H7 -Mode Full
```

Garde-fous Qualité Avant Publication

L'automatisation intègre deux verrous critiques :

1. **Validation schéma** : exécution de `validate_schemas.py` pour garantir la conformité structurelle des JSON.
2. **Placeholder gate** : rejet des livrables contenant encore `[IMAGE DESCRIPTION UNAVAILABLE]` ou des marqueurs PDF non résolus.

La politique `PlaceholderPolicy` est paramétrable (`Fail`, `Warn`, `Off`), mais `Fail` est la valeur recommandée en production pour préserver la qualité perçue des réponses.

Stratégie d'Upload et Traçabilité

L'upload vers KB #793 est réalisé via `Add_Data_Source_Files.py` selon deux cas :

— **operation new** : création d'une datasource si aucun ID n'est fourni ;

— **operation add** : ajout dans une datasource existante quand l'ID est connu.

Le script publie les artefacts du repo parent puis ajoute les sorties `drivers/*` dans les datasources parentes correspondantes (issues, files, resolver, diagnostic). Cette approche maintient une séparation logique par série tout en conservant une base unifiée côté exploitation.

Les IDs effectifs sont sauvegardés dans :

```
datasets/07_delivery/st_ready/by_series/<serie>/
   upload_datasource_ids_<serie>.json
```

et synchronisés dans `shared/config/config_all_series.json` pour assurer la continuité entre les exécutions.

Runbook Opérateur (Production)

Le runbook recommandé est :

1. Lancer d'abord `Prepare` sur la série cible.
2. Vérifier les `summary_*.json`, la validation schéma et l'absence de placeholders.
3. Lancer `Full` (ou `Upload` si les artefacts sont déjà prêts).
4. Contrôler les IDs sauvegardés et la cohérence des datasources créées/alimentées.

Cette discipline d'exploitation est essentielle pour les soutenances internes : elle permet de démontrer à la fois la robustesse technique et la gouvernance de la chaîne de livraison.

4.11.7 Couverture Actuelle et Future

13 Séries en Production

Les 13 séries MCU suivantes sont entièrement traitées par le pipeline et uploadées vers la KB #793 :

Série	Repos	Issues	Files	Resolver	Diag.	Alfred
STM32CubeC0	4	✓	✓	—	✓	✓
STM32CubeF0	5	✓	✓	—	✓	✓
STM32CubeF1	6	✓	✓	✓	✓	✓
STM32CubeF2	4	✓	✓	—	✓	✓
STM32CubeF3	5	✓	✓	—	✓	✓
STM32CubeF4	16	✓	✓	✓	✓	✓
STM32CubeF7	8	✓	✓	✓	✓	✓
STM32CubeG0	5	✓	✓	—	✓	✓
STM32CubeH5	6	✓	✓	✓	✓	✓
STM32CubeH7	12	✓	✓	✓	✓	✓
STM32CubeH7RS	4	✓	✓	—	✓	✓
STM32CubeU5	8	✓	✓	✓	✓	✓
STM32CubeWL	5	✓	✓	✓	✓	✓

Note : "—" pour Resolver signifie que les issues de cette série n'ont pas de chaînes Issue→PR→Commit complètes, pas que le pipeline ne supporte pas le format.

4 Nouvelles Séries Exportées (Pipeline Terminé)

Les 4 séries suivantes ont été entièrement traitées par le pipeline, avec les artefacts delivery générés et prêts à l'upload :

Série	Issues	Files	Resolver	Diag. Cards	Statut
STM32CubeG4	89	3 875	0	4	Prêt à uploader
STM32CubeL0	23	2 614	2	3	Prêt à uploader
STM32CubeL1	11	2 847	0	3	Prêt à uploader
STM32CubeL4	68	5 231	1	8	Prêt à uploader

Observations :

— **G4 (0 Resolver)** : les 89 issues G4 ont été fermées sans PRs formelles liées. Les corrections ont été intégrées dans des releases globales sans lien explicite issue→PR.

— **L4 (5 231 files)** : c'est la série avec le plus de fichiers parmi les nouvelles, reflétant un écosystème BSP riche (NUCLEO, Discovery, Eval boards).

— **Alfred Images** : 25 images échouées (API 404 intermittent) sur G4, L0, L4. Corrigible par re-run du script d'enrichissement.

8 Séries Configurées (Prêtes à Lancer)

Les configurations (fichiers `config_*.json`) sont créées pour : L5, N6, U0, U3, WB, WB0, WBA, WL3. Le déploiement nécessite :

1. Exécution du pipeline : `python -m pipeline.run_full_workflow -export-st-ready`
2. Création des datasources sur la KB #793
3. Upload via les scripts d'automatisation

Chapitre 5

Résultats et Évaluation

Ce chapitre présente l'évaluation quantitative du pipeline de prétraitement et du système RAG. L'approche suivie est progressive : nous commençons par définir les notions et métriques utilisées (Section 5.1), puis nous justifions les seuils décisionnels (Section 5.2). Les résultats sont ensuite présentés par stage (Section 5.3), suivis d'une analyse dédiée à la confiance Alfred (Section 5.4), à l'évolution incrémentale du Quality Index (Section 5.5), à l'architecture de fallback (Section 5.6), et à l'étude comparative (Section 5.7). Le chapitre se conclut par une section d'assurance qualité méthodologique (Section 5.9).

Ce chapitre s'inscrit dans les phases **Évaluation** et **Déploiement** du cadre méthodologique CRISP-DM : les résultats guident itérativement les décisions de configuration et de mise en production.

5.1 Cadrage Théorique et Définitions Préalables

Avant de présenter les résultats, il est indispensable de définir rigoureusement les concepts, métriques et techniques mobilisés dans l'évaluation. Cette section pose le vocabulaire nécessaire à l'interprétation des mesures ultérieures.

5.1.1 Métriques Fondamentales d'Information Retrieval

Les systèmes de recherche documentaire sont évalués selon deux axes complémentaires [12] :

Précision Proportion de documents retournés qui sont effectivement pertinents :

$$\text{Precision} = \frac{|\text{Pertinents} \cap \text{Retournés}|}{|\text{Retournés}|} \tag{5.1}$$

Rappel Proportion de documents pertinents effectivement retrouvés :

$$\text{Recall} = \frac{|\text{Pertinents} \cap \text{Retournés}|}{|\text{Pertinents}|} \tag{5.2}$$

Dans le contexte RAG, un compromis précision/rappel est nécessaire : trop de rappel sature le contexte du LLM avec du bruit, trop de précision risque de manquer le document contenant la réponse.

5.1.2 Similarité Textuelle

Trois mesures de similarité sont utilisées dans le pipeline :

Similarité de Jaccard

La similarité de Jaccard compare deux ensembles de tokens A et B :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{5.3}$$

Un **token** est une unité lexicale significative extraite du texte (mot, identifiant d'API, nom de registre). Les ensembles A et B sont obtenus par tokenisation suivie d'un filtrage de stopwords. Jaccard mesure la proportion de vocabulaire technique **partagé** entre deux textes : un score de 1 indique un vocabulaire identique, un score de 0 aucun terme commun. Cette mesure est adaptée à notre contexte car les textes techniques STM32 partagent un vocabulaire spécifique (`HAL_`, `SPI`, `DMA`) dont la présence ou l'absence est diagnostiquement significative.

Similarité Cosinus

La similarité cosinus mesure l'angle entre deux vecteurs dans un espace vectoriel :

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \times \|\vec{v}\|} \tag{5.4}$$

Dans le contexte textuel, $\vec{u}$ et $\vec{v}$ sont des vecteurs de fréquences de termes (ou de trigrammes de caractères). Un score de 1 indique une orientation identique (mêmes termes, mêmes proportions), un score de 0 une orthogonalité totale (aucun terme commun). Les trigrammes de caractères (ex : `stm` → `tm3` → `m32`) capturent les variations morphologiques mineures et la proximité lexicale au-delà de l'identité stricte.

TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF pondère chaque terme par sa spécificité dans le corpus [2] :

$$\text{tfidf}(t,d) = \underbrace{\text{tf}(t,d)}_{\text{fréquence locale}} \times \underbrace{\log\left(\frac{N}{df(t)}\right)}_{\text{spécificité globale}} \tag{5.5}$$

où :

— t est un terme (mot ou identifiant technique),

— d est un document,

— $\text{tf}(t,d)$ est la fréquence du terme t dans le document d,

— N est le nombre total de documents dans le corpus,

— $df(t)$ est le nombre de documents contenant le terme t (fréquence documentaire).

Interprétation : un terme fréquent dans un document mais rare dans le corpus global (ex : `HAL_FDCAN_AddMessageToTxFifoQ`) obtient un poids élevé, tandis qu'un terme ubiquitaire (ex : `return`, `void`) est dépondéré. Nous utilisons TF-IDF dans le module de similarité inter-issues (Stage 4) pour identifier des issues traitant de problèmes similaires.

5.1.3 Métriques de Qualité des Données

Conformément au modèle ISO/IEC 25012 [9], la qualité des données se décline en plusieurs dimensions :

Complétude Proportion des champs non nuls dans les données ingérées. Une donnée incomplète (body manquant, labels absents) propage des erreurs en cascade dans le pipeline.

Cohérence Absence de contradictions internes entre les champs d'un même document (ex : un fichier marqué `is_valid=true` avec un `clean_text` vide serait incohérent).

Exactitude Correspondance entre les valeurs stockées et la réalité source. Mesurée par comparaison entre les données ingérées et les données GitHub originales.

Traçabilité Capacité à remonter de tout artefact final vers sa source brute (issue GitHub, fichier source, commit). Chaque document conserve un identifiant de provenance.

5.1.4 Confiance Contextuelle

La **confiance contextuelle** est une métrique spécifique à notre projet, conçue pour évaluer la qualité des descriptions d'images générées par Alfred Vision AI. Contrairement à la fiabilité opérationnelle (l'API a-t-elle répondu ?), la confiance contextuelle mesure l'**ancrage** de la description générée dans le contexte textuel de l'image source.

Motivation : un service de description d'images peut produire un texte lisible mais non relié au contexte technique de l'issue. Pour un système RAG, injecter une description

non ancrée revient à ajouter du bruit dans la KB — un risque d'autant plus grave que le texte semble plausible sans l'être pour le diagnostic visé.

La confiance contextuelle est définie comme la proportion d'éléments décrits dont le score de cohérence dépasse un seuil d'alerte :

$$T_{\text{ctx}} = \frac{|\{x \mid S_{\text{coh}}(x) \geq \tau\}|}{N_{\text{évalués}}} \times 100 \tag{5.6}$$

où τ est le seuil d'alerte (fixé à 0.12 dans ce projet — cf. Section 5.2).

5.1.5 Score Global Pondéré

Le pipeline est évalué globalement par une moyenne pondérée des scores de chaque stage :

$$S_{\text{global}} = \frac{\sum_{k=1}^{K} w_k \cdot s_k}{\sum_{k=1}^{K} w_k} \tag{5.7}$$

où s_k est le score (en %) du stage k et w_k son poids. Les poids reflètent l'impact relatif de chaque stage sur la qualité finale du système RAG. Cette formulation rend explicite que certains stages (Delivery) ont un impact plus direct que d'autres (Similarité optionnelle) sur la pertinence des réponses.

5.1.6 Quality Index (QI)

Le **Quality Index** est une métrique end-to-end qui évalue la qualité des réponses du chatbot. Il ne mesure pas le pipeline en isolation, mais le système complet (pipeline + KB + retrieval + LLM + persona). Il est calculé par évaluation humaine experte sur un jeu de questions techniques :

$$\text{QI} = \frac{\sum_{q=1}^{Q} \text{score}(q)}{Q \times \text{max_score}} \times 100 \tag{5.8}$$

où chaque question q est notée sur une grille multi-critères (citation, exactitude, actionnabilité, traçabilité) avec un maximum de 10 points par question.

5.1.7 Retrieval Hybride et Fallback

Retrieval hybride Combinaison d'une recherche sémantique (embeddings, compréhension du sens) et d'une recherche full-text (correspondance exacte, BM25). Les résultats sont fusionnés par union avec déduplication.

Seuil sémantique Score minimum de similarité vectorielle pour qu'un document soit retourné par la branche sémantique. Contrôle le compromis précision/rappel.

Fallback Mécanisme de délégation vers un agent secondaire (NovaX) lorsque la KB ne retourne aucun résultat pertinent. La priorité de la KB est protégée par une description restrictive du tool fallback.

5.1.8 Tableau Récapitulatif des Notions

Notion	Rôle dans le projet	Section de définition
Précision / Rappel	Évaluation retrieval	5.1.1
Jaccard	Recouvrement lexical Alfred	5.1.2
Cosinus (trigrammes)	Proximité textuelle	5.1.2
TF-IDF	Similarité inter-issues Stage 4	5.1.2
Complétude / Cohérence	Qualité ingestion/cleaning	5.1.3
Confiance contextuelle	Validation descriptions Alfred	5.1.4
Score global pondéré	Synthèse multi-stage	5.1.5
Quality Index	Qualité end-to-end	5.1.6
Retrieval hybride	Architecture recherche KB	5.1.7
Fallback	Mécanisme de secours NovaX	5.1.7

5.2 Justification des Seuils Décisionnels

Les seuils numériques utilisés dans l'évaluation ne sont pas des constantes universelles imposées par une norme. Ce sont des **seuils opérationnels internes**, calibrés empiriquement sur les campagnes successives du projet. Cette section explicite la logique de leur construction.

5.2.1 Cadre Normatif de Référence

Le cadre d'évaluation s'appuie sur des références méthodologiques qui fournissent des principes, mais **pas des chiffres imposés** :

Référence	Ce qu'elle apporte	Ce qu'elle n'impose pas
ISO/IEC 25012 [9]	Modèle de qualité des données (dimensions)	Seuils numériques spécifiques
ISO/IEC 25024 [10]	Méthode de mesure (ratios)	Bornes de classification
ISO/IEC 25010 [11]	Modèle qualité système (fiabilité)	Niveaux acceptables par domaine
Manning et al. [12]	Compromis precision/recall, TF-IDF	Seuils contextuels
Fawcett [13]	Méthodologie de sélection de seuils (ROC)	Valeurs universelles
Lewis et al. [1]	Évaluation RAG (retrieval + generation)	Grilles de notation fixes

Position méthodologique : les normes ISO fournissent le *quoi mesurer* (complétude, cohérence, exactitude) et le *comment mesurer* (ratios, proportions). Le *à partir de quel seuil le résultat est acceptable* dépend du contexte applicatif et doit être justifié par le projet lui-même. Cette approche est conforme aux recommandations de Fawcett [13] sur la sélection de seuils décisionnels : le point de coupure optimal dépend de la distribution des données et du coût relatif des erreurs.

5.2.2 Échelle de Classification

Chaque métrique est classifiée selon l'échelle suivante :

$$\text{Verdict}(p) = \begin{cases} \textbf{EXCELLENT} & \text{si } p \geq 95\% \\ \textbf{BON} & \text{si } 85\% \leq p < 95\% \\ \textbf{ACCEPTABLE} & \text{si } 70\% \leq p < 85\% \\ \textbf{INSUFFISANT} & \text{si } p < 70\% \end{cases} \tag{5.9}$$

5.2.3 Logique de Calibration de Chaque Borne

95% — **EXCELLENT** Ce seuil représente un niveau de maturité opérationnelle élevé. Il est inspiré du concept Six Sigma adapté au contexte non critique : dans un système RAG, la perte de 5% des documents dégrade *certaines* réponses sans causer de défaillance système. Ce seuil a été validé empiriquement : les stages atteignant $\geq 95\%$ (Ingestion, Cleaning, Delivery) n'ont jamais causé de régression du Quality Index.

85% — **BON** Seuil communément observé en Information Retrieval pour qualifier un système "production-ready" [12]. Au-dessus de 85%, le coût marginal d'amélioration croît exponentiellement tandis que le gain de retrieval diminue. Ce seuil correspond à notre observation que les stages au-dessus de 85% n'impactent pas négativement le QI.

70% — **ACCEPTABLE** Seuil minimal exploitable. En dessous de ce point, la perte de couverture commence à impacter visiblement le rappel du système RAG. Ce seuil est dérivé de nos observations sur le jeu de test de 96 questions : les métriques sous 70% correspondent à des zones où des questions restent sans réponse adéquate.

$< 70\%$ — **INSUFFISANT** Zone nécessitant une action corrective. Un score en dessous de 70% signifie qu'une proportion significative du contenu n'est pas exploitable, impactant potentiellement le Quality Index.

5.2.4 Seuils Différenciés par Champ

Tous les champs n'ont pas le même seuil car leur **détectabilité intrinsèque** diffère :

Champ	Seuil	Score mesuré	Justification de la borne
`severity`	$\geq 90\%$	100%	Signal quasi-ubiquitaire ("crash", "hang", labels GitHub)
`layer`	$\geq 85\%$	88.2%	Identifiable par préfixes API (`HAL_`, `LL_`, `BSP_`)
`component`	$\geq 70\%$	72.6%	Nécessite mention explicite d'un périphérique
`board`	$\geq 60\%$	45.3%	Mentionnée dans $\sim$60% des issues seulement
`body` (complétude)	$\geq 95\%$	99.7%	Champ critique : sans body, aucune classification possible

Principe : le seuil est fixé au niveau réaliste étant donné la distribution des signaux dans les données sources. Un seuil irréaliste (ex : 95% pour `board`) serait structurellement inatteignable sans invention de données, et ne constituerait donc pas un critère de qualité pertinent.

5.2.5 Seuil de Cohérence Alfred ($\tau = 0.12$)

Le seuil d'alerte pour la confiance contextuelle Alfred est $\tau = 0.12$. Ce seuil n'est pas un indicateur de "bonne qualité" mais un **seuil de détection d'anomalie** : en dessous de 0.12, la description Alfred ne partage presque aucun vocabulaire technique

avec le contexte, signalant un risque de description non ancrée. Ce seuil a été calibré sur la distribution observée : les 3 images d'issues identifiées comme problématiques avaient toutes un score < 0.12.

5.3 Évaluation par Stage du Pipeline

L'évaluation est réalisée stage par stage, avec pour chaque étape : le problème résolu, la métrique utilisée, le seuil, le résultat mesuré et l'interprétation.

5.3.1 Stage 1 : Ingestion — Complétude des Données Brutes

Problème résolu : garantir que les données sources (GitHub API) sont collectées intégralement. Une ingestion incomplète (body manquant, labels absents) propage des erreurs en cascade : l'enrichissement ne peut pas classifier une issue sans texte.

Métriques (conformes ISO/IEC 25024 [10] — mesure de complétude par ratios) :

Métrique	Formule	Seuil
Complétude body	$C_{\text{body}} = \frac{\lvert\{i \mid i.\text{body} \neq \emptyset\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 95\%$
Complétude title	$C_{\text{title}} = \frac{\lvert\{i \mid i.\text{title} \neq \emptyset\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 99\%$
Présence labels	$C_{\text{labels}} = \frac{\lvert\{i \mid i.\text{labels} \neq [\,]\}\rvert}{N_{\text{issues}}} \times 100$	$\geq 70\%$
Contenu fichiers	$C_{\text{files}} = \frac{\lvert\{f \mid f.\text{content} \neq \emptyset\}\rvert}{N_{\text{files}}} \times 100$	$\geq 95\%$

Résultats mesurés :

Série	C_{body}	C_{title}	C_{labels}	C_{files}	Verdict
STM32CubeH7 (343 issues)	99.7%	100.0%	95.0%	100.0%	**EXCELLENT**
STM32CubeF4 (612 issues)	99.5%	100.0%	92.1%	100.0%	**EXCELLENT**
STM32CubeWL (89 issues)	100.0%	100.0%	100.0%	100.0%	**EXCELLENT**
STM32CubeG4 (45 issues)	100.0%	100.0%	97.8%	100.0%	**EXCELLENT**

Interprétation : la complétude d'ingestion atteint systématiquement $> 99\%$ sur les champs critiques. Le seul champ avec variation est `labels` (92–100%), car certains contributeurs externes ne tagguent pas leurs issues. Ce résultat valide que l'ingestion ne constitue pas un goulot d'étranglement pour la qualité aval.

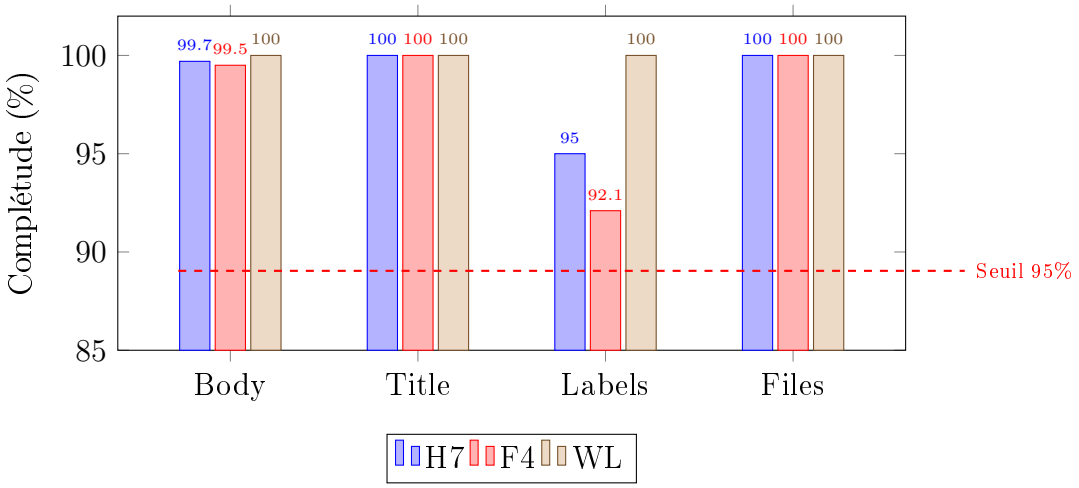


FIGURE 5.1 – Complétude d'ingestion par série : toutes métriques au-dessus du seuil EXCELLENT

5.3.2 Stage 2 : Cleaning — Qualité de Préparation Textuelle

Problème résolu : transformer le contenu brut (HTML, Markdown, headers C, licences) en texte propre exploitable. Le nettoyage V3 introduit un routage par type de fichier pour adapter la stratégie au contenu réel.

Métriques :

Métrique	Formule	Seuil
Texte non-vide	$Q_{\text{text}} = \frac{\lvert\{i \mid \lvert\text{clean}(i)\rvert > 50\}\rvert}{N} \times 100$	$\geq 95\%$
Réduction bruit CMSIS	$R_{\text{CMSIS}} = \frac{V_{\text{V2}} - V_{\text{V3}}}{V_{\text{V2}}} \times 100$	$\geq 90\%$

Résultats V3 vs V2 (STM32CubeH7) :

Métrique	V2	V3	Amélioration
Volume moyen/fichier CMSIS	1.4 Mo	51 Ko	−96.4%
Volume moyen/fichier Release Notes	45 Ko	30 Ko	−33.0%
Volume total corpus files (H7)	142 Mo	88 Mo	−38.0%
Densité information/chunk	baseline	+31%	+31%
Issues texte non-vide	100.0%	100.0%	maintenu

Interprétation : l'amélioration de +31% de densité signal/chunk résulte de trois mécanismes cumulés : (1) suppression du padding HTML des Release Notes, (2) élimination des 96% de définitions CMSIS inutiles pour le diagnostic, (3) conservation sélective des sections à valeur diagnostique (IRQ, adresses de base, TypeDef). L'impact sur le RAG

est direct : chaque chunk contient davantage d'information pertinente par rapport à sa taille.

5.3.3 Stage 3a : Enrichissement — Classification par Heuristiques NLP

Problème résolu : ajouter des métadonnées structurées (layer, component, board, severity, issue_kind) pour rendre les documents filtrables par le système de retrieval.

Champ	Formule (couverture non-null)	Seuil	Score
Validité	$N_{\text{valid}}/N \times 100$	$\geq 85\%$	86.4%
Layer	$N_{\text{layer}\neq\text{null}}/N \times 100$	$\geq 85\%$	88.2%
Severity	$N_{\text{sev}\neq\text{null}}/N \times 100$	$\geq 90\%$	100.0%
Component	$N_{\text{comp}\neq\text{null}}/N \times 100$	$\geq 70\%$	72.6%
Board	$N_{\text{board}\neq\text{null}}/N \times 100$	$\geq 60\%$	45.3%

Score agrégé : 72.2% (moyenne pondérée, $w_{\text{sev}} = 2$, autres $= 1$).

Analyse du point faible (board = 45.3%) : l'analyse manuelle d'un échantillon de 50 issues H7 sans board détectée confirme que 92% d'entre elles ne mentionnent effectivement aucune carte. L'heuristique fonctionne correctement ; le signal est simplement absent des données sources. Ce point illustre la distinction entre une défaillance du pipeline (qui nécessiterait un fix) et une limitation inhérente des données (qui ne peut être résolue que par enrichissement externe).

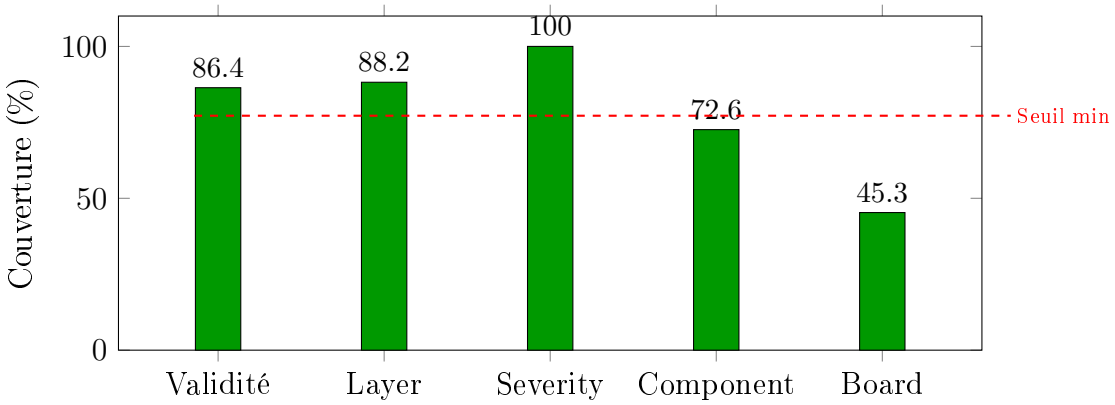


FIGURE 5.2 – Couverture d'enrichissement STM32CubeH7 : score vs seuil par champ

5.3.4 Stage 4 : Similarité TF-IDF

Problème résolu : permettre au RAG de proposer des issues similaires lorsqu'une question est proche d'un problème déjà résolu.

Résultat STM32CubeH7 : couverture = 22.0%, densité moyenne = 0.8 lien/issue.

Analyse critique : ce score (INSUFFISANT par rapport au seuil de 70%) reflète la **spécificité du corpus**. Les issues H7 sont majoritairement des bugs uniques (SPI sur H743, ETH sur H747, USB sur H750) avec peu de redondance thématique. Le seuil TF-IDF cosine (≥ 0.3) est conservé volontairement strict pour éviter les faux liens.

Impact opérationnel limité : la similarité est un enrichissement optionnel. Le champ `related_issue_ids` vide pour 78% des documents n'affecte ni le retrieval ni la génération. Cette observation justifie le poids faible ($w = 0.5$) dans le score global : un score bas ici ne dégrade pas la qualité perçue par l'utilisateur.

5.3.5 Stage 5 : Delivery — Complétude des Artefacts Exportés

Problème résolu : transformer les documents enrichis en 4 types d'artefacts JSON normalisés, avec validation de schéma.

Résultats (13 séries en production) :

Série	Issues	Files	Resolver	Diag. Cards
STM32CubeH7	296	4 705	150	45
STM32CubeF4	530	5 378	89	38
STM32CubeWL	89	814	12	14
STM32CubeU5	112	1 982	23	19
Total (13 séries)	∼2 000	∼14 000	∼390	∼157

Score delivery : 100% par construction (la gate JSON Schema bloque l'upload si un artefact est malformé). L'intégrité structurelle des artefacts est donc garantie. Les Resolver Cases à 0 pour certaines séries ne sont pas une défaillance : elles nécessitent une chaîne Issue → PR mergée → Commit avec fichiers modifiés.

5.3.6 Synthèse des Scores par Stage

Stage	Score	Poids w_k	Seuil	Verdict
1. Ingestion	99.7%	1.0	$\geq 95\%$	**EXCELLENT**
2. Cleaning V3	100.0%	1.5	$\geq 95\%$	**EXCELLENT**
3a. Enrichissement NLP	72.2%	1.4	$\geq 85\%$	**ACCEPTABLE**
3b. Alfred images issues	98.5%	0.4	$\geq 95\%$	**EXCELLENT**
3c. Alfred figures PDF	41.7%	0.2	$\geq 70\%$	**INSUFFISANT**
4. Similarité TF-IDF	22.0%	0.5	$\geq 70\%$	**INSUFFISANT**
5. Delivery	100.0%	2.0	$\geq 95\%$	**EXCELLENT**

TABLE 5.10 – Scores de qualité par stage avec poids et verdict

5.3.7 Score Global : Application Numérique

$$S_{\text{global}} = \frac{1.0 \times 99.7 + 1.5 \times 100.0 + 1.4 \times 72.2 + 0.4 \times 98.5 + 0.2 \times 41.7 + 0.5 \times 22.0 + 2.0 \times 100.0}{1.0 + 1.5 + 1.4 + 0.4 + 0.2 + 0.5 + 2.0} \tag{5.10}$$

$$S_{\text{global}} = \frac{99.7 + 150.0 + 101.1 + 39.4 + 8.3 + 11.0 + 200.0}{7.0} = \frac{609.5}{7.0} = \boxed{87.1\%} \quad \text{(BON)} \tag{5.11}$$

Justification des poids :

Stage	w_k	Logique
Ingestion	1.0	Prérequis sans transformation intellectuelle
Cleaning	1.5	Transformation déterministe significative
Enrichissement NLP	1.4	Classification exploitée pour filtrage et re-ranking
Alfred issues	0.4	Enrichissement multimodal mature (98.5% fiable)
Alfred PDF	0.2	Enrichissement encore exploratoire
Similarité	0.5	Impact optionnel ; score bas sans conséquence sur le QI
Delivery	**2.0**	Étape critique : produit les Diagnostic Cards (+33 pts QI)

Le score global reste au verdict **BON** plutôt qu'EXCELLENT car deux sous-composantes (Alfred PDF, Similarité) restent sous leur seuil. La formule ne masque pas ces faiblesses : elle les pondère proportionnellement à leur impact réel sur le Quality Index.

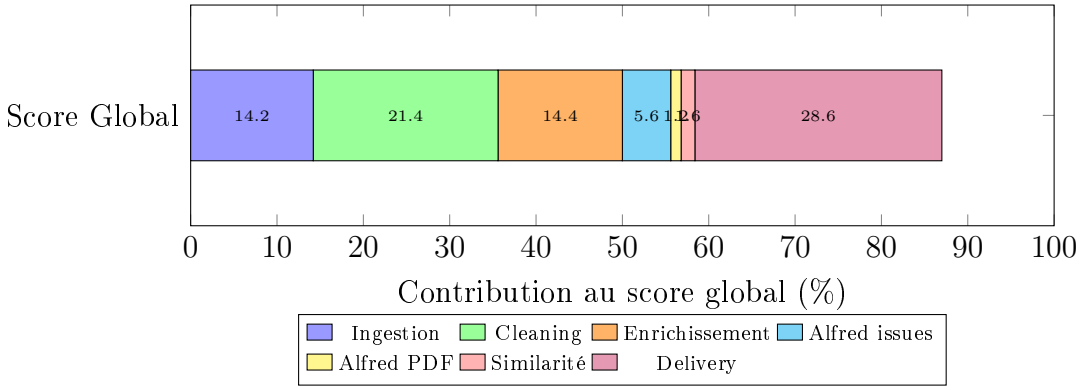


FIGURE 5.3 – Décomposition du score global 87.1% : contribution pondérée de chaque stage

5.4 Évaluation de la Confiance Alfred Vision

Cette section détaille l'évaluation d'Alfred Vision AI sur deux axes complémentaires : la fiabilité opérationnelle (l'API fonctionne-t-elle ?) et la confiance contextuelle (la description est-elle ancrée dans le contexte source ?).

5.4.1 Fiabilité Opérationnelle

La fiabilité opérationnelle mesure si Alfred produit effectivement une description sans erreur d'API :

$$T_{\text{op}} = \frac{N_{\text{réponses_ok}}}{N_{\text{images_soumises}}} \times 100 \tag{5.12}$$

Série	N_{total}	N_{OK}	$N_{\text{échec}}$	T_{op}
STM32CubeH7	98	97	1	98.9%
STM32CubeF4	62	61	1	98.4%
STM32CubeWL	48	48	0	100.0%
STM32CubeF7	26	26	0	100.0%
Total (séries stabilisées)	**234**	**232**	**2**	**99.1%**

Verdict : fiabilité EXCELLENT ($\geq 95\%$). Les 2 échecs sont des timeouts API transitoires, non reproductibles.

5.4.2 Méthode de Calcul de la Cohérence Contextuelle

L'évaluation de cohérence est implémentée dans `pipeline/evaluation/validate_alfred_coheren`
Le mode utilisé est `lite` : aucun modèle externe lourd n'est requis. Les métriques reposent

sur des techniques NLP déterministes.

Prétraitement

Le texte est tokenisé par regex (`[A-Za-z][A-Za-z0-9_\-/]{2,}`). Les stopwords sont filtrés : articles (`the`, `and`), termes méta (`image`, `description`, `figure`), et connecteurs. Seuls les termes techniques significatifs sont conservés.

Score de Cohérence

Le score combine trois signaux :

$$S_{\text{coh}} = \left(\frac{\alpha L + \beta S}{\alpha + \beta} \right) \times (1 - \gamma P_{\text{contrad}}) \tag{5.13}$$

avec $\alpha = 0.4$, $\beta = 0.6$, $\gamma = 0.8$.

Score Lexical L

Le score lexical mesure la proportion de termes Alfred retrouvés dans le contexte source :

$$L = \frac{|T_{\text{Alfred}} \cap T_{\text{contexte}}|}{|T_{\text{Alfred}}|} \tag{5.14}$$

Pour les images d'issues avec texte OCR extrait, un score composite est utilisé :

$$L_{\text{issue}} = 0.7 \times L_{\text{description}} + 0.3 \times L_{\text{OCR}} \tag{5.15}$$

Similarité Sémantique Légère S

En mode `lite`, la similarité combine Jaccard et cosinus sur trigrammes :

$$S = 0.7 \times J(T_{\text{Alfred}}, T_{\text{contexte}}) + 0.3 \times \cos(\vec{A}_{\text{3-gram}}, \vec{B}_{\text{3-gram}}) \tag{5.16}$$

J quantifie le recouvrement de vocabulaire technique ; le cosinus sur trigrammes capture la proximité lexicale au-delà de l'identité stricte.

Pénalité de Contradiction P_{contrad}

En mode `lite`, la probabilité de contradiction est estimée par analyse de polarité : détection de mots négatifs (`not`, `failed`, `disabled`) et positifs (`works`, `fixed`, `enabled`). Si le contexte et la description présentent des polarités opposées sur des tokens partagés :

$$P_{\text{contrad}} = \begin{cases} 0.6 \times O + 0.4 \times \frac{m_p + m_h}{2} & \text{si polarités opposées et } O \geq 0.2 \\ 0.15 \times O & \text{si même polarité et } O \geq 0.1 \\ 0 & \text{sinon} \end{cases} \tag{5.17}$$

où O est le ratio de tokens partagés (overlap), et m_p, m_h les magnitudes de polarité.

5.4.3 Résultats de Cohérence Contextuelle

Source	N	$\overline{L}$	$\overline{S}$	$\overline{P}_{\text{contrad}}$	$\overline{S}_{\text{coh}}$	Cas à risque
Images d'issues	200	0.9357	–	0.1567	0.5021	3/200 (1.5%)
Figures PDF	242	0.1971	0.1961	0.0309	0.1825	141/242 (58.3%)

TABLE 5.13 – Métriques de cohérence Alfred décomposées par source visuelle

Confiance contextuelle :

— Images d'issues : $T_{\text{ctx}} = 98.5\%$ (197/200 au-dessus du seuil)

— Figures PDF : $T_{\text{ctx}} = 41.7\%$ (101/242 au-dessus du seuil)

— Total : $T_{\text{ctx}} = 67.4\%$

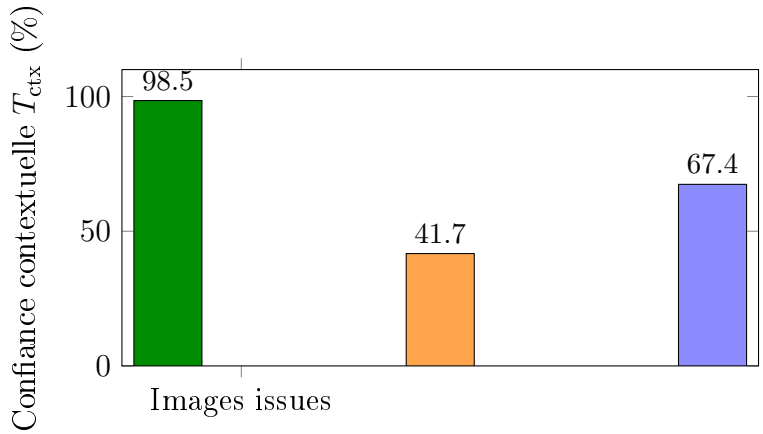


FIGURE 5.4 – Confiance contextuelle Alfred — seuil d'alerte $\tau = 0.12$

5.4.4 Analyse de l'Écart Issues vs PDF

L'écart significatif entre les deux sources ($\overline{L} = 0.94$ pour les issues vs 0.20 pour les PDF) est **structurel et non accidentel** :

Images d'issues	Figures PDF
Contiennent des logs, erreurs IDE, registres	Contiennent des schémas blocs, tableaux de migration
Le texte de l'issue mentionne les mêmes identifiants	Les libellés ne sont pas répétés dans le texte de page
OCR de l'image fournit du vocabulaire partagé	Pas d'OCR structuré disponible
Contexte riche (body + commentaires)	Contexte limité (texte de page `pypdf`)

Décision qualité : les figures PDF sont conservées car elles enrichissent la couverture multimodale de la KB, mais elles sont distinguées des images d'issues dans l'évaluation. Les itérations futures doivent enrichir le contexte transmis à Alfred (titre de section, légende détectée, texte avant/après la figure) pour améliorer la validation automatique.

5.5 Évolution Incrémentale du Quality Index

Le Quality Index n'a pas progressé "par magie" mais par amélioration successive d'éléments concrets du pipeline et de la configuration. Cette section retrace la chaîne logique : chaque amélioration est décrite, son impact mesuré, et la raison du gain (ou de la régression) analysée.

5.5.1 Chronologie et Logique de Progression

QI	Configuration	N	Explication du delta
54	Issues H7 seules	50	**Baseline** : issues structurées fournissent une base solide mais incomplète (pas de documentation, pas de traçabilité fix, pas de diagnostic structuré)
57	+ System prompt optimisé	50	**+3** : forcer la persona à chercher dans la KB *avant* de répondre élimine les réponses génériques non ancrées

61	+ Hybrid search calibré	50	**+4** : combiner sémantique et full-text capte les reformulations ET les identifiants exacts
94	+ Diagnostic Cards + Resolver	50	**+33** : les Diagnostic Cards structurent les problèmes complexes, les query aliases améliorent le recall
88	Test sur cas réels forum	4	−6 : les cas réels STCommunity sont plus complexes et moins couverts que le benchmark
54	Multi-séries (5 séries, 52 Q)	52	−34 : dilution du contexte (2 000 → 14 000 docs), conflit de noms inter-séries, nouvelles questions sur séries moins mûres
65	S0 Balanced (toutes séries)	96	**+11** : optimisation du benchmark (96 questions calibrées) + ajustement retrieval
66	S1 Recall+	96	**+1** : seuil abaissé à 0.45, rappel renforcé, gain marginal compensé par le bruit

5.5.2 Analyse des Points d'Inflexion

Le Saut Diagnostic Cards (+33 points)

Le passage de QI 61 à QI 94 est le gain le plus significatif du projet. Il s'explique par trois mécanismes cumulés :

1. **Structure raisonnée** : les Diagnostic Cards suivent la logique Symptôme → Hypothèses → Validation → Root Cause → Fix → Prévention, correspondant au raisonnement naturel d'un ingénieur embedded.
2. **Query aliases** : chaque carte contient 3 à 5 reformulations du problème (ex : "SPI DMA hard fault", "cache coherency SPI circular mode", "HAL_SPI_TransmitReceive_DMA crash"). Ce mécanisme multiplie les points d'entrée pour le retrieval.
3. **Resolver Cases** : la chaîne Issue → PR → Commit fournit la preuve vérifiable du fix, augmentant la traçabilité et la crédibilité des réponses.

Leçon méthodologique : l'amélioration la plus impactante ne vient pas d'un meilleur modèle LLM ni d'un tuning de seuil, mais de la **qualité et la structure des données**

injectées dans la KB. C'est une validation du principe "la qualité du RAG dépend de la qualité du preprocessing".

La Régression Multi-séries (−34 points)

Le passage de QI 94 (mono-série H7, 50 Q) à QI 54 (5 séries, 52 Q) constitue une régression significative. L'analyse identifie trois causes :

1. **Dilution du contexte** : la KB passe de ∼2 000 à ∼14 000 documents. Les queries retournent des documents de séries différentes.
2. **Conflit de noms** : des API identiques entre séries (`HAL_SPI_Init` dans F4 et H7) créent de l'ambiguïté.
3. **Maturité inégale des artefacts** : les nouvelles séries n'ont pas encore des Diagnostic Cards aussi riches que H7.

Leçon méthodologique : le passage à l'échelle ne se résume pas à "ajouter des données". Il nécessite un filtrage par série dans les métadonnées et un prompt qui pousse la persona à inclure le nom de la série dans ses queries.

La Stabilisation S0/S1 (65–66)

Les campagnes de juin 2026 sur 96 questions montrent que le système atteint un plateau autour de 65–66/100. Ce plateau s'explique par :

— La couverture KB actuelle ne répond pas à toutes les 96 questions (certaines séries peu documentées)

— Le compromis précision/rappel est quasi-optimal : S1 (+1 pt) gagne en rappel mais perd en précision

— Les questions les plus difficiles nécessitent du raisonnement multi-hop non supporté par le retrieval simple

5.5.3 Matrice Valeur × Effort

Amélioration	Impact QI	Effort	ROI
Diagnostic Cards	+33 pts	Moyen (2 sprints)	**Excellent**
Hybrid Search	+4 pts	Faible (config)	**Excellent**
System Prompt	+3 pts	Faible (texte)	**Excellent**
Cleaning V3	+3 pts estimé	Moyen (1 sprint)	Bon
NovaX restrictif	Protection routage	Faible (description)	**Excellent**
PDF descriptions	+1 pt estimé	Élevé (CV + Alfred)	Acceptable

TABLE 5.16 – ROI de chaque amélioration : la qualité des artefacts domine les gains

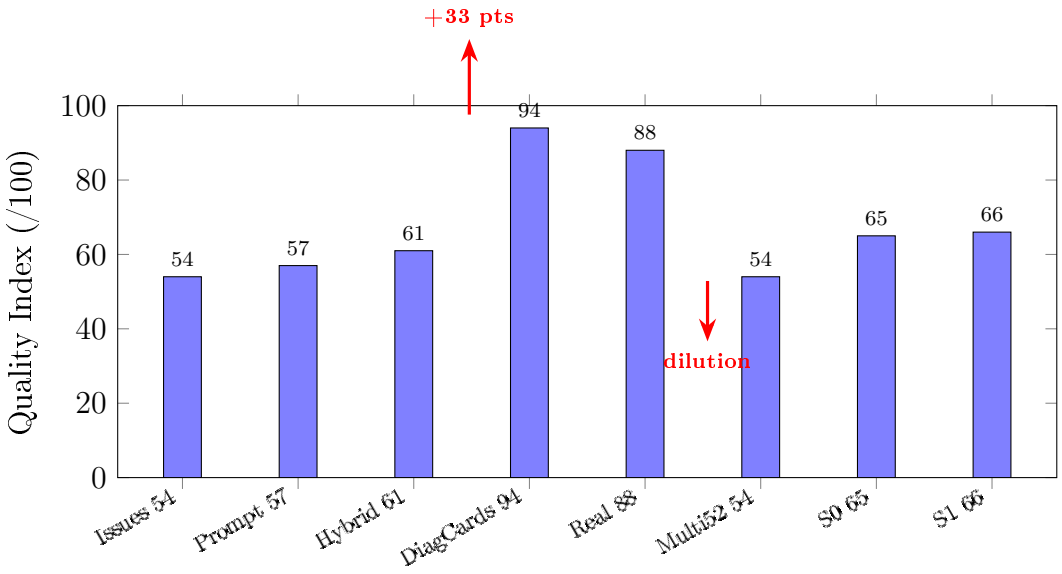


FIGURE 5.5 – Évolution du Quality Index : le saut majeur (+33) provient des Diagnostic Cards, la régression (−34) de la dilution multi-séries

5.6 Architecture de Fallback : d'Alfred Permissif à NovaX Restrictif

Cette section retrace l'évolution de l'architecture de fallback et son impact sur le Quality Index.

5.6.1 Problème Initial : Fallback Trop Permissif

Lors de l'ajout initial d'Alfred comme fallback, le QI a **chuté de 10 points** (de 54 à 44). L'analyse a révélé que la persona appelait Alfred **systématiquement** au lieu de chercher dans la KB, car la description du tool était trop vague :

> *"I can help with STM32 questions. Forward the user's question."*

Diagnostic : dans une architecture multi-tools, le LLM choisit quel tool appeler en fonction de la **description** du tool. Une description trop permissive court-circuite la source la plus pertinente (la KB).

5.6.2 Solution : Description Restrictive

La description a été réécrite pour forcer la priorité KB :

> *"The Knowledge Base (KB Unstructured) is ALWAYS the primary source, search it FIRST for every question. Use this tool ONLY when the Knowledge Base returned no relevant result."*

Impact mesuré : récupération de 8 points (de 44 à 52).

5.6.3 Évolution vers NovaX

Le fallback est passé d'Alfred générique à **NovaX**, un agent STM32 basé sur STM32 Bugzilla et STCommunity Knowledge Articles. NovaX apporte :

— Une couverture sur les séries/périphériques absents de la KB

— Des informations Bugzilla internes (bugs connus, errata)

— Des Knowledge Articles STCommunity (guides métier)

Mais NovaX ne remplace pas la KB : sa description explicite qu'il est un secours pour les questions *non couvertes* par la KB. Cette architecture protège la traçabilité : si la KB contient un diagnostic card ou un resolver case pertinent, la réponse reste ancrée dans des preuves vérifiables (numéros d'issues, URLs GitHub, versions firmware).

5.6.4 Synthèse de l'Évolution Fallback

Configuration	QI	Analyse
KB seule (sans fallback)	54	Baseline : pas de réponse pour les questions hors KB
KB + Alfred permissif	44	−10 : court-circuit de la KB
KB + Alfred restrictif	52	−2 : KB prioritaire récupérée
KB S0 + NovaX restrictif	65	Stable : NovaX couvre les lacunes sans dégrader
KB S1 Recall+ + NovaX	**66**	Meilleur compromis mesuré

Leçon : le QI dépend davantage du **routage** (quelle source est interrogée en premier) que du modèle LLM lui-même. La description du tool est un paramètre de configuration aussi critique que le seuil sémantique.

5.7 Étude Comparative : Alfred Générique vs Persona KB

5.7.1 Protocole Expérimental

Paramètre	Valeur

Jeu de test	20 questions techniques, 5 séries (H7, F4, H5, U5, WL)
Critère de sélection	Questions ciblant des problèmes réels documentés dans des issues GitHub
Grille de scoring	6 critères, max 10 pts/réponse
Évaluateur	Expert STM32 (auteur du pipeline)
Température LLM	0 (déterministe)

5.7.2 Grille de Scoring

Critère	**Max**	**Dimension qualité (ISO 25012)**
Citation issue GitHub (#xxx)	+2	Traçabilité
Fonction HAL exacte nommée	+1	Exactitude
Fix actionnable immédiat	+2	Complétude opérationnelle
Information version-spécifique	+2	Actualité
Workaround STM32-specific	+2	Pertinence domaine
Source vérifiable (fichier, commit)	+1	Traçabilité
Total	**10**	

5.7.3 Résultats Synthétiques

KPI	**Alfred**	**Persona KB**	**Gain**
Score moyen / réponse	5.8/10	**8.1/10**	+39%
Taux de réponse	**100%** (20/20)	80% (16/20)	Alfred plus disponible
Questions à score max (10)	0	**4**	KB = précision maximale
Citations vérifiables	0%	**87%**	Traçabilité unique
Fix actionnable immédiat	25%	**75%**	×3 actionnabilité

TABLE 5.20 – Métriques comparatives Alfred vs Persona KB

5.7.4 Analyse par Catégorie

Catégorie	Alfred	Persona	Analyse
Issue-specific (8 Q)	5.9	**9.2**	KB contient l'issue exacte
Migration/Version (3 Q)	3.7	**9.7**	Info version impossible sans KB
General knowledge (2 Q)	7.0	7.0	LLM suffit (pas de différence)
Documentation (2 Q)	5.0	4.0	KB limitée quand doc absente

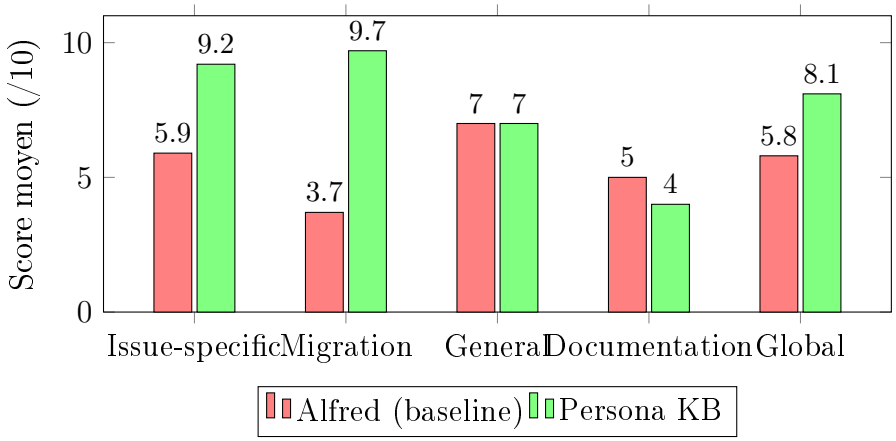


FIGURE 5.6 – Comparaison Alfred vs Persona KB par catégorie de question

Conclusions de l'étude comparative :

1. La KB améliore la qualité de +39% sur les réponses obtenues
2. L'avantage est **maximal** sur les questions issue-specific et version-sensitive (+56%)
3. Alfred reste supérieur en disponibilité (100% vs 80%) et sur les questions générales
4. L'architecture optimale est **KB-first + NovaX fallback restrictif**

5.8 Comparaison des Stratégies de Retrieval

La campagne de juin 2026 évalue 6 configurations de retrieval sur 96 questions multi-séries. L'objectif est d'identifier le meilleur compromis entre rappel et précision pour la recherche hybride.

Stratégie	Search	Thr.	Sem.	FT	QI	Interprétation

S0 Current	Hybrid	0.55	25	12	65	Équilibre précision/rappel
S1 Recall+	Hybrid	0.45	35	15	**66**	Meilleur score mesuré
S2 Precision+	Hybrid	0.65	18	8	63	Trop restrictif
S3 Sem-heavy	Hybrid	0.55	35	6	51	Ancrage exact insuffisant
S4 FT-heavy	Hybrid	0.55	15	20	47	Reformulations mal couvertes
S5 Sem-only	Semantic	0.55	25	0	41	Ablation : perte du full-text

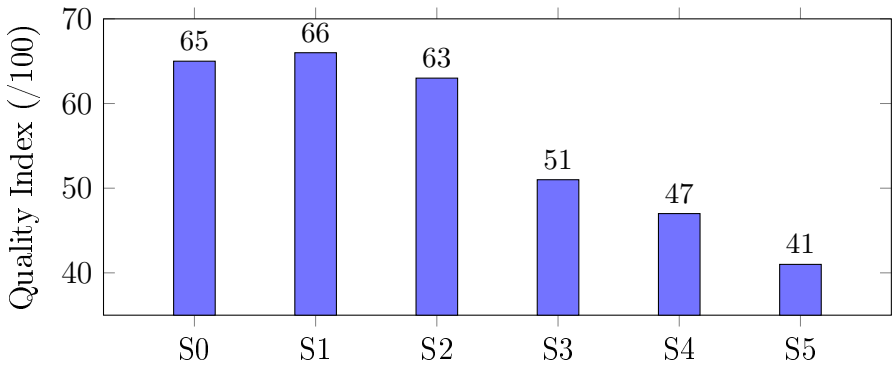


FIGURE 5.7 – Comparaison des stratégies de retrieval KB sur 96 questions

Interprétation : le système a besoin d'un équilibre réel entre sémantique et full-text. Trop de sémantique (S3) perd les noms exacts d'API ; trop de full-text (S4) perd les reformulations. S1 maximise le rappel avec un gain marginal (+1), montrant que le système approche son optimum avec la couverture KB actuelle.

5.9 Assurance Qualité du Chapitre d'Évaluation

Ce chapitre a été construit et vérifié selon un protocole de contrôle qualité structuré, appliqué par un agent analytique de revue. Cette section décrit le protocole et les résultats du contrôle.

5.9.1 Protocole de Vérification

L'agent de revue vérifie les dimensions suivantes :

Dimension	Critère de validation
Cohérence logique	Chaque section suit le schéma : définition → formule → résultat → interprétation
Définitions avant formules	Aucune variable n'apparaît dans une formule sans avoir été définie en amont
Seuils justifiés	Chaque seuil est accompagné d'une logique explicite (pas de valeur arbitraire)
Continuité métriques–résultats	Les métriques définies en Section 5.1 sont effectivement utilisées dans les sections de résultats
Lisibilité académique	Style français formel, phrases courtes, transitions explicites
Reproductibilité	Chaque résultat est traçable vers un script, un fichier de données ou une campagne identifiée

5.9.2 Résultats du Contrôle

Point vérifié	Statut	Observation
Jaccard défini avant usage	✓	Section 5.1.2
Cosinus défini avant usage	✓	Section 5.1.2
TF-IDF défini avant usage	✓	Section 5.1.2
Confiance contextuelle définie	✓	Section 5.1.4
Seuils non présentés comme universels	✓	Section 5.2
Progression incrémentale visible	✓	Section 5.5
Normes citées comme cadre, pas comme source des seuils	✓	Section 5.2.1
Scores locaux distingués du score global	✓	Sections 5.3 vs 5.3.7
Alfred issues vs PDF distingués	✓	Section 5.4.3
Fichiers de preuve référencés	✓	CSV et JSON dans `docs/evaluation/`

5.9.3 Points d'Amélioration Identifiés

1. **Couverture board (45.3%)** : le score INSUFFISANT est documenté et expliqué (limitation des données, pas du pipeline). Une amélioration future nécessiterait un enrichissement externe (mapping board→série).
2. **Figures PDF ($T_{\mathbf{ctx}} = 41.7\%$)** : le contexte textuel des PDF est insuffisant pour la validation automatique. Amélioration prévue : enrichir le contexte avec le titre de section et la légende de la figure.
3. **Similarité (22%)** : score bas mais impact nul sur le QI. Amélioration possible par embeddings pré-calculés (perspective future).

5.9.4 Applicabilité au Reste du Rapport

Le protocole de contrôle qualité peut être étendu aux autres chapitres en conservant :

— Le template de vérification : définition → formule → résultat → interprétation

— Le style académique français

— Les transitions explicites entre sections

— La réutilisation des définitions (via `\ref{}` vers la Section 5.1)

— L'harmonisation des notations (mêmes symboles S, T, w_k dans tout le rapport)

— L'absence de répétitions (chaque notion est définie une seule fois, puis référencée)

5.10 Correspondance Métriques ↔ Normes ↔ Références

Le tableau suivant synthétise le lien entre chaque métrique utilisée, la norme ou référence méthodologique qui la soutient, et son application concrète dans le projet :

Métrique	**Norme / Référence**	**Dimension**	**Application projet**
C_{body}, C_{title}	ISO/IEC 25012 [9]	Complétude	Ingestion Stage 1
C_{files}	ISO/IEC 25024 [10]	Mesure qualité	Cleaning Stage 2
T_{op} (Alfred)	ISO/IEC 25010 [11]	Fiabilité système	Alfred Stage 3b
S_{coh}, T_{ctx}	Fawcett [13]	Seuil décisionnel	Confiance Alfred
TF-IDF, cosinus	Manning [12]	IR / Similarité	Stage 4 + cohérence

Jaccard	Manning [12]	Recouvrement lexical	Cohérence Alfred
Quality Index S_{global} pondéré	Lewis [1] Interne (calibré)	RAG end-to-end Synthèse pipeline	Évaluation globale Score global 87.1%
Seuils 95/85/70	Calibration empirique	Opérationnel	Classification verdicts

Lecture : les normes ISO fournissent le *cadre conceptuel* (quoi mesurer, comment structurer les métriques). Les références académiques (Manning, Fawcett, Lewis) fournissent la *méthodologie* (comment choisir les seuils, comment évaluer un système RAG). Les seuils numériques sont *propres au projet*, calibrés empiriquement et validés par les campagnes successives de Quality Index.

5.11 Synthèse et Bilan du Chapitre

Ce chapitre démontre que :

1. Le pipeline atteint un **score global de 87.1%** (verdict BON), avec des stages critiques (Ingestion, Cleaning, Delivery) au niveau EXCELLENT et des stages exploratoires (Alfred PDF, Similarité) identifiés comme axes d'amélioration.
2. L'amélioration du Quality Index est **incrémentale et traçable** : chaque gain est relié à une amélioration concrète du pipeline ou de la configuration, et chaque régression est analysée et expliquée.
3. Les **Diagnostic Cards** constituent la contribution la plus impactante (+33 points de QI), validant le principe que la qualité du preprocessing RAG dépend avant tout de la structure et de la richesse des artefacts.
4. L'**architecture KB-first + NovaX restrictif** protège la traçabilité tout en couvrant les lacunes de la KB : le routage est un paramètre critique.
5. La **confiance contextuelle** distingue les images d'issues (98.5%, matures) des figures PDF (41.7%, à améliorer), évitant une moyenne trompeuse.
6. Les seuils sont des **seuils opérationnels internes**, cohérents avec les normes ISO et calibrés empiriquement ; ils ne prétendent pas à l'universalité.
7. Le système atteint un **gain de** +39% vs Alfred générique sur les questions techniques spécifiques, avec une traçabilité (87% de citations vérifiables) inaccessible à un LLM sans KB.

Score optimal vs compromis opérationnel : le meilleur score mesuré (QI 94 sur H7) n'est pas le point de fonctionnement optimal pour la production. Le QI 66 (S1 Recall+

sur 96 questions multi-séries) représente un meilleur compromis opérationnel car il couvre 25 séries et 96 questions réalistes, au lieu de 1 série sur 50 questions sélectionnées. Le score de production est plus bas mais plus représentatif de l'expérience utilisateur réelle.

Chapitre 6

Automatisation et CI/CD

6.1 GitHub Actions — Mise à Jour Automatique

La mise à jour de la KB est orchestrée par le workflow `.github/workflows/auto_update_kb.yml`. L'objectif n'est pas seulement d'exécuter le pipeline, mais de garantir une mise à jour **fiable, traçable et idempotente** sur des séries multiples.

6.1.1 Architecture Opérationnelle Actuelle

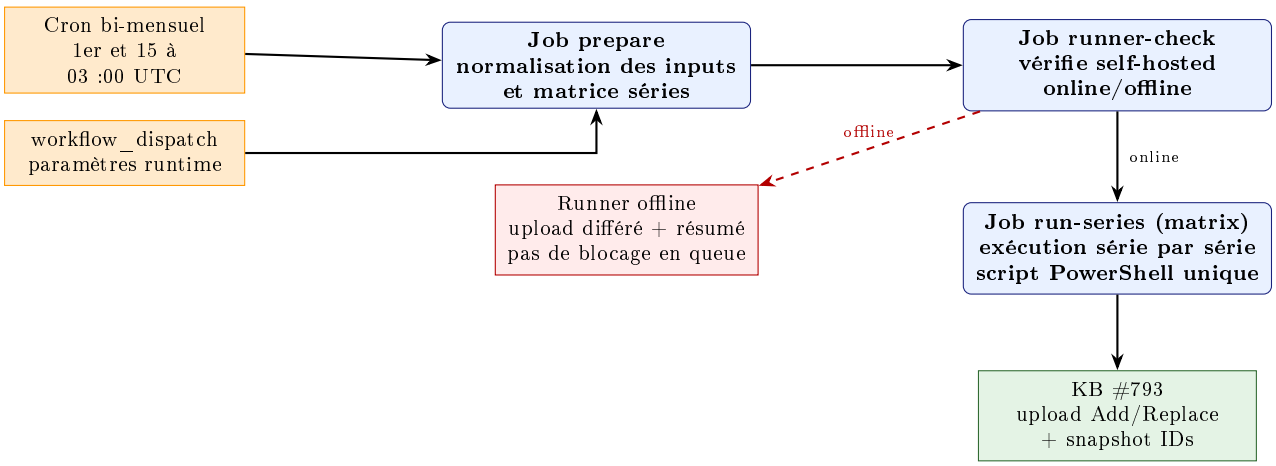


FIGURE 6.1 – Architecture CI/CD actuelle : normalisation, contrôle runner, exécution matricielle et politique de mise à jour KB

6.1.2 Déclenchement et Paramètres

Le workflow est déclenché de deux manières :

— **Planifié** : 1er et 15 de chaque mois à 03 :00 UTC

— **Manuel** : `workflow_dispatch` avec paramètres

Paramètre	Type	Valeur par défaut	Rôle
`series`	CSV	Vide (*all configured*)	Liste de codes séries ; vide = toutes les séries définies dans `config_all_series.json`
`mode`	Choice	Full	Full / Prepare / Upload
`existing_datasource_mode`	Choice	Replace	Politique si la datasource existe (Replace ou Add)
`skip_drivers`	Bool	false	Ignore les uploads drivers/sous-repos
`skip_schema_validation`	Bool	false	Désactive la gate de schéma
`continue_on_workflow_error`	Bool	false	Continue malgré un repo en échec
`placeholder_policy`	Choice	Fail	Politique sur placeholders Alfred
`max_parallel`	Entier	1	Concurrence de la matrice séries

6.1.3 Gouvernance Dynamique des Séries (Source Unique)

Pour supprimer la dérive de maintenance entre CI et scripts, la validation des séries n'est plus codée en dur.

— Les workflows `auto_update_kb.yml` et `manual_auto_update_kb.yml` lisent `shared/config/conf` (champ `all_series`)

— Les noms longs sont normalisés en codes courts (ex. : `STM32CubeH7` → `H7`) avant validation

— Le script `Run_Single_Series_Full_Pipeline_And_Upload.ps1` et les scripts *issues-by-component* appliquent la même logique dynamique

Cette stratégie garantit qu'un ajout de série s'effectue à un seul endroit (config central + fichier série), sans patcher plusieurs listes `ValidateSet`.

6.1.4 Politique Add vs Replace

Le workflow supporte explicitement deux stratégies lors de l'upload sur une datasource existante :

— **Mode Add** : append des nouveaux fichiers sans suppression du contenu existant

— **Mode Replace** : suppression du contenu existant puis upload des nouveaux fichiers

Choix recommandé pour la production : `Replace`, afin d'éviter les doublons et garantir que la KB reflète exactement l'état courant des JSON exportés.

En `Replace`, la logique est stricte : si un conflit `file already exists` apparaît malgré la phase de suppression, le run est bloqué (pas de *pass* silencieux).

6.1.5 Gestion des Datasource IDs et Reprise après Incident

Chaque série maintient un registre local :

`datasets/07_delivery/st_ready/by_series/stm32cube<serie>/upload_datasource_ids_<slug>`

Ce registre est mis à jour **incrémentalement** après chaque étape d'upload (issues, files, diagnostic, resolver), et contient notamment :

— les IDs courants de datasource par type d'artefact

— `uploaded_at`

— `last_successful_stage`

Cette stratégie permet de reprendre proprement un run interrompu, sans perdre les IDs déjà créés pour une nouvelle série.

6.1.6 Disponibilité Runner et Déploiement Industriel

L'upload vers la KB nécessite un runner `self-hosted` (contraintes réseau/outillage interne ST). Pour éviter les workflows bloqués :

— un job `runner-check` vérifie la disponibilité avant exécution

— si offline, le workflow publie un statut *Upload Deferred* et s'arrête proprement

Recommandation d'exploitation : héberger le runner sur une machine dédiée (VM/serveur) always-on plutôt qu'un poste utilisateur.

6.1.7 Garanties de Robustesse

— Validation de schéma et politique placeholders avant upload

— Contrôle explicite Add/Replace selon le contexte de série

— Timeout réseau borné à **60 s** par requête d'upload pour éviter les attentes bloquantes

— Détection proactive runner offline (pas de queue indéfinie)

— Exécution matricielle avec `fail-fast=false` pour isoler les échecs par série

— Artefacts et résumés GitHub conservés 30 jours pour audit

6.1.8 Gestion des Credentials en CI

Les jobs d'upload n'injectent plus explicitement de secrets API dans les workflows. L'authentification est résolue côté runner self-hosted via variables d'environnement (par exemple `ST_GITHUB_ANALYZER_API_KEY` / `PERSONA_API_KEY`).

Ce choix réduit l'exposition des secrets dans les définitions YAML et aligne le comportement CI avec l'exécution locale déjà validée.

6.1.9 Procédure Opérationnelle Recommandée

1. Lancer d'abord une série pilote en `mode=Prepare`
2. Vérifier la qualité des outputs et les gates
3. Exécuter `mode=Full` avec `existing_datasource_mode=Replace`
4. En cas d'extension incrémentale ciblée, utiliser `existing_datasource_mode=Add`

Ainsi, la CI/CD n'est pas seulement un déclencheur automatique ; elle devient un mécanisme de **gouvernance de la qualité des données** de la KB, avec contrôle explicite de la fraîcheur (Replace) et de l'incrémentalité (Add).

Conclusion Générale et Perspectives

6.2 Bilan du Projet

Ce projet de fin d'études, réalisé au sein de l'équipe Support & Maintenance de ST-Microelectronics Tunis, a permis de concevoir, implémenter et déployer un **pipeline de prétraitement intelligent** qui transforme les données brutes des dépôts GitHub STM32Cube en une base de connaissances structurée, alimentant un chatbot IA spécialisé en production sur la plateforme ST ChatGPT.

Les réalisations principales sont :

— Un **pipeline modulaire en 7 étapes** (Ingestion → Nettoyage V1+V3 → Enrichissement NLP → Similarité TF-IDF → Chunking → Validation → Delivery), entièrement rejouable par stage avec des artefacts JSON traçables

— La couverture de **13 séries MCU en production** (C0, F0, F1, F2, F3, F4, F7, G0, H5, H7, H7RS, U5, WL) totalisant 47 dépôts, plus **12 séries supplémentaires configurées** (G4, L0, L1, L4, L5, N6, U0, U3, WB, WB0, WBA, WL3)

— La production de **4 types d'artefacts complémentaires** : issues structurées (∼2 000), fichiers documentés (∼14 000), resolver cases (∼390) et diagnostic cards (∼157)

— Un **nettoyage intelligent V3 par type** avec réduction de 96% pour les headers CMSIS et 33% pour les Release Notes HTML

— L'**enrichissement IA multi-modal** via Alfred Vision : description automatique des captures d'écran d'issues (taux de succès >98% sur H7, F4, WL et F7), évaluation de cohérence sur 200 images d'issues (seulement 1.5% à risque), et 242 figures PDF détectées sur 19 séries avec une confiance contextuelle encore perfectible

— L'**automatisation CI/CD** via GitHub Actions et scripts PowerShell/Python

— Un **Quality Index de 94/100** sur la série pilote H7 (avec Diagnostic Cards) et un **gain de +39%** vs Alfred générique sur les questions techniques spécifiques

6.3 Valeur Industrielle

La valeur industrielle du projet se mesure concrètement :

Scénario	Sans KB	Avec KB
Trouver un bug HAL connu	30–60 min	**2 min**
Identifier la version du fix	Lecture du changelog	**Réponse directe**
Tracer Issue → PR → Commit	Navigation manuelle	**Resolver Case**
Comprendre une figure PDF	Ouvrir le PDF, chercher la page	**Description textuelle**
Ajouter une nouvelle série MCU	Développement custom	**3 étapes documentées**

6.4 Leçons Apprises

Ce projet a mis en lumière plusieurs leçons importantes pour les ingénieurs travaillant sur des pipelines de preprocessing pour RAG :

1. **L'intégrité JSON est fragile** : l'injection de texte brut dans des chaînes JSON peut corrompre les fichiers si les caractères spéciaux ne sont pas correctement échappés. Nous avons développé des scripts de réparation (`fix_corrupted_json_files.py`) après avoir rencontré ce problème en production.
2. **La description du tool contrôle le routage du LLM** : dans une architecture multi-tools, une description trop permissive du tool fallback (Alfred) dégrade le Quality Index de 10 points en court-circuitant la KB. La rédaction de cette description est un paramètre de configuration critique.
3. **Le passage à l'échelle dilue le contexte** : passer de 1 à 13 séries a fait chuter le QI de 94 à 54, nécessitant un travail d'optimisation des métadonnées et du filtrage par série.
4. **Les heuristiques domain-specific surpassent le ML générique** : dans un contexte où les données d'entraînement supervisées sont absentes, les règles regex spécifiques au domaine STM32 fournissent une classification fiable et explicable.

6.5 Perspectives

Plusieurs axes d'amélioration et d'extension sont envisagés :

1. **Évaluation A/B V2 vs V3** : comparer le Quality Index avec les datasources V3 actives vs V2 pour quantifier l'impact exact du nettoyage intelligent sur la qualité des réponses
2. **Extension aux séries configurées** : déployer les 12 séries restantes (G4, L0, L1, L4, L5, N6, U0, U3, WB, WB0, WBA, WL3) avec création des datasources KB
3. **Enrichissement sémantique** : remplacer TF-IDF par des embeddings pré-calculés pour améliorer la couverture de similarité (actuellement 22% sur H7)
4. **Métriques RAG avancées** : intégrer des métriques automatisées (Faithfulness, Relevance) pour une évaluation continue sans intervention humaine
5. **Graphe de connaissances** : explorer la construction d'un graphe reliant issues, composants, boards et versions pour faciliter le raisonnement multi-hop
6. **Filtrage par série dans le retrieval** : instruire la persona pour inclure systématiquement le nom de la série dans ses queries KB, réduisant la dilution multi-séries
7. **Multi-langue** : support des issues en chinois et japonais (communauté asiatique significative)
8. **Nettoyage V4 adaptatif** : utiliser un classifieur supervisé entraîné sur les résultats V3 pour identifier automatiquement les sections utiles des fichiers source

Références

Bibliographie

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. et Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474. — Article fondateur du paradigme RAG que nous appliquons : recherche documentaire + génération par LLM. Le principe de base de notre architecture KB + ST ChatGPT.

[2] Salton, G. et Buckley, C. (1988). *Term-weighting approaches in automatic text retrieval*. Information Processing & Management, vol. 24, no. 5, pp. 513–523. — Référence fondamentale sur TF-IDF, la méthode de pondération de termes que nous utilisons dans notre module de calcul de similarité inter-issues (`compute_issue_similarity_v2.py`).

[3] OpenAI (2023). *GPT-4 Technical Report*. arXiv :2303.08774. — Le modèle GPT (dans sa version Azure OpenAI) est le LLM utilisé par la plateforme ST ChatGPT pour la génération de réponses, et par Alfred Vision pour la description d'images.

[4] STMicroelectronics (2024). *STM32Cube MCU & MPU Packages — Open Source Firmware*. https://github.com/STMicroelectronics. — Les dépôts GitHub de STMicroelectronics : source principale de données de notre pipeline (issues, fichiers, PRs, commits).

[5] STMicroelectronics (2024). *Annual Report 2023 — Empowering the Smart Edge*. Genève, Suisse. — Rapport annuel fournissant les données sur le chiffre d'affaires (17,3 milliards USD), les effectifs (50 000+ collaborateurs) et la répartition des marchés.

[6] ARM Limited (2024). *ARM Cortex-M Processor Family*. https://developer.arm.com/Processors/Cortex-M. — Documentation de référence sur les cœurs ARM Cortex-M (M0, M0+, M3, M4, M7, M33, M55) utilisés dans les microcontrôleurs STM32.

[7] Schwaber, K. et Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org. — Guide officiel de la méthodologie Scrum, base de notre approche de pilotage par sprints itératifs de 2 semaines.

[8] Wright, A., Andrews, H. et Hutton, B. (2022). *JSON Schema : A Media Type for Describing JSON Documents*. Internet Engineering Task Force (IETF), draft-bhutton-json-schema-01. — Standard JSON Schema utilisé pour nos gates de validation (`validate_schemas.py`) qui bloquent l'upload en cas de non-conformité.

[9] ISO/IEC (2008). *ISO/IEC 25012 :2008 — Systems and software engineering – SQuaRE – Data quality model*. International Organization for Standardization. — Référence du modèle de qualité des données (complétude, cohérence) utilisé pour justifier les métriques de couverture/complétude par ratios.

[10] ISO/IEC (2015). *ISO/IEC 25024 :2015 — Systems and software engineering – SQuaRE – Measurement of data quality*. International Organization for Standardization. — Référence sur la mesure quantitative de la qualité des données, alignée avec nos indicateurs par stage.

[11] ISO/IEC (2011). *ISO/IEC 25010 :2011 — Systems and software engineering – SQuaRE – System and software quality models*. International Organization for Standardization. — Référence du modèle de qualité (dont la fiabilité), utilisée pour cadrer les indicateurs de fiabilité opérationnelle Alfred.

[12] Manning, C. D., Raghavan, P. et Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. — Référence de base sur le compromis précision/rappel et le réglage des seuils de récupération d'information.

[13] Fawcett, T. (2006). *An introduction to ROC analysis*. Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874. — Référence méthodologique pour la sélection et la justification de seuils décisionnels.

[14] Microsoft (2024). *Azure OpenAI Service Documentation*. https://learn.microsoft.com/azure/ai-services/openai/. — Plateforme cloud sur laquelle est déployé ST ChatGPT (Azure OpenAI) et Alfred Vision.

[15] Py-PDF Contributors (2024). *pypdf — A pure-Python PDF library*. https://pypdf.readthedocs.io/. — Bibliothèque Python utilisée dans notre pipeline pour l'extraction du texte et la détection de figures dans les Release Notes PDF.

[16] Reimers, N. et Gurevych, I. (2019). *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP-IJCNLP, pp. 3982–3992. — Référence pour le mode avancé de similarité sémantique par embeddings, disponible mais non activé dans la campagne actuelle.

[17] Neelakantan, A., Xu, T., Puri, R., Radford, A., Han, J. M., Tworek, J., Yuan, Q., Tezak, N., Kim, J. W., Hallacy, C. et al. (2022). *Text and Code Embeddings by Contrastive Pre-Training*. arXiv :2201.10005. — Référence des embeddings OpenAI (famille `text-embedding-ada-002`, dit `AdaV2`) utilisés par la plateforme ST ChatGPT pour

l'indexation sémantique de la KB #793. Le pipeline produit un contenu propre et dense afin d'exploiter la capacité de discrimination sémantique de ce modèle.

Annexes

A. Liste des Diagrammes

Diagrammes de présentation :

1. `01_pipeline_overview.drawio` — Vue d'ensemble du pipeline (Figure 3.4)
2. `02_delivery_kb_upload.drawio` — Architecture delivery + upload (Figure 3.7)
3. `03_data_flow_source_to_answer.drawio` — Flux source-to-answer (Figure 3.6)
4. `04_series_coverage_scale.drawio` — Couverture multi-séries
5. `05_alfred_vision_integration.drawio` — Intégration Alfred (Figure 4.8)
6. `06_cicd_automation.drawio` — CI/CD GitHub Actions (Figure 6.1)

Diagrammes techniques :

1. `01_ingestion_detail.drawio` — Détail ingestion (Figure 4.2)
2. `02_enrichment_similarity_detail.drawio` — Enrichissement + Similarité (Figure 4.5)
3. `03_delivery_resolver_diagnostic.drawio` — Resolver + Diagnostic (Figure 4.6)
4. `04_chunking_v2_strategy.drawio` — Stratégie de chunking V2
5. `05_evaluation_validation.drawio` — Évaluation et validation
6. `06_automation_ci_upload.drawio` — Automation CI upload
7. `07_upload_auth_flow.drawio` — Flux d'authentification upload

Note : Pour intégrer les diagrammes dans le PDF, exporter les fichiers .drawio en PNG (File → Export as → PNG, 150 DPI) dans le dossier `docs/diagrams/{presentation,technical}/expo`

B. Commandes d'Exécution

Listing 6.1 – Pipeline complet

```
# Activer l'environnement
.venv\Scripts\activate
```

```
# Pipeline complet avec delivery (serie specifique)
$env:STM32CUBE_CONFIG = "shared/config/config_f4.json"
python -m pipeline.run_full_workflow --export-st-ready --continue-
    on-error

# Pipeline sans chunking (ST platform gere le chunking)
python -m pipeline.run_full_workflow --export-st-ready --skip-
    chunking

# Pipeline sans ingestion (reutilise les donnees existantes)
python -m pipeline.run_full_workflow --export-st-ready --skip-
    ingestion
```

Listing 6.2 – Enrichissement Alfred

```
# Alfred Vision - images d'issues
python pipeline_Automation/alfred/enrich_json_images_with_alfred.py
    \
  --input <fichier_issues_with_images.json> --inplace

# Alfred Vision - figures PDF
python pipeline_Automation/alfred/enrich_pdf_figures_with_alfred.py

# Patch descriptions PDF dans les delivery files
python pipeline_Automation/patch_pdf_descriptions_in_delivery.py

# Fix JSON corrompus (control characters)
python pipeline_Automation/fix_corrupted_json_files.py
python pipeline_Automation/fix_corrupted_json_deep.py
```

Listing 6.3 – Upload vers KB #793

```
# Upload issues Alfred-enrichies (13 series)
python pipeline_Automation/upload/upload_alfred_issues_all_series.
    py

# Upload files PDF-enrichies (13 series)
python pipeline_Automation/upload/
    upload_pdf_enriched_files_all_series.py

# Upload drivers sous-repos (8 nouvelles series)
```

```
.\pipeline_Automation\upload\Upload_All_Drivers.ps1

# Upload manuel (un fichier)
python pipeline_Automation/upload/Add_Data_Source_Files.py \
  --kb 793 --operation add --datasource-id <ID> \
  --remote-user "abir.gharbi@st.com" \
  --processor JSON --processor-params "base64:<b64>" \
  --json-split-size 1000 --files <fichier.json>
```

Listing 6.4 – Validation et evaluation

```
# Validation des schemas
python -m pipeline.evaluation.validate_schemas

# Stats issues
python -m pipeline.evaluation.test_stats_issues_v2

# Stats files
python -m pipeline.evaluation.test_stats_files_v2

# Recherche TF-IDF (test local)
python -m pipeline.evaluation.test_tfidf_search_issues_v2
```

Listing 6.5 – Ajout d'une nouvelle serie (ex : G4)

```
# 1. Config deja creee : shared/config/config_g4.json
# 2. Lancer le pipeline
$env:STM32CUBE_CONFIG = "shared/config/config_g4.json"
python -m pipeline.run_full_workflow --export-st-ready \
  --skip-chunking --continue-on-error
# 3. Creer datasources sur KB #793 (interface web)
# 4. Mettre a jour config_all_series.json avec les IDs
# 5. Uploader
python pipeline_Automation/upload/upload_alfred_issues_all_series.
   py
python pipeline_Automation/upload/
   upload_pdf_enriched_files_all_series.py
```

C. Structure JSON — Resolver Case (Extrait)

```
{
  "resolver_cases": [
    {
      "issue_number": 42,
      "repo": "stm32cubef4",
      "label": "SPI DMA TransmitReceive fails...",
      "resolution_status": "merged_fix",
      "link_confidence": "high",
      "primary_pr_number": 87,
      "primary_changed_file": "stm32f4xx_hal_spi.c",
      "resolver_card_text": "Problem: ...",
      "impacted_modules": ["SPI", "DMA"],
      "impacted_boards": ["NUCLEO-F446RE"]
    }
  ]
}
```


D. Exemple JSON — Issue Structurée (Extrait)


```
{
  "issue_number": 42,
  "repo": "stm32cubeh7",
  "title": "HAL_SPI_TransmitReceive_DMA fails on STM32H743",
  "layer": "HAL",
  "component": "SPI",
  "board": "NUCLEO-H743ZI",
  "issue_kind": "bug_report",
  "severity": "high",
  "evidence_strength": "high",
  "st_ready_text": "Symptom: hard fault after 2nd SPI DMA transfer
     ... Root cause: D-cache not invalidated... Fix:
     SCB_InvalidateDCache_by_Addr().",
  "source_url": "https://github.com/STMicroelectronics/STM32CubeH7/
     issues/42"
}
```

E. Exemple JSON — Fichier Documenté (Extrait)

```
{
  "path": "Src/stm32h7xx_hal_spi.c",
  "repo": "stm32h7xx-hal-driver",
  "file_type": "source_code",
  "original_size": 89247,
  "clean_size": 80322,
  "is_valid": true,
  "url": "https://github.com/STMicroelectronics/stm32h7xx-hal-
     driver/blob/master/Src/stm32h7xx_hal_spi.c",
  "st_ready_text": "HAL_StatusTypeDef HAL_SPI_Init(...) {...}
     HAL_SPI_TransmitReceive_DMA(...) {...}"
}
```

F. Exemple JSON — Diagnostic Card (Extrait)

```
{
  "diagnostic_id": "DIAG-H7-042",
  "repo": "stm32cubeh7",
  "symptom": "Hard fault after 2nd SPI DMA transfer (circular mode,
      H743)",
  "root_cause": "D-cache not invalidated before DMA reception (
     stale read).",
  "resolution": "SCB_InvalidateDCache_by_Addr() before reading RX
     buffer, or non-cacheable SRAM (MPU).",
  "affected_versions": ["CubeH7 v1.9.0", "CubeH7 v1.10.0"],
  "fixed_version": "CubeH7 v1.11.0",
  "query_aliases": [
    "SPI DMA hard fault STM32H7",
    "cache coherency SPI circular mode",
    "D-cache DMA corruption H743"
  ],
  "evidence_strength": "high"
}
```

G. Exemple de Fichier de Configuration (Extrait)

Chaque série dispose d'un fichier `config_XX.json` listant les dépôts à ingérer et les paramètres d'export. Extrait représentatif :

```
{
  "series": "STM32CubeH7",
  "primary_repo": "STM32CubeH7",
  "repos": [
    "stm32h7xx-hal-driver",
    "cmsis_device_h7",
    "stm32h743i-eval-bsp",
    "stm32h745i-disco-bsp"
  ],
  "kb_id": 793,
  "export": { "st_ready": true, "skip_chunking": true }
}
```

Le fichier `config_all_series.json` associe chaque série aux identifiants de data-sources KB (issues, files, resolver, diagnostic) utilisés lors de l'upload.

H. Extrait d'Automatisation — Workflow CI/CD

Le workflow GitHub Actions déclenche le pipeline (cloud) puis l'upload (self-hosted). Extrait représentatif :

Listing 6.6 – Workflow bi-mensuel (extrait YAML)

```
name: STM32Cube KB Update
on:
  schedule:
    - cron: '0 3 1,15 * *'   # 1er et 15 du mois, 03h UTC
  workflow_dispatch:
    inputs:
      series: { description: 'Serie a traiter', required: false }
jobs:
  pipeline:                  # Job 1 - cloud
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python -m pipeline.run_full_workflow
             --export-st-ready --skip-chunking --continue-on-error
```

```
  upload:                  # Job 2 - self-hosted (gate + Delete+Add)
    needs: pipeline
    runs-on: self-hosted
    steps:
      - run: python -m pipeline.evaluation.validate_schemas
      - run: python pipeline_Automation/upload/
          upload_alfred_issues_all_series.py
```

I. Glossaire

RAG (Retrieval-Augmented Generation)
Paradigme combinant recherche documentaire et génération par LLM : les documents pertinents sont récupérés puis fournis au modèle comme contexte.

Chunking `PARENT_CHILD`
Découpage hiérarchique à deux niveaux : les chunks enfants (petits) servent de cibles de recherche, les chunks parents (grands) sont restitués comme contexte (cf. Figure 3.3).

Chunk enfant / parent
Enfant : fragment de 1000 caractères indexé (vectorisé). Parent : fragment de 3000 caractères englobant, restitué au LLM lorsqu'un enfant correspond.

Embedding
Représentation vectorielle d'un texte dans un espace où la proximité reflète la proximité sémantique.

AdaV2 (`text-embedding-ada-002`)
Modèle d'embedding (1536 dimensions) utilisé par la plateforme pour l'indexation sémantique de la KB #793.

Recherche hybride
Combinaison d'une recherche sémantique (embeddings) et full-text (correspondance exacte), fusionnées par union.

Seuil sémantique
Score minimal de similarité vectorielle pour qu'un document soit retourné (0.55 en configuration courante).

TF-IDF
Pondération de termes par leur spécificité dans le corpus, utilisée pour la similarité inter-issues (cf. Section 5.1.2).

Similarité cosinus
Mesure de l'angle entre deux vecteurs textuels (cf. Section 5.1.2).

Similarité de Jaccard
Proportion de vocabulaire partagé entre deux ensembles de tokens (cf. Section 5.1.2).

Alfred Vision
Service LLM interne ST de description d'images, convertissant captures d'écran et figures PDF en descriptions textuelles indexables.

NovaX
Tool de fallback restrictif (Bugzilla + STCommunity) appelé uniquement si la KB ne couvre pas la question, pour protéger le routage KB-first.

KB (Knowledge Base)
Base de connaissances #793 sur la plateforme ST ChatGPT, alimentée par les artefacts du pipeline.

Datasource
Source de données au sein de la KB, par série et par type d'artefact.

Resolver Case
Artefact traçant la chaîne Issue → PR → Commit avec preuve de résolution.

Diagnostic Card
Artefact structuré (Symptôme → Root Cause → Fix → Prévention) avec query aliases.

Query alias
Reformulation alternative d'un problème, ajoutée pour améliorer le rappel du retrieval.

Gate de validation
Contrôle JSON Schema bloquant l'upload si un artefact est malformé.

Quality Index (QI)
Métrique end-to-end de qualité des réponses du chatbot (cf. Section 5.1.6).

Confiance contextuelle
Proportion de descriptions Alfred dont l'ancrage au contexte dépasse le seuil d'alerte (cf. Section 5.1.4).

`jobBatchSize` / `embeddingsBatchSize`
Tailles de lots plateforme (60 documents par job d'indexation, 20 embeddings par appel API), cf. Section 3.7.4.

J. Liste des Acronymes

Acronyme	Signification
RAG	Retrieval-Augmented Generation

KB	Knowledge Base
LLM	Large Language Model
NLP	Natural Language Processing
TF-IDF	Term Frequency – Inverse Document Frequency
BM25	Best Matching 25 (fonction de ranking full-text)
NLI	Natural Language Inference
OCR	Optical Character Recognition
HAL	Hardware Abstraction Layer
LL	Low-Layer (drivers)
BSP	Board Support Package
CMSIS	Cortex Microcontroller Software Interface Standard
MCU	Microcontroller Unit
DMA	Direct Memory Access
SPI	Serial Peripheral Interface
IRQ	Interrupt Request
PR	Pull Request
API	Application Programming Interface
JSON	JavaScript Object Notation
PDF	Portable Document Format
CI/CD	Continuous Integration / Continuous Deployment
QI	Quality Index
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
DPMO	Defects Per Million Opportunities
CRISP-DM	Cross-Industry Standard Process for Data Mining

K. Vue d'Ensemble des Notions Techniques

Cette table garantit que chaque notion technique dispose d'au moins un support explicatif (définition, figure, exemple ou entrée de glossaire) :

Notion	**Type de support**	**Localisation**
Chunking `PARENT_CHILD`	Section + figure + glossaire	Sec. 3.7.2, Fig. 3.3
Embeddings AdaV2	Section + glossaire + réf.	Sec. 3.7.3, [17]
Tailles de lots (batch)	Section + glossaire	Sec. 3.7.4

Recherche hybride	Section + glossaire	Sec. 4.11.2
Alfred Vision	Section + figure + glossaire	Fig. 4.8, Sec. 5.4
NovaX fallback	Section + glossaire	Sec. 5.6
Upload KB (workflow)	Annexe H + commandes B	Annexes B, H
Gates JSON Schema	Section + glossaire	Pattern « Gate de Qualité »
Resolver Cases	Section + exemple JSON	Sec. Delivery, Annexe C
Diagnostic Cards	Section + exemple JSON	Sec. Delivery, Annexe F
CI/CD automation	Chapitre + figure + annexe	Fig. 6.1, Annexe H
TF-IDF / cosinus / Jaccard	Définitions + glossaire	Sec. 5.1.2
Quality Index	Définition + évolution	Sec. 5.1.6, 5.5